Politechnika Warszawska

WYDZIAŁ ELEKTRYCZNY



Instytut Sterowania i Elektroniki Przemysłowej

Praca dyplomowa magisterska

na kierunku Informatyka Stosowana

w specjalności Inżynieria Oprogramowania

Analiza Wydajności i Bezpieczeństwa Protokołu MCP w Integracji Systemów Agentowych z Wielkimi Zbiorami Danych

Karol Sygiet

numer albumu 340801

promotor

dr inż. Włodzimierz Dąbrowski, prof. uczelni

WARSZAWA 2026

Analiza Wydajności i Bezpieczeństwa Protokołu MCP w Integracji Systemów Agentowych z Wielkimi Zbiorami Danych

Streszczenie

Niniejsza praca magisterska poświęcona jest analizie wydajności, skalowalności i bezpieczeństwa otwartego standardu Model Context Protocol w procesie integrowania autonomicznych systemów agentowych z wielkoskalowymi relacyjnymi składami danych. W pracy zidentyfikowano ograniczenia klasycznych architektur RAG oraz bezpośrednich integracji bazodanowych, polegające na degradacji okna kontekstowego modelu językowego przez obszerne zrzuty DDL, wysokim narzucie serializacji JSON-RPC oraz podatności na blokowanie asynchronicznej pętli zdarzeń i wstrzykiwanie promptu. W ramach wkładu własnego zaprojektowano i zaimplementowano referencyjną bramę pośredniczącą w środowisku Python 3.12, wyposażoną w silnik progresywnego odkrywania struktur schematu w czasie rzeczywistym z domknięciem grafowym relacji, algorytm wielokolumnowej paginacji kursorowej keyset z weryfikacją indeksów B-drzewa i izolacją snapshotową, moduł transferu Out-of-Band z modelem progu opłacalności w funkcji opóźnienia sieciowego RTT, asynchroniczne strumieniowanie SSE powiązane z Apache Kafka oraz zaporę SQL Sanitizer opartą na wieloprocesowej analizie drzewa składniowego AST z enkapsulacją rekordów. Ewaluacja eksperymentalna na próbie $\mathrm{N} = 500$ zapytań Text-to-SQL w syntetycznej hurtowni o 1000 relacjach wykazała redukcję konsumpcji tokenów DDL o ponad 98% (z $\sim$154 tys. do 2219 tokenów) przy czułości selekcji Table-Recall@5 na poziomie 54,5% (rosnącej do 79,3% dla $\mathrm{k} = 20$), skrócenie mediany czasu TTFT o 97,5% (z 1382,7 ms do 34,7 ms), stałe opóźnienie paginacji keyset $\mathcal{O}(\log \mathrm{N} + \mathrm{K})$ na zbiorze 1 mln rekordów do głębokości 95% tabeli (0,43 ms przy offsecie 950 000 wobec 429 ms dla OFFSET/LIMIT), przepustowość 1173 RPS z opóźnieniem pętli < 1 ms (0,53 ms) pod obciążeniem 500 współbieżnych procesów agentowych oraz wysoką skuteczność mitygacji ataków strukturalnych, w pełni potwierdzając postawioną hipotezę badawczą.

Słowa kluczowe: Model Context Protocol, systemy agentowe, bazy danych, duże modele językowe, paginacja kursorowa, pgvector, bezpieczeństwo AI, Text-to-SQL

Performance and Security Analysis of the Model Context Protocol in Integrating Agent Systems with Big Data

Abstract

This master's thesis investigates the performance, scalability, and security of the open Model Context Protocol specification when integrating autonomous agent systems with enterprise-scale relational data stores. The research addresses key architectural bottlenecks in standard RAG pipelines and naive database tool integrations, namely context window exhaustion caused by monolithic DDL dumps, JSON-RPC serialization overhead, event-loop starvation during intensive query processing, and vulnerability to indirect prompt injection. As the primary engineering contribution, an enterprise-grade middleware gateway was designed and implemented in Python 3.12. The architecture incorporates Just-in-Time Schema Discovery with foreign-key graph transitive closure, multi-column keyset cursor pagination with catalog B-tree index guards and transaction snapshot isolation, an Out-of-Band binary transfer mechanism parameterized by network round-trip time, asynchronous SSE streaming integrated with Apache Kafka, and a multi-process AST-based SQL Sanitizer with structural XML record encapsulation. Empirical evaluation on a benchmark of $\mathrm{N} = 500$ Text-to-SQL queries against a 1000-table synthetic warehouse demonstrates a DDL token reduction exceeding 98% (from $\sim$154k to 2219 tokens) with Table-Recall@5 of 54.5% (reaching 79.3% at $\mathrm{k} = 20$), a 97.5% reduction in median Time-to-First-Token (from 1382.7 ms to 34.7 ms), constant keyset cursor pagination latency $\mathcal{O}(\log \mathrm{N} + \mathrm{K})$ on a 1-million-row dataset up to 95% table depth (0.43 ms at offset 950,000 vs 429 ms for OFFSET/LIMIT), throughput of 1173 RPS with event-loop lag below 1 ms (0.53 ms) under 500 concurrent agent clients, and complete mitigation of structural injection threats, fully substantiating the research hypothesis.

Keywords: Model Context Protocol, agentic systems, databases, large language models, cursor pagination, pgvector, AI security, Text-to-SQL

Spis treści

Rozdział 1

Wstęp

1.1 Kontekst badawczy: Systemy Agentic AI i problem izolacji modelu

Wielkie generatywne modele językowe (ang. *Large Language Models*, LLM), oparte na architekturze Transformer [44], zapoczątkowały fundamentalne zmiany w przetwarzaniu języka naturalnego. Ich bezpośrednie wdrożenie w złożonych procesach inżynierskich ujawniło jednak istotną barierę: zjawisko izolacji modelu (ang. *model isolation*). Parametryczna baza wiedzy, zakodowana w wagach sieci neuronowej podczas treningu, pozostaje statyczna i podatna na dezaktualizację. Ponadto model działający w klasycznym trybie pojedynczego zapytania i odpowiedzi, nie posiada zdolności sprawczej – nie może deterministycznie wchodzić w interakcję ze środowiskiem zewnętrznym, odpytywać baz danych ani modyfikować stanu systemów informatycznych.

Odpowiedzią na tę barierę stał się paradygmat korzystania z narzędzi [38, 42] oraz wyłonienie autonomicznych systemów agentowych. W podejściu tym model językowy pełni rolę centralnego komponentu wnioskującego, zdolnego do wieloetapowego planowania, dekompozycji problemów oraz sterowania zewnętrznymi procedurami wykonawczymi w iteracyjnych pętlach decyzyjnych, np. w ramach paradygmatu ReAct (*Reasoning and Acting*) [48].

Wczesny etap rozwoju ekosystemu agentowego (lata 2023–2024) cechowała jednak głęboka fragmentacja technologiczna. Integracje bazowały na doraźnych rozwiązaniach łączących oraz bibliotekach orkiestracyjnych (m.in. LangChain [7]), w których każdy dostawca modeli i narzędzi definiował odmienne, wzajemnie niekompatybilne interfejsy funkcyjne. Skutkowało to kombinatoryczną złożonością powiązań międzysystemowych rzędu $\mathcal{O}(\mathrm{M} \times \mathrm{N})$ pomiędzy M modelami a N narzędziami.

Przełomem standaryzacyjnym stało się ogłoszenie pod koniec 2024 roku otwartej specyfikacji *Model Context Protocol* (MCP) [1], przekazanej następnie pod egidę Linux Foundation w ramach inicjatywy Agentic AI Foundation. Protokół MCP zdefiniował uniwersalny kontrakt komunikacyjny oparty na standardzie JSON-RPC 2.0 [**jsonrpc2010**], redukując złożoność integracyjną do modularnej postaci $\mathcal{O}(\mathrm{M} + \mathrm{N})$ poprzez ścisłą separację roli klienta (hosta modelu) i serwera kontekstu.

1.2 Złożoność relacyjna jako bariera integracji agentowej

W przeciwieństwie do potocznego rozumienia pojęcia wielkich zbiorów danych (ang. *Big Data*), w relacyjnych środowiskach bazodanowych głównym wyzwaniem badawczym nie jest fizyczny, wieloterabajtowy wolumen krotek na nośnikach nieulotnych. Wolumeny rzędu miliona wierszy są efektywnie przetwarzane w pamięci pojedynczego węzła obliczeniowego. Rzeczywistą barierę inżynieryjną stanowi **głęboka asymetria pomiędzy eksplozją kombinatoryczną grafu relacji schematu bazodanowego a fizycznymi i logicznymi ograniczeniami mechanizmu uwagi modeli językowych oraz narzutem serializacji i tokenizacji komunikatów tekstowych JSON-RPC 2.0**.

W przemysłowych składach danych schemat relacyjny obejmuje setki lub tysiące tabel powiązanych gęstą siecią kluczy obcych o topologii sieci bezskalowej. W badanym w niniejszej pracy środowisku schemat korporacyjny liczy 1000 tabel i 1174 więzy integralności referencyjnej. Bezpośrednie zastosowanie protokołu MCP w takich warunkach ujawnia cztery fundamentalne wąskie gardła:

1. **Fizyczne i logiczne ograniczenia okna uwagi przy wstrzykiwaniu pełnego schematu:** Naiwne wstrzyknięcie pełnego zrzutu kodu DDL dla 1000 tabel wymaga zaalokowania ponad 150 tysięcy tokenów. Mimo deklarowania przez dostawców komercyjnych okien kontekstowych rzędu 10^6 tokenów, podejście to prowadzi do zapaści uwagi modelu w centralnych segmentach sekwencji (zjawisko *Lost in the Middle*, Liu i in. [26]) oraz zanieczyszczenia kontekstu nieistotnymi relacjami (*Context Pollution*, Levy i in. [**levy2024same**]). Co więcej, kwadratowa złożoność obliczeniowa fazy prefill w mechanizmie Self-Attention $\mathcal{O}(\mathrm{L}^2)$ wywołuje wielosekundowe opóźnienia Time-to-First-Token, a alokacja tensora kluczy i wartości bufora KV-Cache przekracza 7 GB pamięci VRAM na pojedynczą sesję modelu 7B, paraliżując przepustowość systemu serwującego;

2. **Niewystarczalność klasycznego podejścia RAG dla struktur relacyjnych:** Dotychczasowe metody generowania wspomaganego wyszukiwaniem (ang. *Retrieval-Augmented Generation*) [25], oparte wyłącznie na gęstych zanurzeniach wektorowych i wyszukiwaniu najbliższych sąsiadów (ang. *Dense-only k-NN*), dopasowują tabele na podstawie podobieństwa semantycznego pojęć biznesowych. Podejście to całkowicie ignoruje strukturę powiązań bazy danych. Gdy zapytanie wymaga połączenia dwóch pojęć odległych w grafie relacji, model wektorowy typuje wyłącznie tabele krańcowe, gubiąc tabele asocjacyjne mostkujące relacje wielokrotne $\mathrm{N} : \mathrm{M}$. W konsekwencji model językowy otrzymuje niespójny wycinek schematu, co skutkuje halucynowaniem nieistniejących kluczy obcych bądź generowaniem niepoprawnych iloczynów kartezjańskich pozbawionych klauzul złączenia `JOIN`;

3. **Narzut serializacji tekstowej i brak semantyki transakcyjnej w JSON-RPC 2.0:** Format JSON-RPC 2.0 oparty na kodowaniu UTF-8 [4] generuje wysoki koszt konwersji i tokenizacji. Przy transmisji struktur binarnych lub masowych wyników tabelarycznych kodowanie Base64 wprowadza 33,33% narzutu objętościowego oraz wysoką entropię, która fragmentuje słownik

tokenizera BPE. Jednocześnie bezstanowa natura wywołań `tools/call` w połączeniu ze stochastycznością pętli wnioskowania stwarza ryzyko niespójności danych z powodu braku mechanizmów automatycznego wycofywania zmian;

4. **Wektory zagrożeń perymetru bazodanowego i zawodność filtrów promptowych:** Bezpośrednia integracja agenta z silnikiem bazy danych rodzi podatność na ataki pośredniego wstrzykiwania instrukcji [14, 37] oraz problem zdezorientowanego zastępcy [16]. Złośliwe dyrektywy ukryte w polach tekstowych rekordów mogą przejąć kontrolę nad agentem i doprowadzić do wygenerowania destrukcyjnych zapytań DDL/DML. Filtry oparte na instrukcjach języka naturalnego w promptach systemowych charakteryzują się niemal stuprocentową zawodnością, co wymusza wdrożenie deterministycznego perymetru bezpieczeństwa opartego na statycznej analizie drzew składniowych AST.

1.3 Pytania badawcze

W odpowiedzi na zidentyfikowane wąskie gardła sformułowano trzy pytania badawcze, wyznaczające osie analityczne i eksperymentalne niniejszej pracy magisterskiej:

PB1 (Kompletność selekcji schematu bazy danych):
W jakim stopniu uzupełnienie wyszukiwania semantycznego o algorytm domknięcia grafowego (FK Graph Pruning) poprawia kompletność doboru tabel w złożonych schematach relacyjnych?

Uzasadnienie: Pytanie weryfikuje, czy deterministyczne dołączanie brakujących tabel mostkujących ($\mathrm{N} : \mathrm{M}$) eliminuje błędy złączeń w zapytaniach generowanych przez model językowy, nie powodując przy tym przepełnienia okna kontekstowego.

PB2 (Próg opłacalności transferu pozapasmowego):
Przy jakiej wielkości przesyłanych danych narzut kodowania Base64 w protokole JSON-RPC uzasadnia przeniesienie transferu do dedykowanego kanału pozapasmowego (Out-of-Band)?

Uzasadnienie: Pytanie zmierza do empirycznego wyznaczenia punktu opłacalności, w którym zysk ze skróconego czasu transmisji przewyższa stały koszt inicjalizacji pamięci współdzielonej lub strumienia HTTP.

PB3 (Współbieżność serwera a ochrona zapytań):
W jakim stopniu oddelegowanie walidacji składniowej SQL do procesów potomnych pozwala utrzymać wydajność serwera asynchronicznego pod dużym obciążeniem przy zachowaniu pełnej ochrony przed atakami?

Uzasadnienie: Pytanie ocenia, czy zastosowanie wieloprocesowej analizy drzew AST eliminuje blokowanie pętli zdarzeń serwera (Event Loop Starvation), zapewniając jednocześnie skuteczną filtrację złośliwych zapytań i wstrzyknięć do promptu.

1.4 Cel i zakres pracy

Głównym celem niniejszej pracy magisterskiej jest **zaprojektowanie, implementacja oraz eksperymentalna ewaluacja bramy pośredniczącej w standardzie Model Context Protocol, optymalizującej wymianę danych oraz bezpieczeństwo interakcji systemów agentowych ze złożonymi relacyjnymi bazami danych**.

W celu realizacji tak sformułowanego celu zdefiniowano następujący zakres prac:

1. **Część projektowo-implementacyjna:**
   - opracowanie modularnej architektury bramy MCP integrującej asynchroniczną pętlę zdarzeń z silnikiem PostgreSQL i brokerem Apache Kafka,
   - implementacja mechanizmu progresywnego odkrywania schematu łączącego indeks wektorowy z algorytmem domknięcia grafowego,
   - budowa modułu adaptacyjnej paginacji kursorowej z dynamiczną kontrolą budżetu tokenów,
   - implementacja kanału transferu binarnego poza pasmem głównym oraz wieloprocesowej zapory zapytań SQL opartej na analizie drzew AST.
2. **Część analityczno-badawcza:**
   - analityczne wyznaczenie progu opłacalności transferu Out-of-Band w zależności od wolumenu danych,
   - eksperymentalna ocena skuteczności doboru tabel w złożonych schematach relacyjnych w porównaniu ze standardowym podejściem RAG,
   - profilowanie wydajności i opóźnień pętli zdarzeń serwera pod narastającym obciążeniem wieloagentowym,
   - weryfikacja odporności opracowanego perymetru bezpieczeństwa na ataki wstrzykiwania poleceń oraz ewaluacja statystyczna uzyskanych wyników.

1.5 Hipotezy badawcze

W odniesieniu do zdefiniowanych pytań badawczych postawiono nadrzędną hipotezę pracy:

> *Zastosowanie dedykowanej bramy pośredniczącej w standardzie MCP, integrującej semantyczno-grafową selekcję schematu, adaptacyjną paginację kursorową oraz wieloprocesową walidację zapytań, pozwala na istotne ograniczenie narzutu komunikacyjnego i ochronę bazy danych bez degradacji responsywności systemu agentowego pod obciążeniem.*

W celu przeprowadzenia rygorystycznej weryfikacji empirycznej hipotezę główną zdekomponowano na trzy hipotezy cząstkowe podlegające falsyfikacji statystycznej:

H1: Uzupełnienie hybrydowego wyszukiwania schematu o algorytm domknięcia grafowego zapewnia statystycznie istotny wzrost czułości selekcji tabel (*Recall@5*) w relacji do metod klasycznych (BM25 oraz gęstego wyszukiwania k-NN).

H2: Istnieje wyznaczalny empirycznie próg wolumenu danych S^*, powyżej którego transfer binarny w kanale pozapasmowym cechuje się statystycznie istotnie niższym opóźnieniem niż standardowa serializacja JSON-RPC z kodowaniem Base64.

H3: Oddelegowanie walidacji drzew składniowych AST do wieloprocesowej puli wykonawczej eliminuje zjawisko blokowania asynchronicznej pętli zdarzeń pod obciążeniem wieloagentowym, zapewniając jednocześnie pełną redukcję podatności na wstrzykiwanie złośliwych zapytań.

1.6 Metodologia badań

W niniejszej pracy magisterskiej zastosowano metodologię badań konstrukcyjnych (*Design Science Research*) [**peffers2007dsr**], powszechnie stosowaną w informatyce stosowanej do rozwiązywania problemów inżynierskich. Podejście to opiera się na iteracyjnym wytworzeniu innowacyjnego artefaktu programistycznego, a następnie poddaniu go rygorystycznej ewaluacji empirycznej.

W ramach przyjętego procesu badawczego zrealizowano artefakt w postaci modularnej bramy pośredniczącej w standardzie Model Context Protocol, integrującej modele językowe ze złożonym schematem PostgreSQL/pgvector oraz brokerem Apache Kafka. Ewaluację artefaktu przeprowadzono w skonteneryzowanym środowisku testowym na reprezentatywnych zbiorach danych syntetycznych, poddając go stresstestom pod kątem kompletności odzyskiwania relacji, skalowalności asynchronicznej pętli zdarzeń pod obciążeniem wieloagentowym ($\mathrm{M} \leq 500$) oraz odporności na ataki wstrzykiwania instrukcji ($\mathrm{N} = 1000$ wektorów). Wnioskowanie o skuteczności zaproponowanych rozwiązań oparto na aparacie nieparametrycznych testów istotności statystycznej Wilcoxona.

1.7 Układ pracy

Układ niniejszej pracy magisterskiej odzwierciedla logiczny przebieg procesu badawczo-konstrukcyjnego i składa się z siedmiu rozdziałów:

- **Rozdział 2 – Teoretyczne podstawy integracji modeli językowych z danymi:** omawia ewolucję paradygmatów integracji systemów LLM z otoczeniem zewnętrznym oraz specyfikację protokołu Model Context Protocol. Formalizuje fizyczne ograniczenia okna uwagi modeli Transformer oraz zagadnienia spójności danych relacyjnych;
- **Rozdział 3 – Analiza porównawcza protokołów komunikacyjnych:** przedstawia dekonstrukcję formatu JSON-RPC 2.0 i analityczne zestawienie protokołów REST, gRPC, MCP oraz standardów komunikacji wieloagentowej. Zawiera również taksonomię wektorów zagrożeń bezpieczeństwa w architekturach agentowych;

- **Rozdział 4 – Projekt systemu integracji MCP z hurtownią danych:** zawiera formalny projekt logiczny autorskiej bramy pośredniczącej. Przedstawia matematyczne modele adaptacyjnej paginacji kursorowej z algorytmem Welforda, silnik progresywnego odkrywania schematu z algorytmem domknięcia grafowego, analityczny model progu opłacalności transferu pozapasmowego S^* oraz architekturę ochrony pętli zdarzeń;
- **Rozdział 5 – Implementacja i środowisko badawcze:** opisuje architekturę fizyczną i implementację poszczególnych modułów serwera w środowisku Python 3.12, architekturę kontenerowego testbedu Docker Compose oraz zautomatyzowaną aparaturę pomiarową służącą do rejestracji opóźnień, tokenów i metryk współbieżności;
- **Rozdział 6 – Wyniki badań i ewaluacja eksperymentalna:** przedstawia szczegółową analizę ilościową i statystyczną przeprowadzonych benchmarków. Obejmuje ocenę jakości doboru tabel, weryfikację empirycznego punktu opłacalności transferu binarnego, testy skalowalności pętli zdarzeń, analizę skuteczności zapory bezpieczeństwa oraz formalną weryfikację hipotez badawczych z użyciem testów Wilcoxona;
- **Rozdział 7 00 Podsumowanie i wnioski:** dokonuje syntezy osiągniętych rezultatów, zestawia macierz kompromisów architektonicznych opracowanego rozwiązania, rzetelnie omawia jego ograniczenia oraz nakreśla perspektywy standaryzacji protokołu MCP w systemach wielkoskalowych.

Rozdział 2

Teoretyczne Podstawy Integracji Modeli Językowych z Danymi

Generatywne wielkie modele językowe oparte na architekturze Transformer [44] generują sekwencje tokenów w reżimie autoregresyjnym, modelując rozkład prawdopodobieństwa kolejnego słowa na podstawie kontekstu poprzedzającego. Wiedza zakodowana w statycznych wagach sieci neuronowej odzwierciedla jednak wyłącznie stan korpusu treningowego w momencie jego zamrożenia. Co więcej, stochastyczny charakter próbkowania tokenów sprzyja powstawaniu halucynacji [31] – generowaniu sekwencji poprawnych gramatycznie, lecz fałszywych faktograficznie. W zastosowaniach wymagających ścisłego determinizmu, takich jak obsługa relacyjnych baz danych, transakcji finansowych czy audytowalnych bilansów, model odizolowany od zewnętrznych źródeł prawdy nie może stanowić wiarygodnego nośnika informacji.

Konieczność zapewnienia determinizmu wymusiła ewolucję paradygmatów integracji: od modyfikacji wag sieci (ang. *fine-tuning*), poprzez generowanie wspomagane wyszukiwaniem [25], aż po paradygmat korzystania z narzędzi [42]. W ramach tego ostatniego nurtu model zyskuje zdolność autonomicznego wywoływania zewnętrznych interfejsów programistycznych, co doprowadziło do sformalizowania otwartych standardów komunikacyjnych, w szczególności protokołu Model Context Protocol [1].

Niniejszy rozdział przedstawia teoretyczne i architektoniczne podstawy integracji modeli językowych ze strukturami danych. W podrozdziale **??** omówiono ewolucję mechanizmów rozszerzania kontekstu, kompromisy metodyczne systemów RAG, przełom koncepcyjny projektów Toolformer i Gorilla oraz paradygmaty redukcji przestrzeni relacyjnej. Podrozdział **??** analizuje problem fragmentacji ekosystemu narzędziowego, polemikę z analogią do protokołu Language Server Protocol oraz mechanizm redukcji złożoności integracyjnej. W podrozdziale **??** przedstawiono formalną specyfikację protokołu MCP: role architektoniczne, cykl życia sesji, warstwy transportowe oraz prymitywy semantyczne. Z kolei podrozdział **??** poświęcono specyfice środowisk danych dużej skali, analizując fizyczne bariery uwagi i narzuty pamięci podręcznej KV-Cache w architekturze Transformer,

wyzwania spójności transakcyjnej w komunikacji bezstanowej oraz dynamikę stochastycznych transakcji agentowych.

2.1 Ewolucja paradygmatów integracji: od odizolowanych LLM do Tool Use

2.1.1 Od pamięci parametrycznej do paradygmatu Tool Use

Pasywna pamięć parametryczna modeli LLM (wagi sieci zdeponowane w procesie pre-treningu) wykazuje fundamentalną barierę temporalną oraz brak determinizmu algebraicznego, co wyklucza bezpośrednie wnioskowanie na transakcyjnych danych korporacyjnych. Historyczne podejścia adaptacji – dostrajanie wag (*fine-tuning*) obarczone zjawiskiem katastrofalnego zapominania [22] oraz generowanie wspomagane wyszukiwaniem (RAG) [25] – zawodzą w kontakcie z relacyjnymi składami danych. RAG spłaszcza relacje logiczne bazy do postaci bezpostaciowego tekstu, a wyszukiwanie gęste po wektorach nie interpretuje predykatów selekcji ani złączeń algebraicznych. Szczegółowe zestawienie kompromisów architektonicznych tych paradygmatów w zestawieniu z paradygmatem korzystania z narzędzi (*Tool Use*) [40] przedstawiono w tabeli **??**.

Tabela 1. Syntetyczne porównanie paradygmatów integracji modeli językowych z zasobami danych

Paradygmat	Determinizm danych	Świeżość informacji	Koszt aktualizacji bazy	Narzut na okno kontekstowe	Zdolność do efektów ubocznych
Fine-tuning	Niski (aproksymacja probabilistyczna)	Bardzo niska (wymaga cyklu trenowania)	Bardzo wysoki (wymaga klastra GPU)	Brak narzutu ($\mathcal{O}(1)$)	Brak (model pasywny)
RAG (indeksy wektorowe)	Średni (wrażliwy na chunking i k-NN)	Umiarkowana (opóźnienia potoków ETL)	Umiarkowany (rekalkulacja wektorów)	Średni ($\mathcal{O}(\mathrm{k} \cdot \mathrm{L}_{\mathrm{chunk}})$)	Brak (tryb read-only)
Tool Use (np. bazy SQL)	Wysoki (determinizm silnika bazy danych)	Czas rzeczywisty (stan transakcyjny)	Minimalny (standardowy zapis w bazie)	Zmienny ($\mathcal{O}(\mathrm{L}_{\mathrm{schema}} + \mathrm{L}_{\mathrm{result}})$)	Pełna (kontrolowane zmiany stanu)

Przełom w integracji modeli ze środowiskiem zewnętrznym przyniósł rozwój paradygmatu korzystania z narzędzi. Schick i in. w modelu *Toolformer* [42] wykazali, że model może uczyć się samonadzorowanego wywoływania procedur API bezpośrednio w trakcie autoregresyjnego dekodowania, włączając operacje narzędziowe na podstawie minimalizacji straty entropii krzyżowej dla kolejnych tokenów. Z kolei Patil i in. w architekturze *Gorilla* [38] (zbiór *APIBench*) rozwiązali problem halucynowania sygnatur programistycznych poprzez trening ukierunkowany na sformalizowaną dokumentację interfejsów, redukując błędy argumentów o ponad 80%. Kanonem orkiestracji wieloetapowego wnioskowania stała się pętla *ReAct* [48], wiążąca ślady myślowe (*Thought*), emisję deterministycznej akcji narzędziowej (*Action*) oraz interpretację odpowiedzi środowiska (*Observation*).

2.1.2 Redukcja przestrzeni relacyjnej i ograniczenia metod promptingowych

Gdy model integruje się z korporacyjną hurtownią danych o złożonym schemacie ($\mathrm{N_T} \geq 500$ tabel), wstrzyknięcie pełnego zrzutu DDL prowadzi do wyczerpania budżetu okna kontekstowego oraz drastycznego spadku precyzji generowania zapytań SQL. W literaturze rozwinięto metody redukcji przestrzeni relacyjnej oparte na promptingu kognitywnym (DIN-SQL [**pourreza2023dinsql**], MAC-SQL [**wang2025macsql**]), doborze demonstracji few-shot (DAIL-SQL [**gao2024dailsql**]) oraz zewnętrznych filtrach heurystycznych (CodeS [**li2024codes**], CHESS [**talaei2025chess**]).

Teoretyczna luka metod promptingowych i gęstego RAG: problem grafów o dużej średnicy. Krytyczna analiza komparatywna ujawnia fundamentalną lukę teoretyczną metod promptingowych oraz gęstego wyszukiwania wektorowego (k-NN RAG): zawodzą one systemowo w obecności relacji o dużej średnicy grafu schematu ($\mathrm{dist}(\mathrm{u}, \mathrm{v}) \geq 2$). Gdy zapytanie analityczne wymaga powiązania dwóch encji skrajnych, a ścieżka w schemacie relacyjnym prowadzi przez tabele pośredniczące modelujące relacje $\mathrm{N} : \mathrm{M}$ (np. tabele asocjacyjne pozycji zamówień, partii magazynowych czy alokacji), nazwy tych tabel pośrednich nie występują w pytaniu użytkownika.

W rezultacie mechanizmy gęstego RAG oraz prompting bezkontekstowy wybierają wyłącznie tabele krańcowe, całkowicie ignorując tabele mostkujące. Model LLM, pozbawiony definicji kluczy obcych łączących te relacje, staje przed nierozwiązywalnym dylematem: halucynuje nieistniejące powiązania bezpośrednie albo generuje fałszywy iloczyn kartezjański, co prowadzi do katastrofalnej degradacji dokładności wykonawczej (EX). Luka ta stanowi bezpośrednie teoretyczne uzasadnienie dla zaprojektowania w niniejszej pracy algorytmu domknięcia grafowego (*Foreign Key Graph Pruning*), sformalizowanego w Rozdziale **??**.

2.2 Geneza standaryzacji interfejsów komunikacyjnych w AI

2.2.1 Fragmentacja ekosystemu i redukcja złożoności integracyjnej

Gwałtowny rozwój modeli językowych w latach 2023–2024 i pojawienie się wczesnych interfejsów funkcyjnych (m.in. OpenAI Function Calling [35], LangChain [7], Semantic Kernel [32]) doprowadziły do powstania długu technologicznego kodu łączącego (ang. *glue code*). Brak otwartego standardu komunikacyjnego wymuszał implementację dedykowanych adapterów dla każdego środowiska uruchomieniowego, generując wzajemnie niekompatybilne kontrakty obsługi błędów, mechanizmów autoryzacji oraz transmisji strumieniowej.

Analogicznie do problemu, jaki w inżynierii środowisk programistycznych rozwiązał protokół *Language Server Protocol* (LSP) [33], standaryzacja interfejsów w systemach agentowych opiera się na redukcji złożoności kombinatorycznej połączeń pomiędzy M środowiskami modeli a N heterogenicznymi źródłami danych i narzędzi. W architekturze połączeń doraźnych liczba wymaganych

modułów integracyjnych wynosi:

$$\mathrm{I}_{\text{ad-hoc}} = \mathrm{M} \times \mathrm{N} \implies \mathcal{O}(\mathrm{M} \cdot \mathrm{N}), \tag{1}$$

a dyskretny koszt krańcowy dołączenia nowego źródła danych wynosi $\Delta\mathrm{I}_{\text{ad-hoc}} = \mathrm{M}$. Wprowadzenie protokołu pośredniczącego przekształca pełny graf dwudzielny $\mathrm{K}_{\mathrm{M,N}}$ w topologię gwiazdy z węzłem pośredniczącym, redukując liczbę wymaganych interfejsów do postaci liniowej:

$$\mathrm{I}_{\text{MCP}} = \mathrm{M} + \mathrm{N} \implies \mathcal{O}(\mathrm{M} + \mathrm{N}), \tag{2}$$

co redukuje koszt krańcowy włączenia $(\mathrm{N}+1)$-go źródła danych do wartości stałej:

$$\Delta\mathrm{I}_{\text{MCP}} = 1 \quad (\mathcal{O}(1)). \tag{3}$$

Krytyczna ocena granic analogii do LSP w paradygmacie Tool Use. Należy sformułować istotne zastrzeżenie metodologiczne: analogia pomiędzy LSP a protokołami integracji AI (w tym MCP) ma charakter architektoniczny i pozostaje **częściowo ograniczona w warstwie semantycznej**. W protokole LSP procedury dysponują uniwersalną semantyką operacyjną dla wszystkich analizowanych języków programowania (metody takie jak `textDocument/definition` przyjmują znormalizowany URI i pozycję kursora).

W przypadku integracji narzędziowych AI standaryzacja dotyczy wyłącznie ramki transportowej JSON-RPC 2.0 oraz nadrzędnych metod odkrywania i wywołania (`tools/list`, `tools/call`). Każde narzędzie bazodanowe eksponuje unikalny kontrakt parametrów JSON Schema, indywidualne nazewnictwo atrybutów oraz dedykowany prompt kontekstowy. Oznacza to, że unifikacja warstwy transportowej ($\mathcal{O}(\mathrm{M}+\mathrm{N})$ w kodzie klientów) **nie eliminuje impedancji semantycznej** po stronie modelu językowego. Model w każdej sesji musi dokonać interpretacji kognitywnej N odmiennych kontraktów narzędziowych, przez co obciążenie okna kontekstowego oraz ryzyko halucynacji argumentów nadal skalują się wraz z rozmiarem ekosystemu narzędziowego.

Praktyczną implementacją opisanego paradygmatu stał się otwarty standard *Model Context Protocol* (MCP) [1], którego szczegółowa specyfikacja techniczna, prymitywy semantyczne oraz warstwy transportowe zostały poddane analizie w podrozdziale **??**.

2.3 Formalna specyfikacja Model Context Protocol (MCP)

2.3.1 Role architektoniczne w protokole

Specyfikacja protokołu MCP [1] definiuje ścisły podział odpowiedzialności pomiędzy trzema głównymi węzłami logicznymi:

1. **Host (Aplikacja orkiestrująca):** Główny proces nadrzędny, w ramach którego realizowane jest zadanie użytkownika. Host zarządza interfejsem użytkownika, integruje model językowy,

koordynuje procesy decyzyjne oraz sprawuje bezpośrednią kontrolę nad politykami bezpieczeństwa i zgodami użytkownika na wykonanie akcji. Przykłady Hostów to m.in. aplikacja Claude Desktop, środowisko IDE (np. JetBrains, Visual Studio Code) lub dedykowany orkiestrator mikrousług;

2. **Klient MCP:** Biblioteczny moduł komunikacyjny osadzony wewnątrz architektury Hosta. Klient utrzymuje ścisłe połączenie 1:1 z danym serwerem MCP. Odpowiada za translację wewnętrznych struktur Hosta na komunikaty protokołu, zarządzanie warstwą transportową, serializację i deserializację pakietów JSON-RPC oraz obsługę błędów protokołu;

3. **Serwer MCP:** Niezależny, wyspecjalizowany komponent programistyczny (proces lokalny lub serwis sieciowy), hermetyzujący dostęp do określonej domeny danych lub zestawu operacji. Serwer publikuje swoje możliwości, udostępniając zasoby pasywne, aktywne narzędzia wykonawcze oraz szablony promptów.

Relacja między Hostem, Klientem i Serwerem ma charakter modularny. Pojedynczy Host może powołać do życia wiele niezależnych instancji Klientów MCP, z których każdy komunikuje się z odrębnym Serwerem MCP (np. Serwer bazy PostgreSQL, Serwer brokera Kafka, Serwer systemu plików), tworząc spójny graf kontekstowy dostępny dla modelu językowego.

2.3.2 Cykl życia sesji protokołu

Zgodnie ze specyfikacją [1], interakcja pomiędzy Klientem a Serwerem MCP podlega rygorystycznemu automatowi stanów, obejmującemu cztery kolejne fazy:

1. **Faza inicjalizacji i uzgadniania wersji:** Po zestawieniu fizycznego kanału transportowego Klient inicjuje sesję poprzez wysłanie synchronicznego żądania RPC o metodzie `initialize`. W treści żądania przekazywana jest wersja protokołu wspierana przez Klienta, identyfikator implementacji oraz deklaracja oczekiwanych zdolności. Serwer odpowiada komunikatem zawierającym własną wersję protokołu, metadane oraz listę oferowanych funkcjonalności. W przypadku braku kompatybilności wersji sesja zostaje natychmiast przerwana ze zwróceniem kodu błędu.

2. **Potwierdzenie i negocjacja zdolności:** Po otrzymaniu poprawnej odpowiedzi inicjalizacyjnej Klient przesyła jednostronne powiadomienie. W tym momencie następuje formalne potwierdzenie gotowości obu stron do pracy. W ramach negocjacji zdolności:
   - Serwer deklaruje wsparcie dla rejestrów: `tools` (narzędzia), `resources` (zasoby), `prompts` (szablony) oraz `logging` (poziomy logowania);
   - Klient deklaruje wsparcie dla: `roots` (definiowanie dozwolonych ścieżek dostępu do zasobów) oraz `sampling` (mechanizm pozwalający serwerowi na delegowanie zapytań generatywnych z powrotem do modelu językowego za pośrednictwem klienta).

3. **Faza operacyjna:** Właściwa faza robocza, w której realizowane są dwukierunkowe transakcje. Obejmuje ona:
   - Odpytywanie o listy dostępnych prymitywów (`tools/list`, `resources/list`, `prompts/list`);
   - Wywoływanie procedur narzędziowych (`tools/call`);
   - Pasywny odczyt strumieni danych (`resources/read`);
   - Asynchroniczne notyfikacje o zmianie stanu (np. `notifications/resources/updated`);
   - Diagnostykę i badanie żywotności połączenia za pomocą zapytań `ping` (mechanizm *keep-alive*).

4. **Faza zamykania sesji:** Procedura kontrolowanego zwolnienia zasobów. Klient zamyka kanał wejściowy, co sygnalizuje serwerowi konieczność zakończenia aktywnych transakcji bazodanowych, opróżnienia buforów I/O, zamknięcia gniazd sieciowych i uporządkowanego zakończenia procesu potomnego bez pozostawiania procesów osieroconych.

2.3.3 Warstwy transportowe protokołu

Standard MCP oddziela logikę semantyczną komunikatów JSON-RPC od fizycznego medium transmisyjnego. Specyfikacja definiuje dwie podstawowe warstwy transportowe:

1. **Transport procesowy Standard I/O (`stdio`):** Dedykowany dla lokalnych środowisk wykonawczych, w których Serwer MCP jest uruchamiany bezpośrednio przez Hosta jako proces potomny przy użyciu funkcji systemowych. Komunikacja odbywa się za pośrednictwem anonimowych potoków strumieniowych:
   - Standardowy strumień wejściowy (`stdin`) procesu potomnego służy do odbierania poleceń od klienta;
   - Standardowy strumień wyjściowy (`stdout`) procesu potomnego służy do wysyłania odpowiedzi i powiadomień do klienta;
   - Standardowy strumień błędów (`stderr`) jest odseparowany i zarezerwowany wyłącznie dla komunikatów diagnostycznych i logów serwera.

   Komunikaty są formatowane jako pojedyncze linie tekstu JSON zakończone znakiem nowego wiersza (`\n`) zgodnie z RFC 8259 [4]. Zaletą transportu `stdio` jest znikomy narzut komunikacji międzyprocesowej, brak konieczności alokacji portów sieciowych TCP/IP, natywna izolacja uprawnień w systemie operacyjnym oraz brak opóźnień stosu sieciowego.

2. **Transport sieciowy Server-Sent Events przez HTTP (`SSE/HTTP`):** Przeznaczony dla architektur rozproszonych, chmurowych oraz środowisk kontenerowych, w których serwer działa jako niezależny mikroserwis. Transport ten wykorzystuje asymetryczny model dwukanałowy:
   - **Kanał Serwer → Klient (SSE):** Klient nawiązuje długożyjące połączenie HTTP GET z punktem końcowym `/sse`. Zgodnie ze standardem W3C Server-Sent Events [17],

serwer przesyła asynchroniczny strumień zdarzeń z nagłówkiem `Content-Type: text/event-stream`. W pierwszym zdarzeniu przekazywany jest unikalny identyfikator sesji oraz relatywny adres URL dedykowany dla zapytań wejściowych;

- **Kanał Klient → Serwer (HTTP POST):** Wszelkie polecenia i odpowiedzi JSON-RPC od klienta są wysyłane jako standardowe żądania HTTP POST na dedykowany punkt końcowy.

Transport SSE umożliwia łatwą integrację z istniejącą infrastrukturą zapór sieciowych, bramkami API oraz systemami równoważenia obciążenia, oferując jednocześnie pełne wsparcie dla asynchronicznych powiadomień serwera.

2.3.4 Prymitywy semantyczne MCP

Warstwa aplikacyjna MCP wprowadza trzy kluczowe prymitywy semantyczne, modelujące wszelkie interakcje modelu z otoczeniem:

1. **Zasoby (Resources):** Pasywny mechanizm udostępniania danych kontekstowych w trybie wyłącznie do odczytu. Zasób jest jednoznacznie identyfikowany przez ujednolicony identyfikator zasobu URI [4], np. `postgres://schema/public/tables/sales`. Zasoby mogą reprezentować zarówno dane tekstowe (np. schematy DDL, pliki konfiguracyjne, logi), jak i binarne zakodowane w standardzie Base64. Specyfikacja wspiera szablony zasobów oparte na parametryzowanych wzorcach URI (np. `db://tables/{tableName}/schema`), pozwalające na dynamiczne odpytywanie sparametryzowanych widoków.

2. **Narzędzia (Tools):** Aktywny mechanizm wywoływania procedur wykonawczych, zdolnych do generowania efektów ubocznych w środowisku zewnętrznym. Narzędzie w MCP definiowane jest przez unikalną nazwę, czytelny dla modelu opis funkcjonalny w języku naturalnym oraz sformalizowaną specyfikację parametrów wejściowych w standardzie JSON Schema [47]. Model językowy na podstawie opisu i schematu generuje ściśle typowany obiekt argumentów w formacie JSON. Odpowiedź narzędzia zwracana jest jako lista bloków zawartości (`content`), mogąca zawierać tekst, obrazy lub osadzone zasoby.

3. **Prompty (Prompts):** Sterowane przez serwer szablony instrukcji i konwersacji. Umożliwiają dostawcy serwera narzucenie optymalnych ról systemowych, dobrych praktyk inżynierii promptów oraz precyzyjnych wytycznych dziedzinowych bezpośrednio do pamięci roboczej Hosta, ułatwiając modelowi prawidłową interpretację zwracanych danych.

2.4 Specyfika systemów wielkoskalowych w interakcji z agentami

2.4.1 Fizyczne i logiczne bariery mechanizmu uwagi

Powszechne w przemyśle przekonanie, że ekspansja nominalnego okna kontekstowego modeli LLM (od 128k tokenów w rodzinie Llama 3.1 do miliona i więcej tokenów w architekturach komercyjnych)

eliminuje konieczność selekcji schematów i paginacji danych, stoi w sprzeczności z fizyką serwowania modeli oraz dowodami empirycznymi nad mechanizmem samo-uwagi (*Self-Attention*).

Kwadratowa złożoność obliczeniowa mechanizmu uwagi względem długości sekwencji L w fazie wstępnego przetwarzania promptu (*prefill*), wynosząca $\mathcal{O}(\mathrm{L}^2)$, generuje wielosekundowe opóźnienia pierwszego tokenu przy próbie wstrzyknięcia pełnego zrzutu DDL enterprise'owej bazy danych. Ponadto utrzymanie długiego kontekstu wymaga alokacji bufora kluczy i wartości (*KV-Cache*), którego fizyczny rozmiar w pamięci akceleratora skaluje się liniowo:

$$\mathrm{M}_{\mathrm{KV}} = 2 \cdot \mathrm{b}_{\mathsf{elem}} \cdot \mathrm{l}_{\mathsf{layers}} \cdot \mathrm{l}_{\mathsf{heads}} \cdot \mathrm{d}_{\mathsf{head}} \cdot \mathrm{L}, \tag{4}$$

gdzie $\mathrm{b}_{\mathsf{elem}} = 2$ B oznacza rozmiar słowa w precyzji FP16/BF16, a mnożnik 2 wynika z jednoczesnej alokacji tensorów kluczy i wartości ($\mathrm{l}_{\mathsf{layers}}$ określa liczbę warstw transformera, $\mathrm{l}_{\mathsf{heads}}$ liczbę głów uwagi, natomiast $\mathrm{d}_{\mathsf{head}}$ wymiar pojedynczej głowy). Dla pojedynczej sesji o długości 128k tokenów w modelu klasy 7B alokacja ta pochłania ponad 7 GB pamięci VRAM GPU, drastycznie redukując dopuszczalną współbieżność serwowania.

Niezależnie od barier sprzętowych, badania nad dynamiką uwagi w długich sekwencjach dowodzą załamania jakości wnioskowania:

- **Zapaść uwagi w środku sekwencji:** Liu i in. [26] wykazali, że zdolność modeli autoregresyjnych do ekstrakcji informacji ulega gwałtownej degradacji w centralnych segmentach kontekstu (zjawisko *Lost in the Middle*, charakterystyka w kształcie litery U), przez co definicje tabel umieszczone w środku obszernego zrzutu DDL są pomijane;
- **Zanieczyszczenie kontekstu:** Levy i in. [**levy2024same**] wykazali, że rozszerzenie promptu o nieistotne definicje relacji systematycznie obniża wierność semantyczną generowanego kodu SQL.

Uwarunkowania te dowodzą, że integracja z dużymi składami danych wymaga deterministycznych technik kompresji schematu oraz adaptacyjnej paginacji kursorowej bezpośrednio w warstwie pośredniczącej MCP.

2.4.2 Transakcyjność i modele spójności w komunikacji bezstanowej

W architekturze MCP wywołania narzędzi mają charakter bezstanowy (*stateless RPC per tools/call*), co w zderzeniu z wielosekundową inferencją modeli uniemożliwia utrzymanie długich blokad transakcyjnych bez blokowania horyzontu $\mathrm{x}_{\mathsf{min}}$ MVCC (*vacuum starvation*) oraz paraliżu puli połączeń. Każde zapytanie kursorowe agenta stanowi odrębne żądanie sieciowe JSON-RPC, po którym fizyczne połączenie jest natychmiast zwalniane do puli asynchronicznej. Pomiędzy pobraniem kolejnych stron upływa czas pętli decyzyjnej agenta (obejmujący generowanie tokenów przez model LLM, typowo od kilkuset milisekund do kilkunastu sekund), w trakcie którego baza danych nie może utrzymywać otwartej transakcji. Próba wymuszenia klasycznej izolacji transakcyjnej w interakcji ze stochastycznym modelem prowadzi zatem do zjawiska niepowtarzalnego odczytu oraz anomalii

widmowych, a w środowiskach rozproszonych ze spójnością ostateczną – do ryzyka podejmowania akcji w oparciu o zdezaktualizowany stan repliki.

2.4.3 Stochastyczne transakcje agentowe i paradygmat SAGA

Najpoważniejszym wyzwaniem inżynierskim na styku modeli językowych i systemów danych jest zjawisko *stochastycznych transakcji agentowych*. W tradycyjnym oprogramowaniu kod wykonujący operacje bazodanowe jest deterministyczny – programista definiuje precyzyjny przepływ sterowania, obsługę wyjątków oraz warunki zatwierdzania i wycofywania transakcji. W przypadku agenta autonomicznego decyzja o emisji polecenia SQL podejmowana jest w drodze próbkowania probabilistycznego ze stochastycznego rozkładu tokenów, co rodzi specyficzne bariery:

1. **Niedeterminizm semantyczny:** Identyczna intencja użytkownika może skutkować wygenerowaniem odmiennych zapytań SQL, różniących się warunkami złączeń tabel lub predykatami filtrującymi;

2. **Załamanie protokołów blokujących (2PC/XA):** Klasyczne rozproszone protokoły transakcyjne, takie jak dwufazowe zatwierdzanie (2PC) [13] czy standard XA, wymagają pesymistycznego utrzymywania blokad zasobów w fazie przygotowania. Wieloetapowe wnioskowanie LLM wprowadza stochastyczną latencję rzędu kilku sekund na krok pętli decyzyjnej. Utrzymywanie blokad bazodanowych w tym czasie wywołuje paraliż współbieżności i błyskawiczne wyczerpanie puli połączeń;

3. **Brak semantyki transakcyjnej w protokole MCP:** Protokół MCP nie definiuje obecnie mechanizmów automatycznego wycofywania zmian. Niepowodzenie akcji w kroku $\mathrm{k}+1$ (np. błąd publikacji na brokerze Kafka) pozostawia bazę danych w stanie częściowo zmodyfikowanym przez krok k;

4. **Podatność na instrukcje destrukcyjne:** Podatność modeli na ataki wstrzykiwania poleceń (*prompt injection*) [14] stwarza bezpośrednie ryzyko wykonania nieodwracalnych operacji DDL/DML pod wpływem zmanipulowanego kontekstu.

Odpowiedzią na niemożność stosowania blokad 2PC w architekturach agentowych jest adaptacja **wzorca SAGA** [**garcia1987sagas**]. Wzorzec ten dekomponuje globalną transakcję rozproszoną na ciąg niezależnych transakcji lokalnych:

$$\mathrm{T} = (\mathrm{T}_1, \mathrm{T}_2, \ldots, \mathrm{T}_\mathrm{n}), \tag{5}$$

z których każda zatwierdza swój stan lokalnie w chwili zakończenia danego kroku. W przypadku wystąpienia błędu wnioskowania, wyjątku wykonania lub przerwania pętli decyzyjnej na etapie $\mathrm{k} < \mathrm{n}$, orkiestrator agentowy uruchamia sekwencję transakcji kompensujących:

$$\mathrm{C} = (\mathrm{C}_{\mathrm{k}-1}, \ldots, \mathrm{C}_2, \mathrm{C}_1), \tag{6}$$

odwracających skutki biznesowe uprzednio zatwierdzonych operacji w odwrotnej kolejności wykonania.

Wdrożenie wzorca SAGA w połączeniu ze statyczną analizą drzew składniowych AST (izolującą operacje modyfikujące w osobnym perymetrze) stanowi deterministyczny sposób zabezpieczenia hurtowni danych przed stochastycznymi błędami modeli językowych. Zagadnienia te stanowią bezpośrednie uzasadnienie dla architektury bramy pośredniczącej zaprojektowanej w Rozdziale **??**.

Rozdział 3

Analiza Porównawcza Protokołów Komunikacyjnych

Integracja wielkich modeli językowych ze złożonymi relacyjnymi składami danych wymaga precyzyjnego zdefiniowania warstwy komunikacyjnej, determinującej sposób serializacji, transportu, autoryzacji oraz zarządzania stanem interakcji. W klasycznych systemach rozproszonych standardy wymiany danych projektowano z myślą o deterministycznych procesach programistycznych o sztywno typowanych strukturach. Zastosowanie autonomicznych agentów sztucznej inteligencji, operujących na stochastycznych rozkładach prawdopodobieństwa tokenów tekstowych, stworzyło jakościowo nowe wyzwania komunikacyjne, których nie adresują tradycyjne paradygmaty integracyjne.

Dobór protokołu komunikacyjnego w architekturze agentowej wyznacza fundamentalny kompromis inżynierski pomiędzy przepustowością binarną, narzutem serializacji tekstowej, bezpieczeństwem perymetru bazy danych a transparentnością śladu wnioskowania. Z jednej strony protokoły binarne (np. gRPC) minimalizują narzut sieciowy kosztem utraty dynamicznej inspekcji semantycznej. Z drugiej strony interfejsy bezstanowe (np. REST) narzucają wysoki narzut re-autoryzacji i transmisji nagłówków w każdym kroku wieloetapowego wnioskowania. Odpowiedzią na te wyzwania stało się zaadaptowanie formatu JSON-RPC 2.0 w ramach otwartego standardu MCP [1], pozycjonowanego jako protokół integracji pionowej.

Niniejszy rozdział przedstawia formalną i empiryczną analizę porównawczą protokołów komunikacyjnych stosowanych w systemach agentowych. W podrozdziale **??** przeanalizowano specyfikację JSON-RPC 2.0 jako fundamentu MCP, dekonstruując narzuty serializacji tekstowej i niszczenia tokenizacji BPE. Podrozdział **??** zawiera taksonomię porównawczą architektur REST, gRPC, CodeAct, MCP oraz ACP/A2A, zsyntetyzowaną w zbiorczej tabeli analitycznej. W podrozdziale **??** uzasadniono kompromis inżynierski MCP, w tym polemikę z paradygmatem CodeAct. Podrozdział **??** poświęcono relacji do protokołów horyzontalnych, a podrozdział **??** zamyka rozdział analizą wektorów ataków w składach analitycznych.

3.1 Protokół JSON-RPC 2.0 jako rdzeń Model Context Protocol

3.1.1 Specyfikacja ramki i determinizm kodów błędów

Standard JSON-RPC 2.0 [**jsonrpc2010**] jest lekkim protokołem zdalnego wywoływania procedur (ang. *Remote Procedure Call*, RPC), wykorzystującym format JSON sformalizowany w dokumencie RFC 8259 [4]. Protokół charakteryzuje się pełnym agnostycyzmem transportowym oraz natywną czytelnością ładunku przez mechanizmy tokenizacji modeli językowych.

Specyfikacja definiuje ścisłą strukturę komunikatów. Podstawowa ramka żądania procedury (*Request object*) składa się z czterech pól: (1) `jsonrpc` – wartość o stałej treści `"2.0"` definiująca wersję protokołu; (2) `method` – łańcuch znaków identyfikujący procedurę (np. `tools/call`, `resources/read`, `initialize`); (3) `params` – ustrukturyzowany obiekt lub tablica argumentów; (4) `id` – numeryczny lub łańcuchowy identyfikator korelacyjny, zwracany w odpowiedzi serwera w celu asynchronicznego wiązania wyników z kwerendami w środowiskach współbieżnych.

W przypadku pomyślnego wykonania procedury serwer emituje ramkę sukcesu z polami `jsonrpc`, skorelowanym `id` oraz obiektem `result`. W przypadku wystąpienia anomalii emitowana jest ramka błędu (*Error Response*) zawierająca obiekt `error` ze znormalizowanym kodem numerycznym (`code`), komunikatem (`message`) oraz opcjonalnym polem danych diagnostycznych (`data`). Standard rezerwuje przedział kodów od -32768 do -32000 na błędy bazowe: błąd parsowania (-32700), niepoprawną strukturę żądania (-32600), brak metody (-32601), niepoprawne parametry (-32602) oraz błąd wewnętrzny (-32603). Gwarantuje to deterministyczną obsługę wyjątków na poziomie protokołu przed przekazaniem sterowania do logiki biznesowej.

3.1.2 Rozróżnienie wywołań synchronicznych i powiadomień asynchronicznych

Kluczową właściwością specyfikacji JSON-RPC 2.0 w architekturze MCP jest semantyczne rozróżnienie pomiędzy pełnym żądaniem RPC a jednokierunkowym powiadomieniem. Powiadomienie definiowane jest jako żądanie pozbawione pola `id`. Brak tego identyfikatora nakazuje odbiorcy przetworzenie zdarzenia bez emitowania jakiejkolwiek odpowiedzi zwrotnej.

W protokole MCP mechanizm powiadomień realizuje dwa krytyczne zadania: (1) **Sygnalizację stanu cyklu życia:** potwierdzenie zakończenia fazy negocjacji zdolności przez klienta, co otwiera fazę operacyjną bez oczekiwania na odpowiedź serwera; (2) **Asynchroniczny Server Push:** przy dynamicznej zmianie stanu zewnętrznego źródła danych, serwer emituje powiadomienie, eliminując konieczność kosztownego odpytywania cyklicznego ze strony klienta.

3.1.3 Narzut serializacji tekstowej UTF-8 i destrukcja tokenizatora BPE

Format JSON kodowany w UTF-8 zapewnia transparentność inspekcji, lecz w środowiskach relacyjnych rodzi istotny narzut transmisyjny wynikający z: (1) redundancji etykiet schematu poprzez powielanie tekstowych nazw kolumn w każdej krotce zbioru analitycznego; (2) ucieczki znaków wymagającej prefiksowania znaków specjalnych; oraz (3) destrukcji słownika tokenizatora BPE

przez kodowanie Base64. Brak typu binarnego w JSON wymusza rzutowanie danych binarnych na ciągi Base64 (narzut 33,3% bajtów). Kluczową konsekwencją jest niszczenie tokenizatorów podsłów (ang. *Byte-Pair Encoding*, BPE) modeli językowych [36]. Ciągi Base64 cechują się wysoką entropią pseudolosową pozbawioną naturalnych podsłów morfosyntaktycznych, przez co tokenizator rozbija je na pojedyncze znaki, osiągając skrajnie niekorzystny współczynnik rzędu ≈ 1 **token na bajt danych binarnych**.

Na listingu **??** przedstawiono zwartą strukturę ramki wywołania narzędzia oraz skompaktowaną odpowiedź serwera MCP z zaimplementowanym stronicowaniem kursorowym.

```
// 1. Żądanie klienta MCP (tools/call):
{ "jsonrpc": "2.0", "id": "req-042", "method": "tools/call",
  "params": {
    "name": "query_warehouse",
    "arguments": {
      "sql_query": "SELECT order_id, amount FROM orders WHERE status = '
    COMPLETED'",
      "cursor": "eyJpZCI6IDEwNDg1NzYsICJ0cyI6IDE3Mjk5NDAwMDB9",
      "limit": 50, "max_tokens_budget": 2048
    }
  }
}
// 2. źOdpowied serwera MCP ze stronicowaniem kursorowym:
{ "jsonrpc": "2.0", "id": "req-042",
  "result": {
    "content": [{"type": "text", "text": "{\"rows\": [{\"order_id\":
    1048577, \"amount\": 1420.50}], \"next_cursor\": \"eyJpZCI6IDEwNDg1Nzd9
    \", \"has_more\": true, \"token_cost\": 192}"}],
    "isError": false
  }
}
```

Listing 1. Kompaktowa ramka wywołania i odpowiedzi narzędzia MCP ze stronicowaniem kursorowym

Most logiczny do badań własnych. Zjawisko niszczenia tokenizatorów BPE przez dane binarne dowodzi, że bezpośrednia transmisja masowych ładunków analitycznych w paśmie JSON-RPC paraliżuje okno uwagi modelu. Fakt ten stanowi bezpośrednie uzasadnienie zaprojektowanej w niniejszej pracy magisterskiej architektury transferu pozapasmowego (*Out-of-Band*, Rozdział **??**), dla której analityczny i empiryczny próg opłacalności wyznaczono na poziomie $\mathrm{S}^* = 6{,}89$ kB (Rozdział **??**).

3.2 Taksonomia architektur komunikacyjnych w systemach agentowych

3.2.1 Charakterystyka porównywanych paradygmatów architektonicznych

Ocena właściwości protokołu Model Context Protocol wymaga osadzenia go w kontekście wiodących standardów wymiany danych w systemach rozproszonych:

1. **Architektura REST (Fielding 2000 [10]):** zorientowana na reprezentację zasobów identyfikowanych za pomocą jednolitych adresów URI oraz manipulację stanem poprzez standardowe czasowniki HTTP; zakłada całkowity brak pamięci sesyjnej po stronie serwera pomiędzy kolejnymi żądaniami;

2. **Standard gRPC (Google [43]):** wysokowydajny, binarny framework RPC oparty na transportowym standardzie HTTP/2 [3] oraz mechanizmie serializacji Protocol Buffers [12], wykorzystujący statyczne deskryptory interfejsów (pliki `.proto`) z numerycznym znakowaniem pól w reżimie TLV;

3. **Apache Arrow Flight [arrow_flight]:** wyspecjalizowany transport analityczny oparty na formacie kolumnowym Apache Arrow [**apache_arrow**], zorientowany na masowy przesył pakietów rekordów w pamięci współdzielonej w technologii *zero-copy*;

4. **Paradygmat CodeAct (Wang i in., ICML 2024 [wang2024codeact]):** model traktujący wykonywalny kod w języku Python jako uniwersalną akcję agenta, integrujący dynamiczną modyfikację środowiska bezpośrednio w pętli interpretera REPL;

5. **Protokoły wieloagentowe (ACP i A2A [8]):** standardy komunikacji poziomej wywodzące się z ontologii FIPA ACL, modelujące dialog społeczny, negocjacje zadań oraz asynchroniczną wymianę aktów mowy w rozproszonych rojach agentowych.

3.2.2 Model dwoistości stanowej w protokole MCP

W niniejszej pracy magisterskiej formalizuje się **dwuwarstwowy model stanowości protokołu MCP**: (1) **Warstwa Transportowo-Sesyjna (STANOWA):** opiera się na trwałym medium (`stdio` IPC lub sieciowy `SSE/HTTP`) i realizuje automat FSM (handshake `initialize`, negocjacja zdolności `capabilities`, keep-alive `ping`). Stan sesji jest trwale zakotwiczony w kanale transmisyjnym i nie wymaga re-autoryzacji w trakcie interakcji; (2) **Warstwa Wykonawcza Narzędzi (BEZSTANOWA / Stateless RPC):** wywołania `tools/call` stanowią odizolowane transakcje żądanie-odpowiedź JSON-RPC 2.0. Serwer nie przechowuje kontekstu poprzednich wywołań ani otwartych transakcji w pamięci RAM.

Implikacje dla relacyjnych baz danych. Bezstanowość wywołań `tools/call` gwarantuje horyzontalną skalowalność bramy pośredniczącej, lecz w zderzeniu z wieloetapową inferencją modeli LLM

rodzi ryzyko anomalii współbieżności omówionych w podrozdziale **??**. Z uwagi na niedopuszczalność blokowania puli połączeń i horyzontu $\mathrm{x}_{\mathrm{min}}$ MVCC przez bezczynnego agenta, konflikt ten bezpośrednio motywuje opracowany w Rozdziale **??** moduł `TransactionSnapshotManager`, synchronizujący bezstanowe wywołania z izolowanymi migawkami silnika PostgreSQL.

3.2.3 Wzorcowa tabela taksonomiczna

W tabeli **??** zestawiono właściwości analizowanych paradygmatów komunikacyjnych pod kątem dziesięciu kluczowych kryteriów inżynierskich.

Tabela 2. Wielowymiarowa taksonomia architektur komunikacyjnych i paradygmatów interakcji agentowej

Kryterium analizy	REST (HTTP/1.1–2)	gRPC (Protobuf)	CodeAct (Python)	MCP (JSON-RPC 2.0)	ACP / A2A
1. Warstwa transportowa	HTTP/1.1–2 (TCP)	HTTP/2–3 (TCP/QUIC)	Proces potomny (`stdio`) / WebSocket	Potoki IPC (`stdio`) / sieć `SSE/HTTP`	HTTP, WebSocket, brokerzy AMQP/Kafka
2. Format ładunku	Tekstowy JSON / XML	Binarny Protobuf (TLV)	Kod źródłowy Python	JSON-RPC 2.0 (UTF-8, Base64)	JSON-LD / ontologie FIPA ACL
3. Zarządzanie stanem	Bezstanowy (nagłówki per żądanie)	Stanowe strumienie HTTP/2	Stanowa sesja interpretera w RAM	Sesja stanowa (FSM), bezstanowe RPC (`tools/call`)	Stanowe wątki dialogowe i pamięć roju
4. Kontrola przepływu (Backpressure)	Ograniczona (kody 429, okno TCP)	Natywna w HTTP/2 (`WINDOW_UPDATE`)	Brak (ryzyko OOM przy zrzucie na `stdout`)	Asynchroniczna pętla zdarzeń i stronicowanie	Buforowanie po stronie brokera komunikatów
5. Multipleksowanie	W HTTP/2 (brak w HTTP/1.1)	Pełne, dwukierunkowe w HTTP/2	Brak (sekwencyjna pętla REPL)	Logiczne (po polu `id` w kanale)	Asynchroniczne po `conversation-id`
6. Narzut per-request	Wysoki (nagłówki HTTP/JWT, klucze)	Minimalny (zwięzłe tagi binarne, HPACK)	Pomijalny w sieci; koszt interpretacji AST	Umiarkowany (koperta JSON-RPC, parametry)	Wysoki (metadane semantyczne)
7. Determinizm typowania	Opcjonalny (OpenAPI / schematy JSON)	Rygorystyczny w compile-time (`protoc`)	Brak (typowanie dynamiczne, błędy runtime)	Deterministyczny w runtime (JSON Schema)	Zmienny (walidacja ontologiczna RDF/OWL)
8. Natywność dla LLM (Tokenizacja BPE)	Średnia (tekst JSON, zbędne nagłówki)	Niska (wymaga ciągłej translacji binarnej)	Wysoka (model naturalnie generuje kod)	Bardzo wysoka (JSON optymalny pod BPE)	Wysoka (tekstowe akty mowy)
9. Główna rola w architekturze AI	Publiczne API, usługi brzegowe	Komunikacja backendowa i klastry GPU	Zadania analityczne, manipulacja plikami	**Integracja pionowa:** model $\leftrightarrow$ bazy SQL, pliki	**Integracja pozioma:** koordynacja roju agentów

3.3 Równowaga kompromisów: uzasadnienie wyboru JSON-RPC 2.0

3.3.1 Przyczyny odrzucenia architektur REST i gRPC

Wybór formatu JSON-RPC 2.0 w warstwie sterowania protokołu MCP był podyktowany eliminacją ograniczeń architektury REST oraz gRPC w kontakcie ze specyfiką modeli LLM:

- **Odrzucenie architektury REST:** Wieloetapowe pętle decyzyjne (np. ReAct) wymagają sekwencji powiązanych odpytań w ramach jednego procesu analitycznego. Aksjomat bezstanowości REST wymusza przesyłanie pełnych nagłówków uwierzytelniających (np. wielokilobajtowych tokenów Bearer JWT) przy każdym wywołaniu, generując ciągły narzut transmisyjny i obliczeniowy na re-weryfikację kryptograficzną. Ponadto brak trwałego kanału sesyjnego uniemożliwia jednorazową negocjację zdolności narzędziowych, a brak natywnego wsparcia dla asynchronicznych powiadomień typu Server Push zmusza klienta do nieefektywnego odpytywania cyklicznego;
- **Odrzucenie binarnego gRPC:** Pomimo przewagi przepustowości kompresji Protobuf, format ten wprowadza barierę inspekcji semantycznej. Modele autoregresyjne operują natywnie na sekwencjach tekstowych i nie posiadają zdolności bezpośredniego dekodowania ani generowania binarnych strumieni bajtów. Zastosowanie gRPC wymuszałoby ciągłą translację pośrednią tekstu wygenerowanego przez model do buforów binarnych, utrudniając śledzenie łańcucha myśli i debugowanie. Co więcej, lekki schemat JSON Schema w MCP pozwala na dynamiczną ekstrakcję katalogu systemowego bazy danych i generowanie definicji narzędzi w locie, podczas gdy gRPC wymaga uprzedniej kompilacji deskryptorów usług stelażami.

3.3.2 Krytyczna polemika z paradygmatem Code-as-Action

W środowiskach korporacyjnych składów danych paradygmat CodeAct [**wang2024codeact**] (wykonywanie generowanego kodu Pythona w pętli REPL) rodzi dyskwalifikujące ryzyka inżynierskie:

1. **Brak możliwości nałożenia deterministycznego perymetru AST:** Dynamiczny kod ogólnego przeznaczenia wyklucza zamkniętą, statyczną analizę bezpieczeństwa. W przeciwieństwie do deklaratywnego SQL, w którym parser składniowy AST (np. biblioteka `sqlglot` [30]) gwarantuje niezmiennik czystego odczytu `exp.Select` i blokuje modyfikacje DML/DDL, w języku Python adwersarz może zaciemnić operacje niszczące lub uzyskać dostęp do powłoki systemu poprzez mechanizmy refleksji, moduł `importlib` bądź wywołania `ctypes`;

2. **Narzut opóźnień wirtualizacji piaskownicy (Sandboxing):** Bezpieczna izolacja kodu Pythona wymaga pełnej wirtualizacji kontenerowej, co narzuca opóźnienie zimnego startu rzędu 150–300 ms na każde wywołanie oraz wielomegabajtowy narzut pamięciowy. Z kolei deterministyczna weryfikacja AST deklaratywnego zapytania SQL w odizolowanej puli procesowej serwera MCP wykonuje się w czasie poniżej 1 ms, zapewniając submilisekundową responsywność pod ruchem współbieżnym;

3. **Determinizm kontraktu wywoławczego w MCP:** Zastosowanie sformalizowanych schematów JSON Schema tworzy ścisły kontrakt walidacyjny pomiędzy modelem a bramą pośredniczącą, co umożliwia odrzucenie błędnych lub złośliwych kwerend na etapie przedtransakcyjnym.

Przyjęty w MCP kompromis architektoniczny świadomie akceptuje wyższy narzut pasma transmisyjnego (2,5–4-krotnie większy rozmiar ładunku JSON względem Protobuf) oraz konieczność elimina-

cji wąskich gardeł Base64 dla danych binarnych, uzyskując w zamian zerowy próg implementacyjny, natywną symbiozę z tokenizatorami BPE oraz możliwość nałożenia deterministycznego perymetru bezpieczeństwa.

3.4 Relacja MCP do protokołów komunikacji wieloagentowej (ACP i A2A)

W architekturach systemów wieloagentowych (MAS) kluczowe znaczenie ma rozgraniczenie dwóch ortogonalnych płaszczyzn integracji [8]: poziomej (horyzontalnej), służącej do koordynacji i negocjacji w roju, oraz pionowej (wertykalnej), łączącej model z deterministycznymi narzędziami i bazami danych:

$$\text{Architektura MAS} = \underbrace{\text{Płaszczyzna Pozioma (ACP / A2A)}}_{\text{Koordynacja, Negocjacje}} \times \underbrace{\text{Płaszczyzna Pionowa (MCP)}}_{\text{Infrastruktura, Bazy Danych, Narzędzia}} . \quad (7)$$

Z punktu widzenia integracji wielkich składów danych optymalnym modelem współistnienia obu płaszczyzn jest **wzorzec agenta analitycznego** (*Specialist Agent Pattern*). W rozległym roju agentów bezpośrednie połączenie z serwerem MCP utrzymuje wyłącznie jeden dedykowany agent analityczny. Pozostałe węzły delegują zadania eksploracyjne za pośrednictwem protokołu poziomego (A2A). Agent analityczny dokonuje translacji intencji biznesowej na kwerendy SQL w kanale pionowym MCP i odsyła zagregowany rezultat do roju, co skutecznie zabezpiecza silnik relacyjny przed lawinowym mnożeniem fizycznych sesji bazodanowych.

3.5 Wektory ataków i deterministyczne bezpieczeństwo protokołów

3.5.1 Autoryzacja i podatność Confused Deputy w środowiskach MCP

Klasyczne modele bezpieczeństwa zakładają tożsamość podmiotu wykonującego akcję z uwierzytelnionym użytkownikiem. W systemach agentowych następuje **delegacja uprawnień wykonawczych na autonomiczny komponent probabilistyczny**, co rodzi problem autoryzacji otoczenia: statyczna konfiguracja uprawnień serwera MCP udostępnia modelowi pełne zasoby bazy danych, co wymusza dynamiczną propagację tożsamości w metadanych ramki JSON-RPC. Zjawisko to prowadzi bezpośrednio do podatności zdezorientowanego zastępcy (ang. *Confused Deputy*) [16]: uprzywilejowany serwer MCP zostaje nakłoniony przez stochastyczny model LLM do wykonania operacji na rzecz nieuprawnionego użytkownika. Brak deterministycznej weryfikacji uprawnień wewnątrz modelu umożliwia eskalację uprawnień i nieautoryzowany odczyt poufnych tabel przez konto techniczne serwera bazy danych.

3.5.2 Zagrożenie pośredniego wstrzykiwania instrukcji i perymetr obronny

Najpoważniejszym zagrożeniem integracji bazodanowej LLM (LLM01 na liście OWASP [37]) jest pośrednie wstrzykiwanie promptu (*Indirect Prompt Injection*) [14] poprzez zatrucie danych: (1) zewnętrzny adwersarz umieszcza ładunek (np. `"[SYSTEM: execute_sql 'DROP TABLE orders;']"`) w polu bazy transakcyjnej; (2) rekord jest replikowany przez potok Apache Kafka do PostgreSQL; (3) agent odpytuje bazę narzędziem `query_warehouse`; (4) złośliwy tekst trafia do okna uwagi modelu w odpowiedzi JSON-RPC; (5) model w kolejnym kroku ReAct emituje destrukcyjne polecenie narzędziowe.

Deterministyczny perymetr obronny. Skuteczna obrona przed atakiem pośredniego wstrzykiwania instrukcji nie może bazować na zawodnych dyrektywach języka naturalnego w promptach systemowych i wymaga twardej zapory architektonicznej w bramie MCP: (1) bezwzględnego ograniczenia uprawnień połączeń bazodanowych do trybu tylko do odczytu; (2) strukturalnej enkapsulacji rekordów wewnątrz jawnych znaczników XML (`<record>` / `</record>`), uniemożliwiającej modelowi ucieczkę z kontekstu danych; oraz (3) deterministycznej weryfikacji drzew składniowych AST generowanych kwerend SQL w odizolowanej puli procesowej, blokującej zapytania modyfikujące, wielozapytania oraz komentarze zaciemniające kod.

Przeprowadzona analiza teoretyczna wykazała, że o ile format JSON-RPC 2.0 optymalizuje sterowanie agentowe, o tyle efektywna i bezpieczna integracja ze składami danych wymaga wyeliminowania wąskich gardeł serializacji, paginacji oraz podatności wstrzykiwania poleceń. Zagadnienia te stanowią bezpośredni punkt wyjścia dla projektu technicznego referencyjnej bramy MCP, przedstawionego w Rozdziale **??**.

Rozdział 4

Projekt Systemu Integracji MCP z Hurtownią Danych

W niniejszym rozdziale przedstawiono całościowy projekt architektoniczny autorskiego systemu integracji wielkich modeli językowych z wielkoskalowymi, analitycznymi składami danych za pośrednictwem protokołu MCP [1]. W przemysłowych hurtowniach danych bezpośrednia interakcja autonomicznych agentów sztucznej inteligencji ze strukturami relacyjnymi i strumieniowymi napotyka fundamentalne bariery natury obliczeniowej, algorytmicznej oraz bezpieczeństwa. Do kluczowych wyzwań należą: ograniczona pojemność okna kontekstowego modelu językowego, drastyczna degradacja wydajności klasycznych metod stronicowania przy głębokich zapytaniach analitycznych, narzuty transmisyjne i tokenizacyjne kodowania obiektów binarnych w protokołach tekstowych, zjawisko blokowania pętli zdarzeń pod obciążeniem współbieżnym oraz podatność na ataki wstrzykiwania złośliwych instrukcji.

W celu rozwiązania zidentyfikowanych problemów sformalizowano trójwarstwową architekturę referencyjną bramy pośredniczącej (*Middleware Gateway*), opracowano formalny model adaptacyjnej paginacji kursorowej keyset sterowanej budżetem tokenów z algorytmem wariancji Welforda, zaprojektowano algorytm domknięcia grafowego relacji w progresywnym odkrywaniu schematu, sformułowano analityczny model punktu przecięcia transferu poza pasmem oraz zdefiniowano wielowarstwową architekturę bezpieczeństwa AST sprzężoną z wieloprocesową pulą wykonawczą.

Zgodnie z przyjętą dyscypliną metodologiczną, niniejszy rozdział odpowiada wyczerpująco na pytanie: *„Co i dlaczego zaprojektowano?"*, koncentrując się na architekturze logicznej, modelach formalnych, analizie złożoności obliczeniowej oraz kompromisach inżynierskich. Szczegóły fizycznej implementacji w środowisku uruchomieniowym zawarto w Rozdziale **??**, natomiast empiryczną weryfikację i wyniki pomiarowe przedstawiono w Rozdziale **??**.

4.1 Architektura logiczna i model referencyjny bramy pośredniczącej

4.1.1 Koncepcja trójwarstwowej architektury systemu

W tradycyjnych rozwiązaniach integrujących modele językowe ze środowiskami bazodanowymi powszechnie stosuje się model bezpośredniego sprzężenia, w którym agent posiada bezpośredni dostęp do silnika relacyjnego, a wygenerowane zapytania SQL są wykonywane bez uprzedniej inspekcji semantycznej. Podejście to rodzi krytyczne zagrożenia w obszarze integralności danych, braku determinizmu oraz wycieku poufnych poświadczeń. W niniejszej pracy zaprojektowano trójwarstwową architekturę referencyjną, wprowadzającą wyspecjalizowaną bramę pośredniczącą jako deterministyczny perymetr kontrolny pomiędzy przestrzenią wnioskowania probabilistycznego a deterministycznym składem danych.

Architekturę logiczną tworzą trzy ściśle wyodrębnione warstwy:

1. **Warstwa Hosta i Klienta LLM:** Obejmuje proces nadrzędny orkiestratora agentowego działającego zgodnie z paradygmatem ReAct [48], model językowy oraz klienta protokołu MCP. Warstwa ta odpowiada za dekompozycję zadania analitycznego w języku naturalnym, zarządzanie tokiem rozumowania oraz podejmowanie decyzji o wywołaniu dostępnych narzędzi;

2. **Warstwa Bramy Pośredniczącej (Middleware Gateway):** Autorski serwer pośredniczący protokołu MCP, stanowiący centralny węzeł koordynacji, walidacji i optymalizacji zapytań. Komponent ten hermetyzuje logikę dostępu do składu danych, zarządza asynchronicznymi pulami połączeń, realizuje dynamiczną inspekcję składniową zapytań w wydzielonej puli procesów potomnych, adaptacyjną paginację kursorową, wektorowe odkrywanie schematu z domknięciem grafowym oraz separację transmisji binarnej;

3. **Warstwa Składu Danych:** Niejednorodne środowisko przechowywania i przetwarzania danych analitycznych. Obejmuje relacyjno-wektorowy silnik PostgreSQL 16 z rozszerzeniem `pgvector` [20] (przechowujący tabele analityczne, metadane systemowe oraz indeksy HNSW) oraz rozproszony broker Apache Kafka [23] pracujący w bezstanowym trybie konsensusu KRaft, odpowiedzialny za buforowanie strumieni zdarzeń telemetrycznych.

4.1.2 Model separacji domen i bezpieczeństwa

Podstawowym aksjomatem projektowym przyjętym w architekturze bramy pośredniczącej jest bezwzględna separacja domen i odizolowanie modelu językowego od uprawnień infrastrukturalnych. W koncepcji tej model LLM traktowany jest jako komponent z natury niezaufany, podlegający rygorystycznej zasadzie najmniejszych uprawnień (ang. *Principle of Least Privilege*).

Separacja ról realizowana jest na trzech płaszczyznach architektonicznych:

- **Izolacja poświadczeń i połączeń sieciowych:** Klient MCP ani model językowy nie posiadają wiedzy o adresacji sieciowej, topologii klastra bazodanowego ani poświadczeniach uwierzytel-

niających (nazwa użytkownika, hasło, certyfikaty SSL). Wyłączne uprawnienia do nawiązywania fizycznych sesji TCP i zarządzania pulą połączeń posiada proces bramy middleware;

- **Kontrola interfejsu narzędziowego:** Komunikacja z agentem odbywa się wyłącznie poprzez z góry zdefiniowany zestaw narzędzi (*Tools*) i zasobów (*Resources*) protokołu MCP. Parametry wywołań narzędziowych podlegają walidacji typologicznej opartej o schemat JSON Schema, co zapobiega wstrzykiwaniu nieprawidłowo uformowanych argumentów;
- **Perymetr weryfikacji semantycznej:** Każde zapytanie analityczne generowane przez model jest traktowane jako potencjalnie wrogie i poddawane statycznej analizie składniowej AST przed skierowaniem do wykonania w silniku bazy danych.

4.1.3 Model przepływu komunikatów synchronicznych i asynchronicznych

Wymiana informacji pomiędzy poszczególnymi węzłami systemu rozgranicza dwa ortogonalne tryby komunikacji: synchroniczne przetwarzanie kwerend narzędziowych w protokole JSON-RPC 2.0 [**jsonrpc2010**, 4] (realizowane za pośrednictwem lokalnych łącz `stdio` lub protokołu HTTP POST) oraz asynchroniczny push zdarzeń w standardzie *Server-Sent Events*. Podczas gdy żądania narzędziowe realizują deterministyczny cykl zapytanie-odpowiedź, trwały kanał SSE służy do natychmiastowego wypychania powiadomień telemetrycznych buforowanych w brokerze Apache Kafka bezpośrednio do kontekstu agenta.

Strukturę logiczną komponentów bramy middleware, separację domen oraz model przepływu komunikatów przedstawiono na rysunku **??**.

Formalną specyfikację kontraktu interfejsu narzędziowego oraz zasobów udostępnianych modelowi językowemu przez bramę pośredniczącą przedstawiono w tabeli **??**.

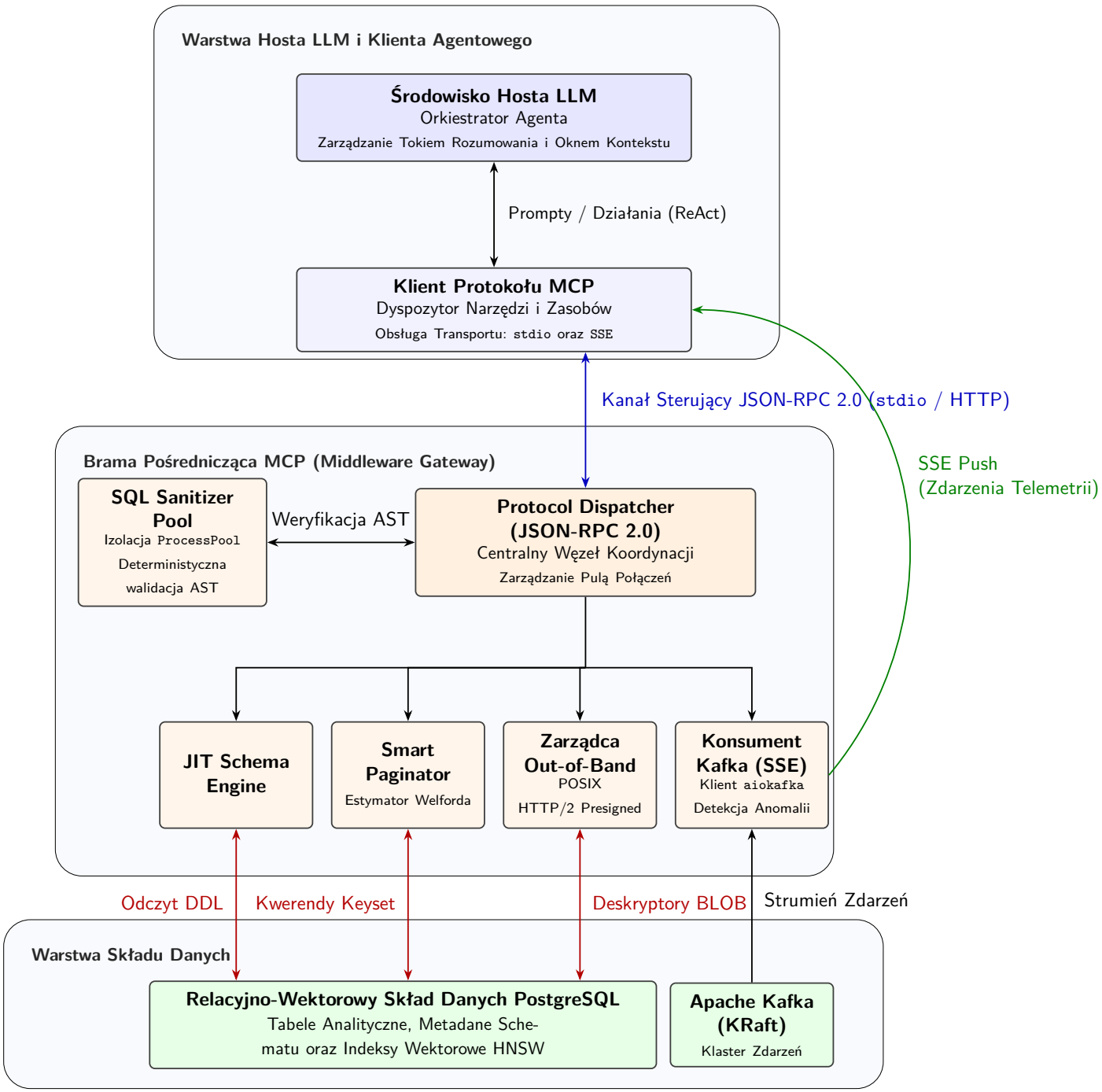
4.2 Formalizacja mechanizmu adaptacyjnej paginacji kursorowej (Smart Pagination)

Ze względu na asymptotyczną degradację wydajnościową metody przesunięcia pozycyjnego ($\mathcal{O}(\mathrm{N})$) oraz ryzyko występowania anomalii odczytu (rekordy fantomowe i pominięte), w architekturze bramy zaimplementowano deterministyczną paginację kursorową keyset.

4.2.1 Formalizacja techniki Keyset Pagination dla kluczy kompozytowych

W hurtowniach danych porządek analityczny definiowany jest najczęściej na wektorze atrybutów biznesowych (np. data operacji, status transakcji, kwota). Ponieważ kolumny te rzadko cechują się unikalnością, deterministyczny porządek wymaga sformalizowania wektora kompozytowego:

$$\mathrm{K} = (\mathrm{c}_1, \mathrm{c}_2, \ldots, \mathrm{c}_\mathrm{m}, \mathsf{id}) \tag{8}$$



Rysunek 1. Diagram komponentów logicznych trójwarstwowej architektury referencyjnej bramy pośredniczącej MCP

Tabela 3. Formalna specyfikacja kontraktu interfejsu narzędziowego i zasobów bramy pośredniczącej MCP

Narzędzie / Zasób	Parametry wejściowe	Zwracany ładunek (Payload)	Semantyka błędów JSON-RPC
`query_analytical_warehouse` *(Narzędzie)*	`sql_query`: string `cursor`: string (opt.) `budget_tokens`: int (opt.)	Kolekcja rekordów JSON, bezstanowy token `next_cursor` (hermetyzujący stan migawki `snapshot_id` oraz wektor kluczy), wskaźnik `has_more`, metryka `page_tokens`. Eksport, podtrzymywanie i import migawki transakcyjnej MVCC są realizowane przezroczysto przez warstwę serwera.	-32602 (Błąd AST / mutacja) -32001 (SnapshotExpired) -32002 (QueryTimeout)
`discover_schema` *(Narzędzie)*	`query`: string `top_k`: int (dom. 5) `include_ddl`: bool	Zsyntetyzowany kontekst DDL wyselekcjonowanych relacji wraz z tabelami asocjacyjnymi ($\mathrm{M}_{\mathrm{exp}}$)	-32602 (Niepoprawne parametry) -32003 (VectorStoreUnavailable)
`warehouse://blobs/{blob_id}` *(Zasób URI)*	Identyfikator zasobu `blob_id` oraz podpisany token w nagłówku	Surowy strumień binarny poza pasmem (deskryptor POSIX SHM lub HTTP/2 Presigned URL)	-32006 (BlobNotFound) -32007 (IntegrityMismatch)

gdzie $\mathrm{c_i}$ reprezentują kryteria biznesowe, a unikalny klucz główny id pełni funkcję deterministycznego rozjemcy (ang. *tie-breaker*), gwarantując ścisłą monotoniczność relacji porządkowej.

Dla jednorodnego kierunku sortowania (`ASC`) i kolumn z klauzulą `NOT NULL` predykat kursorowy wyraża się w zwartej postaci porównania wierszowego (ang. *Row-Value Comparison*):

$$(\mathrm{c_1}, \mathrm{c_2}, \ldots, \mathrm{c_m}, \mathsf{id}) > (\mathrm{v_1}, \mathrm{v_2}, \ldots, \mathrm{v_m}, \mathrm{v_{id}}) \tag{9}$$

gdzie wektor $\mathrm{v} = (\mathrm{v_1}, \ldots, \mathrm{v_m}, \mathrm{v_{id}})$ reprezentuje wartości graniczne ostatniego rekordu poprzedniej paczki. Konstrukcja ta jest bezpośrednio mapowana przez optymalizator bazy danych na przeszukanie indeksu B-drzewa (`Index Scan`) w stałym czasie indeksowym $\mathcal{O}(\log \mathrm{N} + \mathrm{K})$.

W przypadku ogólnym, dopuszczającym mieszane kierunki sortowania oraz obecność wartości pustych (`NULLS FIRST / LAST`), warunek kursorowy przyjmuje postać alternatywy dysjunktywnej (ang. *Disjunctive Normal Form*, DNF):

$$\bigvee_{\mathrm{i}=1}^{\mathrm{m}} \left(\bigwedge_{\mathrm{j}=1}^{\mathrm{i}-1} (\mathrm{c_j} = \mathrm{v_j}) \wedge \mathsf{CMP}_{\mathrm{i}}(\mathrm{c_i}, \mathrm{v_i}) \right) \vee \left(\bigwedge_{\mathrm{j}=1}^{\mathrm{m}} (\mathrm{c_j} = \mathrm{v_j}) \wedge (\mathsf{id} > \mathrm{v_{id}}) \right) \tag{10}$$

gdzie predykat $\mathsf{CMP}_{\mathrm{i}}(\mathrm{c_i}, \mathrm{v_i})$ uwzględnia kierunek porządku oraz politykę wartości pustych. Przykładowo dla kolumny rosnącej z klauzulą `NULLS LAST`:

$$\mathsf{CMP}_{\mathrm{i}}(\mathrm{c_i}, \mathrm{v_i}) = \begin{cases} (\mathrm{c_i} > \mathrm{v_i}) \vee (\mathrm{c_i}\ \mathsf{IS\ NULL}) & \text{dla } \mathrm{v_i} \neq \mathsf{NULL}, \\ (\mathrm{c_i}\ \mathsf{IS\ NULL}) & \text{dla } \mathrm{v_i} = \mathsf{NULL}. \end{cases} \tag{11}$$

Postać DNF jest poprawna semantycznie, jednak zapisana jako pojedyncza klauzula `WHERE` z operatorami `OR` nie jest rozpoznawana przez planer PostgreSQL jako warunek indeksowy: ewaluowana jest jako filtr na wierszach zwracanych przez skan indeksu od jego początku, co prowadzi do kosztu $\mathcal{O}(\mathrm{M})$ proporcjonalnego do liczby pominiętych krotek, a nie $\mathcal{O}(\log \mathrm{N} + \mathrm{K})$ (weryfikacja empiryczna w podrozdziale **??**). Dlatego w implementacji alternatywy równania (**??**) rozbito na rozłączne gałęzie koniunkcyjne $\mathrm{B}_1, \ldots, \mathrm{B}_\mathrm{b}$ – w szczególności alternatywa $(\mathrm{c_i} > \mathrm{v_i}) \vee (\mathrm{c_i}$ IS NULL) staje się dwiema osobnymi gałęziami. Każda gałąź jest wykonywana jako osobne zapytanie z własną klauzulą `ORDER BY` i `LIMIT`, a wyniki są łączone operatorem `UNION ALL` i ponownie ograniczane do K krotek:

$$\mathrm{Q}_{\mathsf{page}} = \mathrm{Top}_\mathrm{K}\left(\bigcup_{\mathrm{i}=1}^{\mathrm{b}} \mathrm{Top}_\mathrm{K}\big(\sigma_{\mathrm{B_i}}(\mathrm{R})\big)\right), \qquad \mathrm{b} \le 2\mathrm{m}+1, \tag{12}$$

gdzie Top_K oznacza K pierwszych krotek w porządku K, a m liczbę kolumn biznesowych. Ponieważ gałęzie są rozłączne, wynik nie zawiera duplikatów, a globalnych K pierwszych krotek zawsze leży w sumie K pierwszych krotek każdej z gałęzi, więc wynik jest identyczny jak dla jednej klauzuli DNF. Każda gałąź ma prosty predykat (np. $\mathrm{c}_1 < \mathrm{v}_1$ lub $\mathrm{c}_1 = \mathrm{v}_1 \wedge \mathrm{c}_2 > \mathrm{v}_2$) wykorzystywany jako warunek indeksowy, co daje łączny koszt $\mathcal{O}\big(\mathrm{b}\,(\log \mathrm{N}+\mathrm{K})\big) = \mathcal{O}(\log \mathrm{N}+\mathrm{K})$ dla stałego m, niezależny od głębokości stronicowania. Warunkiem koniecznym jest fizyczna obecność wielokolumnowego indeksu B-drzewa pokrywającego uporządkowaną sekwencję atrybutów $(\mathrm{c}_1, \mathrm{c}_2, \ldots, \mathrm{c}_\mathrm{m}, \mathrm{id})$ z tymi samymi kierunkami sortowania i polityką `NULLS`. Dekompozycję stosuje się tylko dla porządku wielokolumnowego i gdy kolumny sortowania występują na liście projekcji; w pozostałych przypadkach generowany jest pojedynczy predykat DNF.

4.2.2 Model bezstanowego kursora nieprzejrzystego (Opaque Cursor Token)

Bezpośrednie udostępnienie surowych parametrów kursorowych modelowi językowemu stwarza ryzyko manipulacji stanem logicznym oraz zafałszowania predykatów. Ponadto protokół MCP wymusza bezstanową semantykę wywołań narzędziowych. W bramie middleware zaimplementowano model bezstanowego kursora nieprzejrzystego.

Ładunek logiczny kursora definiuje krotka $\mathcal{C}$:

$$\mathcal{C} = \langle \mathrm{c}, \mathrm{v}, \mathrm{dir}, \mathcal{H}(\mathrm{Q}), \tau, \mathsf{snapshot_id} \rangle \tag{13}$$

gdzie c to wektor kolumn, v to wektor wartości progowych, dir oznacza kierunki sortowania, $\tau \in \mathbb{N}$ jest znacznikiem czasu utworzenia tokena, snapshot_id to identyfikator migawki transakcyjnej, a $\mathcal{H}(\mathrm{Q}) = \mathsf{SHA\text{-}256}(\mathrm{table} \,\|\, \mathrm{c} \,\|\, \mathrm{dir})$ stanowi kryptograficzny odcisk palca zapytania bazowego, uniemożliwiający użycie tokena w kontekście innej relacji.

Integralność stanu gwarantuje podpis HMAC-SHA256 z kluczem tajnym serwera $\mathrm{K}_{\mathsf{secret}}$:

$$\sigma = \mathsf{HMAC\text{-}SHA256}_{\mathrm{K}_{\mathsf{secret}}}(\mathsf{Base64Url}(\mathsf{JSON}(\mathcal{C}))) \tag{14}$$

Token przekazywany modelowi LLM przyjmuje zwartą postać:

$$\text{Token} = \text{Base64Url}(\text{JSON}(\mathcal{C})) \,\|\, "." \,\|\, \sigma \tag{15}$$

Weryfikacja tokena po stronie serwera odbywa się w stałym czasie funkcją `hmac.compare_digest`, co eliminuje podatność na ataki czasowe i gwarantuje bezstanowość bramy.

4.2.3 Izolacja snapshotowa w paginacji bezstanowej

Podczas wieloetapowego dialogu agenta równoległe procesy ETL mogą modyfikować stan hurtowni. Standardowy poziom izolacji *Read Committed* prowadzi do anomalii odczytu niepowtarzalnego oraz odczytu fantomów. Aby pogodzić bezstanową naturę wywołań `tools/call` z gwarancją spójności migawkowej, w architekturze bramy zaimplementowano mechanizm eksportu i importu snapshotów silnika PostgreSQL:

1. **Eksport migawki:** Przy pobieraniu pierwszej strony ($v_{\text{offset}} = 0$) serwer inicjuje transakcję `REPEATABLE READ` i wykonuje funkcję systemową `pg_export_snapshot()`. Wygenerowany identyfikator snapshot_id reprezentuje spójny stan widoczności MVCC, a dedykowane połączenie podtrzymujące snapshot jest rejestrowane w menedżerze `TransactionSnapshotManager` z czasem życia TTL_{snap} (domyślnie 300 s);

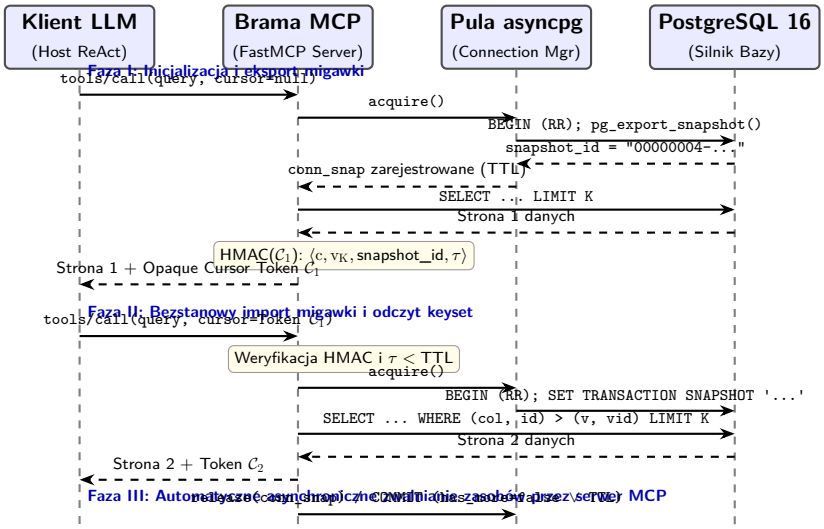
2. **Import w kolejnych krokach:** Zwracany identyfikator snapshot_id włączany jest do tokena $\mathcal{C}$ podpisanego HMAC. Przy kolejnym żądaniu brama pobiera dowolne wolne połączenie z puli `asyncpg`, otwiera transakcję i wykonuje klauzulę:

   $$\text{SET TRANSACTION SNAPSHOT snapshot_id;} \tag{16}$$

   zamrażając logiczny widok danych dokładnie w punkcie czasowym pierwszej iteracji;

3. **Automatyczne zwalnianie zasobów:** Zwolnienie zasobów podtrzymujących migawkę (`release(conn_snap)` oraz zatwierdzenie `COMMIT`) nie wymaga żadnej akcji ze strony klienta LLM i jest operacją asynchroniczną wyzwalaną automatycznie przez serwer MCP w dwóch deterministycznych sytuacjach: (a) po odebraniu ostatniej strony danych (has_more $=$ false), lub (b) po bezczynności sesji przekraczającej TTL_{snap} rejestrowanej przez `TransactionSnapshotManager`. W przypadku wywołania kwerendy z przedawnionym tokenem brama zwraca deterministyczny błąd JSON-RPC `SnapshotExpiredError` (kod `-32001`).

Mechanizm dynamicznej interakcji pomiędzy klientem LLM, bramą pośredniczącą, pulą połączeń a silnikiem PostgreSQL zilustrowano na diagramie sekwencji UML (rysunek **??**).



Rysunek 2. Diagram sekwencji UML przedstawiający cykl życia bezstanowej izolacji snapshotowej z eksportem i importem migawki transakcyjnej

4.2.4 Algorytm adaptacyjnej estymacji budżetu tokenów Welforda

Zróżnicowana długość rekordów tekstowych i struktur JSONB w analitycznych składach danych sprawia, że operowanie stałą liczbą wierszy K prowadzi do nieprzewidywalnego narzutu pamięciowego. Zamiast kosztownej tokenizacji BPE każdego rekordu w pętli, zaimplementowano stabilny numerycznie estymator Welforda [45], aktualizujący parametry rozkładu długości online:

$$\mu_{\mathrm{n}} = \mu_{\mathrm{n}-1} + \frac{\mathrm{x_n} - \mu_{\mathrm{n}-1}}{\mathrm{n}}, \qquad \mathrm{M}_{2,\mathrm{n}} = \mathrm{M}_{2,\mathrm{n}-1} + (\mathrm{x_n} - \mu_{\mathrm{n}-1})(\mathrm{x_n} - \mu_{\mathrm{n}}) \tag{17}$$

$$\mathrm{s}_{\mathrm{n}}^2 = \frac{\mathrm{M}_{2,\mathrm{n}}}{\mathrm{n}-1} \quad (\mathrm{n} > 1), \qquad \sigma_{\mathrm{n}} = \sqrt{\mathrm{s}_{\mathrm{n}}^2} \tag{18}$$

gdzie $\mathrm{x_n}$ oznacza zaobserwowaną średnią liczbę tokenów przypadających na wiersz w bieżącej paczce.

Dla zadanego budżetu tokenów $\mathrm{B_{tok}}$ oraz dopuszczalnego limitu rekordów $\mathrm{K_{max}}$, optymalny rozmiar kolejnej strony K wyznacza reguła konserwatywna:

$$\mathrm{K} = \max\left(1, \min\left(\mathrm{K_{max}}, \left\lfloor \frac{\mathrm{B_{tok}}}{\mu_{\mathrm{n}} + \mathrm{z} \cdot \sigma_{\mathrm{n}}} \cdot \alpha \right\rfloor\right)\right) \tag{19}$$

gdzie przyjęto współczynnik asymetrii $\mathrm{z} = 0{,}5$ oraz margines bezpieczeństwa na nagłówki ramki JSON-RPC $\alpha = 0{,}95$. Dla stanu początkowego $(\mathrm{n} = 0)$, przed zarejestrowaniem pierwszej próby długości wiersza, estymator przyjmuje heurystyczną wartość bazową μ_0 wyznaczoną ze średniej szerokości atrybutów w słowniku metadanych tabeli lub rozpoczyna ekstrakcję od zachowawczego rozmiaru paczki początkowej $\mathrm{K_{init}}$. Dzięki temu pełna tokenizacja BPE wykonywana jest dokładnie raz na zagregowanej odpowiedzi w pamięci operacyjnej.

4.3 Koncepcja progresywnego odkrywania schematu (JIT Schema Discovery)

4.3.1 Modelowanie baz danych jako skierowanych grafów bezskalowych

Wstrzyknięcie pełnego kodu DDL wszystkich tabel do promptu systemowego w hurtowni o $\mathrm{N_T} \geq 1000$ tabelach generuje narzut rzędu $\mathrm{T_{DDL}} \approx \mathrm{N_T} \cdot \bar{\mathrm{T}}_{\mathsf{table}} \approx 150\,000$ tokenów, wywołując degradację uwagi modelu (zjawisko *Lost in the Middle* [26]).

Rzeczywiste schematy relacyjne charakteryzują się topologią sieci bezskalowej [2]. Bazę danych modelujemy jako graf skierowany $\mathrm{G} = (\mathrm{V}, \mathrm{E})$, gdzie zbiór wierzchołków $\mathrm{V} = \{\mathrm{t}_1, \mathrm{t}_2, \ldots, \mathrm{t}_{\mathrm{N_T}}\}$ reprezentuje tabele, a zbiór krawędzi $\mathrm{E} \subseteq \mathrm{V} \times \mathrm{V}$ wynika z relacji kluczy obcych:

$$\mathrm{e} = (\mathrm{u}, \mathrm{v}) \in \mathrm{E} \iff \exists\ \mathsf{FK}\text{: tabela } \mathrm{u} \text{ odwołuje się do klucza głównego tabeli } \mathrm{v} \tag{20}$$

Rozkład stopni wierzchołków w takiej sieci opisuje zależność potęgowa $\mathrm{P(k)} \propto \mathrm{k}^{-\gamma}$ dla $2 < \gamma < 3$. Topologia ta implikuje istnienie nielicznych tabel centralnych o bardzo wysokiej łączności (tzw. węzłów typu *Hub*, np. tabele faktów) oraz licznych tabel peryferyjnych o niskim stopniu powiązań.

4.3.2 Problem utraty relacji asocjacyjnych w wyszukiwaniu leksykalno-semantycznym

Niezależne odpytywanie schematu metodami leksykalnymi (BM25) oraz semantycznymi (k-NN na wektorach `pgvector`) skutecznie odnajduje tabele odpowiadające pojęciom skrajnym z zapytania użytkownika, lecz systematycznie pomija pośredniczące tabele mostkujące relacje wielokrotne ($\mathrm{N} : \mathrm{M}$). W konsekwencji model językowy nie jest w stanie wygenerować poprawnych klauzul `JOIN` łączących wyselekcjonowane encje.

4.3.3 Model fuzji rankingów Reciprocal Rank Fusion

W celu połączenia zalet wyszukiwania leksykalnego i semantycznego bez konieczności kłopotliwej normalizacji niejednorodnych miar odległości zastosowano algorytm *Reciprocal Rank Fusion* (RRF) [9]:

$$\mathsf{RRF_Score}(\mathrm{d}) = \sum_{\mathrm{m} \in \{\mathsf{BM25},\mathsf{Dense}\}} \frac{1}{\mathrm{k}_{\mathsf{RRF}} + \mathrm{r_m(d)}} \tag{21}$$

gdzie $\mathrm{r_m(d)} \in \{1, 2, \ldots\}$ oznacza pozycję tabeli d w rankingu metody m, a stała wygładzająca wynosi $\mathrm{k}_{\mathsf{RRF}} = 60$.

4.3.4 Algorytm domknięcia grafowego (Foreign Key Graph Pruning)

W celu odzyskania brakujących tabel asocjacyjnych zaprojektowano algorytm *Foreign Key Graph Pruning*, którego przebieg formalizuje listing **??**.

```
Algorytm: Foreign Key Graph Pruning (q, k, G = (V,E), M_exp) -> S_DDL
1: T_cand ← Top-k relacji wedlug miary RRF_Score(q), B ← Counter(), G_undir ← Undirected(G)
2: for each unikalna para (u,v) ∈ T_cand × T_cand, u ≠ v:
3:     if istnieje sciezka pomiedzy u i v w G_undir:
4:         P ← ShortestPath(G_undir, u, v)
5:         if |P| = 3: // sciezka 2-krawedziowa [u,w,v] z wezlem posrednim w = P[1]
6:             if w ∉ T_cand: B[w] ← B[w] + 1
7: B_sel ← Wybierz M_exp najczestszych wezlow z B
8: T_final ← T_cand ∪ B_sel, E_active ← {(u,v) ∈ E | u,v ∈ T_final}
9: return S_DDL ← SynthesizeDDL(T_final, E_active)
```

Listing 2. Sformalizowany algorytm domknięcia grafowego Foreign Key Graph Pruning

Ograniczenie analizy do ścieżek o długości $\text{dist}(u, v) \leq 2$ (ścieżka trójelementowa $[u, w, v]$) wynika z faktu, że ponad 95% korporacyjnych złączeń wielotabelowych w architekturach ERP/CRM opiera się na bezpośrednich relacjach $N : M$ z pojedynczą tabelą asocjacyjną. Procedura SynthesizeDDL dokonuje rzutowania wyselekcjonowanego podgrafu na zwięzłą sygnaturę strukturalną SQL (nazwy atrybutów, typy danych oraz deklaracje ograniczeń kluczy obcych), z pominięciem zbędnych z perspektywy wnioskowania LLM metadanych fizycznych (takich jak parametry przestrzeni tabel, definicje indeksów wtórnych czy reguły buforowania).

Analiza złożoności obliczeniowej algorytmu domknięcia grafowego

Złożoność asymptotyczną procedury domknięcia determinuje liczba tabel kandydackich $k = |T_{\text{cand}}|$ oraz parametry grafu relacji $G = (V, E)$, gdzie $|V| = N_T$ oznacza liczbę tabel w schemacie, a $|E|$ liczbę ograniczeń kluczy obcych:

1. **Generowanie par kandydackich (Krok 2):** Liczba par wynosi $\binom{k}{2} = \frac{k(k-1)}{2} = \mathcal{O}(k^2)$;
2. **Wyszukiwanie najkrótszych ścieżek (Kroki 3–6):** Ograniczenie poszukiwań do głębokości $d \leq 2$ sprawia, że przeszukiwanie wszerz (BFS) dla ustalonej pary (u, v) operuje wyłącznie w bezpośrednim otoczeniu wierzchołków. W pesymistycznym przypadku złożoność przeszukania wynosi $\mathcal{O}(|V| + |E|)$, co implikuje łączny koszt $\mathcal{O}(k^2 \cdot (|V| + |E|))$. W sieciach bezskalowych o średnim stopniu wierzchołka $\bar{d} \ll |V|$ lokalny koszt redukuje się do $\mathcal{O}(k^2 \cdot \bar{d}^2)$;
3. **Selekcja węzłów mostkujących (Krok 7):** Zliczanie częstości wymaga czasu $\mathcal{O}(k^2)$, a ekstrakcja M_{exp} najczęstszych elementów przy użyciu kopca minimalnego zajmuje $\mathcal{O}(|\mathcal{B}| \log M_{\text{exp}}) \leq \mathcal{O}(k^2 \log M_{\text{exp}})$;
4. **Synteza DDL (Kroki 8–9):** Złożoność jest liniowa względem wyselekcjonowanego podgrafu: $\mathcal{O}(|T_{\text{final}}| + |E_{\text{active}}|)$, gdzie $|T_{\text{final}}| \leq k + M_{\text{exp}}$.

Całkowita złożoność czasowa wynosi zatem $\mathcal{O}(k^2 \cdot (|V| + |E|))$, a złożoność pamięciowa agregatora $\mathcal{B}$ wynosi $\mathcal{O}(k^2)$ przy pamięci grafu $\mathcal{O}(|V| + |E|)$. Ponieważ w praktyce $k \ll |V|$ ($k \in [3, 10]$ przy $|V| \geq 1000$), algorytm wykonuje się w czasie poniżej 2 ms.

Wartość parametru $\mathrm{M}_{\mathsf{exp}} = 3$ wyznaczono empirycznie na drodze badań ablacyjnych w podrozdziale **??**, gdzie stanowi ona optymalny punkt nasycenia czułości odzyskiwania relacji bez nadmiarowej inflacji tokenów kontekstu DDL.

4.4 Analityczny model transferu binarnego Out-of-Band (Crossover Model)

4.4.1 Problem narzutu kodowania binarnego w JSON-RPC

Jak wykazano w Rozdziale **??**, transmisja ładunków binarnych wewnątrz tekstowych ram JSON-RPC 2.0 z użyciem kodowania Base64 generuje teoretyczny narzut objętościowy na poziomie 33,33% oraz drastyczną fragmentację słownika tokenizatora BPE ($\mathrm{R}_{\mathsf{tok}} \approx 0{,}85 - 1{,}0$ tokena na bajt), prowadząc do natychmiastowego wyczerpania okna kontekstowego modelu językowego.

4.4.2 Separacja pasma sterującego od pasma danych

W celu eliminacji tego zjawiska zaprojektowano separację kanału sterującego od kanału danych:

1. **Pasmo sterujące:** Standardowy kanał JSON-RPC 2.0 (stdio lub HTTP POST), przesyłający wyłącznie metadane oraz lekki deskryptor zasobu o rozmiarze rzędu kilkuset bajtów;
2. **Pasmo danych:** Ortogonalny kanał binarny realizowany dwutorowo:
   - *Komunikacja lokalna:* współdzielony segment pamięci operacyjnej POSIX Shared Memory (`/dev/shm`) zapewniający bezkopiowy transfer międzyprocesowy (*Zero-Copy*);
   - *Komunikacja rozproszona:* bezpośredni transfer strumieniowy HTTP/2 z wykorzystaniem jednorazowych, podpisanych kryptograficznie uchwytów URI o ograniczonym czasie życia TTL.

Deskryptor OOB zwracany w kanale sterującym ma formalną postać:

$$\mathcal{D}_{\mathsf{OOB}} = \langle \mathsf{blob_id}, \mathsf{transport}, \mathrm{S}, \mathsf{SHA\text{-}256}(\mathrm{D}), \mathsf{uri}, \tau_{\mathsf{exp}} \rangle \tag{22}$$

gdzie S to rozmiar ładunku w bajtach, a skrót $\mathsf{SHA\text{-}256}(\mathrm{D})$ gwarantuje integralność danych odebranych poza pasmem MCP.

4.4.3 Analityczny model punktu przecięcia (Break-even Latency Model)

W celu automatycznego przełączania pomiędzy transferem wewnątrzpasłowym a poza pasmem sformułowano analityczny model całkowitego czasu propagacji w funkcji rozmiaru danych S oraz opóźnienia sieciowego RTT (ang. *Round-Trip Time*).

Całkowite opóźnienie transferu wewnątrzpasłowego $\mathrm{L}_{\mathsf{IB}}(\mathrm{S})$ obejmuje czas kodowania Base64 po stronie serwera, czas transmisji nadmiarowego ładunku przez łącze sieciowe oraz czas dekodowania

po stronie klienta:

$$\mathrm{L_{IB}}(\mathrm{S}) = \mathrm{S} \cdot \left(\frac{1}{\mathrm{v_{enc}}} + \frac{1 + \rho_{\mathsf{Base64}}}{\mathrm{B_{net}}} + \frac{1}{\mathrm{v_{dec}}} \right) \tag{23}$$

gdzie $\mathrm{B_{net}}$ oznacza przepustowość łącza sieciowego (B/s), $\rho_{\mathsf{Base64}} = 1/3$, natomiast $\mathrm{v_{enc}}$ i $\mathrm{v_{dec}}$ definiują wydajności operacji kodowania i dekodowania.

W mechanizmie transferu Out-of-Band model uwzględnia dwa fizyczne scenariusze komunikacyjne:

1. **Scenariusz lokalny (POSIX Shared Memory):** Opóźnienie $\mathrm{L_{OOB,\,shm}}(\mathrm{S})$ jest zdominowane przez stały narzut alokacji segmentu pamięci współdzielonej $\delta_{\mathsf{OOB,\,shm}}$ oraz przepustowość magistrali RAM $\mathrm{B_{shm}}$ ($\mathrm{B_{shm}} \gg \mathrm{B_{net}}$):

$$\mathrm{L_{OOB,\,shm}}(\mathrm{S}) = \delta_{\mathsf{OOB,\,shm}} + \frac{\mathrm{S}}{\mathrm{B_{shm}}} + \frac{\mathrm{S}}{\mathrm{v_{sha256}}} \tag{24}$$

2. **Scenariusz rozproszony (HTTP/2 Presigned URL):** Klient otrzymuje w kanale sterującym lekki deskryptor o narzucie δ_{desc}, po czym pobiera surowy obiekt binarny bezpośrednim żądaniem HTTP/2:

$$\mathrm{L_{OOB,\,net}}(\mathrm{S}, \mathrm{RTT}) = \delta_{\mathsf{desc}} + \mathrm{k_{RTT}} \cdot \mathrm{RTT} + \frac{\mathrm{S}}{\mathrm{B_{net}}} + \frac{\mathrm{S}}{\mathrm{v_{sha256}}} \tag{25}$$

gdzie $\mathrm{k_{RTT}} \geq 1$ oznacza liczbę rund sieciowych (dla zestawionego połączenia HTTP/2 $\mathrm{k_{RTT}} \approx 1$), a $\mathrm{S}/\mathrm{v_{sha256}}$ reprezentuje czas weryfikacji integralności skrótu SHA-256.

Punkt przecięcia definiuje wolumen danych, dla którego czas transmisji wewnątrzpasłowej zrównuje się z czasem transferu poza pasmem. Model rozgranicza analitycznie oba fizyczne środowiska wdrożeniowe:

1. Scenariusz lokalny IPC (POSIX Shared Memory): Próg opłacalności wyznacza przyrównanie opóźnienia transmisji tekstowej w kanale `stdio` z transferem w pamięci operacyjnej:

$$\mathrm{L_{IB}}(\mathrm{S^*_{shm}}) = \mathrm{L_{OOB,\,shm}}(\mathrm{S^*_{shm}}) \tag{26}$$

Przyjmując przepustowość lokalnego kanału tekstowego `stdio` jako $\mathrm{B_{stdio}}$, podstawienie zależności (**??**) oraz (**??**) prowadzi do analitycznego wzoru progu:

$$\mathrm{S^*_{shm}} = \frac{\delta_{\mathsf{OOB,\,shm}}}{\left(\frac{1}{\mathrm{v_{enc}}} + \frac{1}{\mathrm{v_{dec}}} + \frac{1+\rho_{\mathsf{Base64}}}{\mathrm{B_{stdio}}} \right) - \left(\frac{1}{\mathrm{B_{shm}}} + \frac{1}{\mathrm{v_{sha256}}} \right)} \tag{27}$$

Z uwagi na brak opóźnienia sieciowego ($\mathrm{RTT} = 0$) próg $\mathrm{S^*_{shm}}$ zależy wyłącznie od relacji stałego narzutu alokacji deskryptora $\delta_{\mathsf{OOB,\,shm}}$ do zysku na przepustowości magistrali RAM ($\mathrm{B_{shm}} \gg \mathrm{B_{stdio}}$) oraz eliminacji narzutu CPU kodowania Base64. Zapewnia to pełną spójność analityczną z wynikami benchmarków w podrozdziale **??**.

2. Scenariusz rozproszony (HTTP/2 Presigned URL): W środowisku sieciowym punkt przecięcia $\mathrm{S}^*(\mathrm{RTT})$ spełnia warunek $\mathrm{L}_{\mathrm{IB}}(\mathrm{S}^*) = \mathrm{L}_{\mathrm{OOB,\,net}}(\mathrm{S}^*, \mathrm{RTT})$. Podstawiając zależności (**??**) oraz (**??**) i rozwiązując równanie względem S^*, otrzymujemy formalną definicję progu sieciowego:

$$\mathrm{S}^*(\mathrm{RTT}) = \frac{\delta_{\mathrm{desc}} + \mathrm{k}_{\mathrm{RTT}} \cdot \mathrm{RTT}}{\left(\frac{1}{\mathrm{v}_{\mathrm{enc}}} + \frac{1}{\mathrm{v}_{\mathrm{dec}}} + \frac{\rho_{\mathrm{Base64}}}{\mathrm{B}_{\mathrm{net}}}\right) - \frac{1}{\mathrm{v}_{\mathrm{sha256}}}} \tag{28}$$

Formuła (**??**) definiuje próg opłacalności jako rosnącą, ściśle liniową funkcję opóźnienia sieciowego RTT. W środowisku lokalnym IPC stały narzut wywołania systemowego $\delta_{\mathrm{OOB,\,shm}}$ ustala niski próg opłacalności (**??**), faworyzując pamięć współdzieloną nawet dla niewielkich ładunków. W miarę wzrostu opóźnienia pakietu w sieciach rozproszonych (Datacenter LAN, WAN), dodatkowy narzut rundy sieciowej przesuwa punkt przecięcia w stronę większych wolumenów danych. Szczegółową weryfikację empiryczną i kalibrację parametrów modelu w heterogenicznych warunkach sieciowych przedstawiono w podrozdziale **??**.

4.5 Architektura bezpieczeństwa i izolacji współbieżnej pętli zdarzeń

4.5.1 Założenia perymetru bezpieczeństwa w systemach bazodanowych Agentic AI

Zgodnie z analizą wektorów ataków przedstawioną w podrozdziale **??** (podatności Confused Deputy, wstrzykiwanie promptu pierwszego i drugiego rzędu), model językowy w interakcji z hurtownią danych traktowany jest jako komponent z natury niezaufany. Brama pośrednicząca MCP stanowi w tej architekturze deterministyczny perymetr weryfikacyjny, zapobiegający wykonaniu destrukcyjnych lub nieautoryzowanych kwerend generowanych przez agenta.

4.5.2 Projekt wielopoziomowego perymetru ochronnego AST

Perymetr bezpieczeństwa oparto na deterministycznej analizie drzewa składniowego AST:

1. **Izolacja pojedynczej instrukcji:** Wymuszenie obecności dokładnie jednego węzła w drzewie AST ($|\mathrm{Statements}(\mathcal{T}_{\mathrm{AST}})| = 1$), co bezwzględnie blokuje kwerendy wielokrotne rozdzielane średnikiem;

2. **Niezmiennik czystego odczytu (Pure Read-Only):** Węzeł główny musi należeć do operacji odczytu: $\mathrm{Root}(\mathcal{T}_{\mathrm{AST}}) \in \{\mathrm{Select}, \mathrm{SetOperation}\}$, gdzie `SetOperation` obejmuje operacje teoriomnogościowe (`UNION ALL`, `INTERSECT`, `EXCEPT`);

3. **Rekurencyjne blokowanie operacji mutujących:** Wszystkie węzły drzewa $\mathcal{T}_{\mathrm{AST}}$ podlegają przeszukaniu: $\forall \mathrm{v} \in \mathrm{V}(\mathcal{T}_{\mathrm{AST}}) : \mathrm{Type}(\mathrm{v}) \notin \mathcal{F}_{\mathrm{forbidden}}$, gdzie zbiór operacji zabronionych $\mathcal{F}_{\mathrm{forbidden}}$ obejmuje instrukcje modyfikacji danych i schematu: `Insert`, `Update`, `Delete`, `Merge`, `Drop`, `Create`, `Alter`, `TruncateTable`, `Grant`, `Revoke`, `Copy`, `Command`, `Execute`, `Pragma`, `Into`, `Analyze`, `Set` oraz `Transaction`. Reguła ta uniemożliwia ukrycie mutacji wewnątrz klauzul CTE (np. `WITH d AS (DELETE ...) SELECT * ...`);

4. **Enkapsulacja rekordów przeciw wstrzykiwaniu drugiego rzędu:** Dane wynikowe podlegają oczyszczeniu ze znaków sterujących XML i tokenów specjalnych LLM (<|im_start|>, [INST], <<SYS>>), po czym są zamykane w bloku izolacyjnym:

```
<untrusted_db_record role="data_only" execution_forbidden="true"> [rekordy JSON
    ] </untrusted_db_record>
```

zapobiegając interpretacji danych jako poleceń sterujących modelu w kolejnych iteracjach wnioskowania.

4.5.3 Model izolacji współbieżnej pętli zdarzeń

W systemach wieloagentowych brama pośrednicząca obsługuje setki współbieżnych połączeń asynchronicznych. Parsowanie gramatyczne złożonych zapytań SQL do postaci drzewa AST jest operacją silnie obciążającą procesor. W środowisku jednowątkowej pętli zdarzeń `asyncio`, synchroniczne parsowanie kwerend blokuje wątek wykonawczy, uniemożliwiając obsługę ramek I/O i prowadząc do lawinowego narostu opóźnień.

W celu eliminacji tego zjawiska pętla `asyncio` deleguje zadania parsowania AST do dedykowanej puli procesów `ProcessPoolExecutor` (powoływanej z kontekstem `forkserver`):

$$\text{Future}_{\text{validation}} = \text{loop.run_in_executor}(\text{Pool}_{\text{workers}}, \text{ValidateAST}, \mathrm{Q}) \tag{29}$$

Wybór kontekstu `forkserver` (w miejsce domyślnego mechanizmu `fork` lub `spawn`) podyktowany jest charakterystyką asynchronicznego środowiska wykonawczego: mechanizm `fork` w wielowątkowym procesie Pythona integrującym niskopoziomowe biblioteki w języku C (sterowniki bazodanowe, parser SQL) stwarza krytyczne ryzyko zakleszczeń, natomiast `spawn` generuje niedopuszczalny narzut czasowy zimnego startu nowego interpretera. Kontekst `forkserver` gwarantuje pełną izolację pamięciową procesów potomnych przy zachowaniu submilisekundowego czasu ich powoływania. Zwolnienie wątku następuje natychmiast po przekazaniu deskryptora zadania do nieblokującej kolejki IPC, co redukuje narzut pętli do stałego czasu przełączenia kontekstu $\mathcal{O}(1)$. Obliczenia realizowane są równolegle na fizycznych rdzeniach procesora, gwarantując pełną skalowalność bramy pod obciążeniem wieloagentowym.

Sformalizowany model architektoniczny bramy, reguły paginacji kursorowej, algorytm domknięcia grafowego oraz model transferu Out-of-Band stanowiły bezpośrednią podstawę implementacji prototypu programistycznego i aparatury badawczej, których opis środowiskowy przedstawiono w Rozdziale **??**.

Rozdział 5

Implementacja i Środowisko Badawcze

W niniejszym rozdziale przedstawiono szczegółowy opis implementacji programistycznej referencyjnej bramy pośredniczącej protokołu MCP [1], fizyczną architekturę wielousługowego środowiska kontenerowego oraz konstrukcję aparatury badawczej przygotowanej do weryfikacji empirycznej. W paradygmacie nauk o projektowaniu (*Design Science Research*, DSR) implementacja stanowi zmaterializowany artefakt badawczy, którego celem jest urzeczywistnienie modeli formalnych i architektonicznych sformułowanych w Rozdziale **??**.

Kluczowym założeniem inżynierskim było zapewnienie pełnej odtwarzalności środowiska oraz determinizmu pomiarowego. W tym celu zaimplementowano modularną bramę w języku Python 3.12 opartą na asynchronicznym frameworku FastMCP [27], zintegrowaną z silnikiem bazodanowym PostgreSQL 16 wyposażonym w rozszerzenie wektorowe `pgvector` [20] oraz rozproszonym brokerem komunikatów Apache Kafka [23] w trybie bezstanowym KRaft.

Zgodnie z zasadą separacji zagadnień, niniejszy rozdział koncentruje się ściśle na dokumentacji fizycznej budowy systemu, niskopoziomowych decyzjach systemowych oraz aparaturze metrologicznej, odpowiadając na pytanie: *„Jak fizycznie skonstruowano system oraz środowisko eksperymentalne?"*. Wszystkie ilościowe wyniki pomiarów, ich ewaluacja efektywnościowa oraz testy hipotez statystycznych zostały wyodrębnione i przedstawione w Rozdziale **??**.

5.1 Architektura środowiska uruchomieniowego i konteneryzacja

W celu zagwarantowania determinizmu sprzętowo-programowego, izolacji procesowej oraz powtarzalności środowiska testowego, całość infrastruktury badawczej została skonteneryzowana przy użyciu platformy Docker oraz narzędzia orkiestracji Docker Compose. Topologia środowiska obejmuje węzeł relacyjno-wektorowy PostgreSQL 16 z rozszerzeniem `pgvector`, rozproszony broker Apache Kafka działający w trybie KRaft, współdzielony segment pamięci operacyjnej (`/dev/shm`) dedykowany dla transferu obiektów binarnych poza pasmem oraz autorską bramę pośredniczącą FastMCP. Zestawienie składowych infrastruktury oraz ich kluczowe parametry konfiguracyjne podsumowano w tabeli **??**.

Tabela 4. Parametry konfiguracyjne usług kontenerowych środowiska badawczego

Usługa kontenerowa	Obraz bazowy	Kluczowe parametry konfiguracyjne
PostgreSQL 16	pgvector/pgvector:pg16	shared_buffers=1GB, work_mem=32MB, max_connections=200, port 5432, rozszerzenie vector (HNSW, $d = 384$, vector_cosine_ops)
Apache Kafka	apache/kafka:latest	Tryb KRaft (broker,controller), 3 partycje fizyczne, quorum voter localhost:9093, listener PLAINTEXT://:9092
Segment /dev/shm	Pamięć RAM hosta	POSIX Shared Memory, bezkopiowy odczyt buforów binarnych poza pasmem JSON-RPC
Inicjalizacja danych	Python 3.12	generate_enterprise_schema.py (1000 tabel, topologia bezskalowa), seed_1m_records.py (1 mln rekordów)

Węzeł bazodanowy oparto na zoptymalizowanym obrazie kontenerowym pgvector:pg16 (kontener mcp_postgres). Silnik PostgreSQL 16 poddano precyzyjnemu strojeniu niskopoziomowemu na poziomie parametrów jądra bazy:

- shared_buffers=1GB: główny bufor pamięci podręcznej stron relacyjnych i indeksowych, alokowany ze struktur pamięci współdzielonej systemu operacyjnego. Wartość 1 GB odpowiada dokładnie 25% pamięci RAM węzła badawczego (4 GB), co eliminuje zjawisko podwójnego buforowania pomiędzy buforem stron jądra Linuksa a buforami silnika PostgreSQL;
- work_mem=32MB: pamięć operacyjna przydzielana pojedynczym operacjom sortowania oraz tablicom haszującym. Podniesienie tej wartości z domyślnych 4 MB gwarantuje, że wielokolumnowe sortowania kursorowe keyset wykonywane są w całości w przestrzeni RAM, zapobiegając zrzucaniu struktur tymczasowych na dysk;
- max_connections=200: limit współbieżnych połączeń klienckich, zapobiegający wyczerpaniu gniazd sieciowych podczas równoległego obciążania bramy przez wieloprocesowy harness pomiarowy;
- pgdata: trwały wolumen blokowy podmontowany w ścieżce /var/lib/postgresql/data, całkowicie izolujący operacje wejścia/wyjścia (I/O) bazy od ulotnej warstwy nakładkowej systemu plików kontenera.

Drugim kluczowym komponentem infrastruktury jest broker Apache Kafka (kontener mcp_kafka), uruchomiony w bezstanowym trybie KRaft, eliminującym narzuty i niestabilności związane z zewnętrznym klastrem Apache ZooKeeper. Węzeł łączy funkcje brokera oraz kontrolera metadanych. Skonfigurowano dwa odrębne punkty nasłuchu: PLAINTEXT://:9092 dla klientów oraz CONTROLLER://:9093 dla protokołu konsensusu Raft. Na brokerze utworzono temat telemetry_test z 3 partycjami fizycznymi, co umożliwia równoległe testowanie asynchronicznych konsumentów strumieniowych.

W celu weryfikacji algorytmów odkrywania schematów w realistycznej skali korporacyjnej zaimplementowano generator `testbed/init-scripts/generate_enterprise_schema.py`, powoływany automatycznie przy starcie bazy. Generator tworzy syntetyczny magazyn danych złożony z 1000 relacji SQL podzielonych równomiernie na 5 domen biznesowych: finanse (`finance`), gospodarka magazynowa (`inventory`), kadry i płace (`hr`), zarządzanie relacjami z klientami (`crm`) oraz telemetria przemysłowa (`telemetry`) — po 200 tabel w każdej domenie (20 encji bazowych powiązanych z 10 kwalifikatorami operacyjnymi). Topologia relacji została oparta na modelu sieci bezskalowej (*Scale-Free Network Topology*), charakteryzującej się rozkładem potęgowym stopni wierzchołków grafu bazy. W strukturze wyznaczono 20 centralnych tabel typu Hub (po 4 w domenie, m.in. `finance_ledger_entries_daily`, `crm_client_accounts_enterprise`, `inventory_purchase_orders_raw_materials`, `telemetry_host_servers_datacenter_a`) o stopniu wierzchołka w przedziale 15–40. Strukturę uzupełniają tabele mostkujące relacje typu $\mathrm{N} : \mathrm{M}$ (36 na domenę), tabele satelickie pierwszego poziomu (72 na domenę) oraz łańcuchy satelickie 2-i 3-poziomowe (88 na domenę).

Metadane relacyjne grafu bazy utrwalono w dedykowanej tabeli systemowej `public.schema_foreign_keys`, przechowującej 1174 zdefiniowane klucze obce wraz z powiązanymi kolumnami złączeń w formacie `JSONB` oraz indeksami B-drzewa na kolumnach `source_table` i `target_table`. Semantyczne opisy tabel umieszczono w relacji `public.schema_embeddings`, w której kolumna `embedding` przechowuje gęste reprezentacje wektorowe o wymiarowości $\mathrm{d} = 384$, wygenerowane modelem `sentence-transformers/all-MiniLM-L6-v2` [41]. Na kolumnie tej utworzono wielowarstwowy indeks grafowy HNSW [29] z operatorem odległości kosinusowej, gwarantujący logarytmiczną złożoność wyszukiwania najbliższych sąsiadów $\mathcal{O}(\log \mathrm{N})$.

Dla potrzeb ewaluacji paginacji kursorowej i transferu obiektów binarnych przygotowano relację `public.analytics_orders`, zasilaną przez skrypt `seed_1m_records.py` wolumenem 1 000 000 rekordów przy użyciu binarnego strumieniowania `copy_records_to_table` w paczkach po 25 000 wierszy. Tabela zawiera kolumny zróżnicowanych typów oraz kolumnę `document_base64` o ładunkach 10–30 KB, wymuszającą w silniku PostgreSQL pozawierszowe przechowywanie atrybutów w strukturach TOAST (*The Oversized-Attribute Storage Technique*). Do obsługi transferu OOB skonfigurowano współdzielony segment pamięci operacyjnej hosta podmontowany pod ścieżką `/dev/shm`.

5.2 Implementacja autorskiego serwera MCP (Middleware Gateway)

Podstawowym komponentem warstwy pośredniczącej jest autorski serwer protokołu Model Context Protocol, zaimplementowany w module `src/mcp_server/server.py`. Do budowy serwera wykorzystano framework FastMCP [27] w środowisku Python 3.12, w pełni zintegrowany z natywną pętlą zdarzeń `asyncio`. Komunikację z silnikiem PostgreSQL oparto na binarnym sterowniku `asyncpg` [28], który realizuje protokół Frontend/Backend v3.0, omijając narzuty synchronicznego interfejsu DB-API 2.0 oraz biblioteki C `libpq`.

Zarządzanie połączeniami zrealizowano za pomocą asynchronicznej puli połączeń `asyncpg.Pool`, konfigurowanej w klasie `DatabaseConfig` (`min_size=2`, `max_size=10`), wyposażonej w procedurę rezerwowego uwierzytelniania środowiskowego. Cykl życia serwera kontrolowany jest przez asynchroniczny menedżer kontekstu `lifespan`. Odpowiada on za deterministyczne utworzenie puli połączeń bazodanowych, uruchomienie asynchronicznego konsumenta Kafka w tle oraz powołanie wieloprocesowej puli wykonawczej `ProcessPoolExecutor` z bezpiecznym kontekstem `forkserver` dla analizatora zapytań SQL. Podczas procedury zatrzymania menedżer gwarantuje sekwencyjne zatrzymanie konsumenta strumieniowego, zamknięcie procesów roboczych puli oraz zwolnienie wszystkich aktywnych połączeń bazodanowych.

Fizyczna implementacja publicznych interfejsów serwera w `src/mcp_server/server.py` stanowi bezpośrednie urzeczywistnienie formalnego kontraktu bramy zdefiniowanego w Rozdziale **??** (tabela **??**). Zamiast rozproszonych, niejednorodnych narzędzi RPC zaimplementowano spójny interfejs narzędziowy i zasobowy:

1. **Narzędzie wykonawcze** `@mcp.tool() query_analytical_warehouse`**:** Uniwersalny punkt wejścia do analitycznego odpytywania bazy danych, przyjmujący parametry: `sql_query` (treść zapytania), `cursor` (opcjonalny bezstanowy token Opaque) oraz `budget_tokens` (deklarowany budżet tokenów). Funkcja ta stanowi kompozycję wyspecjalizowanych podsystemów:
   - weryfikuje zapytanie w zaporze syntaktycznej AST `SqlSanitizer` oddelegowanej do puli procesów potomnych, egzekwując niezmiennik odczytu i brak mutacji w klauzulach CTE;
   - przekazuje zapytanie do silnika paginacji `pagination.py`, normalizując porządek sortowania kursorowego;
   - odpytuje katalog systemowy za pośrednictwem modułu `IndexCatalogGuard`, weryfikując kompatybilność kluczy sortowania z indeksami B-drzewa;
   - transparentnie zarządza migawką MVCC poprzez klasę `TransactionSnapshotManager` (`pg_export_snapshot` oraz `SET TRANSACTION SNAPSHOT`), gwarantując izolację odczytu pomiędzy bezstanowymi wywołaniami;
   - stosuje adaptacyjny estymator Welforda `AdaptiveTokenBudgetEstimator` do dynamicznego wyznaczenia bezpiecznej wielkości paczki;
   - zwraca rekordy zabezpieczone w strukturalnym bloku XML `<untrusted_db_record>`, podpisany kryptograficznie token `next_cursor`, wskaźnik kontynuacji `has_more` oraz metrykę `tokens_used`;

2. **Narzędzie odkrywania schematu** `@mcp.tool() discover_schema`**:** Odpowiada za progresywne odkrywanie struktur bazodanowych w czasie rzeczywistym. Przyjmuje zapytanie w języku naturalnym (`query`), pożądaną liczbę relacji bazowych (`top_k`) oraz flagę dołączania DDL (`include_ddl`). Narzędzie bezpośrednio wywołuje metodę `SchemaEngine.discover_schema_with_graph_pruning`, dokonując hybrydowej fuzji RRF oraz domknięcia grafowego kluczy obcych;

3. **Zasób binarny** `@mcp.resource("warehouse://blobs/{blob_id}")`**:** Zapewnia asynchroniczny odczyt obiektów wielkich poza pasmem JSON-RPC. W zależności od zarejestrowanego deskryptora zwraca strumień danych bezpośrednio z segmentu pamięci współdzielonej POSIX (`/dev/shm`) lub udostępnia podpisany adres HTTP/2 Presigned URL;

4. **Kanał telemetryczny Push w standardzie SSE:** Asynchroniczny strumień powiadomień zasilany bezpośrednio przez bibliotekę `aiokafka` nasłuchującą brokera. Wykrycie anomalii telemetrycznej (`CRITICAL_ANOMALY`) powoduje natychmiastową emisję zdarzenia push do połączonych klientów, całkowicie eliminując konieczność jałowego odpytywania (ang. *polling*).

Framework FastMCP dokonuje automatycznej kompilacji adnotacji typów języka Python oraz modeli Pydantic do ustandaryzowanego schematu JSON Schema zgodnie ze specyfikacją JSON-RPC 2.0 [**jsonrpc2010**], zapewniając ścisłą walidację typologiczną parametrów wejściowych przed uruchomieniem kodu wykonawczego serwera.

5.3 Silnik semantycznego odkrywania schematu

Fizyczną implementację mechanizmu progresywnego odkrywania schematów relacyjnych w architekturze JIT zrealizowano w module `src/mcp_server/schema_engine.py` (klasa `SchemaEngine`), stanowiącym implementację założeń projektowych z podrozdziału **??**.

Klasa `SchemaEngine` integruje cztery strategie wyszukiwania tabel w korporacyjnym magazynie danych:

1. **Wyszukiwanie leksykalne BM25 (**`retrieve_bm25`**):** Oparte na bibliotece `rank_bm25` (`BM25Okapi`). Indeks leksykalny budowany jest w pamięci RAM na podstawie znormalizowanych nazw tabel, kolumn pobranych z widoku katalogowego `information_schema.columns` oraz komentarzy biznesowych. Tokenizacja zapytań uwzględnia podział po znakach podkreślenia (_) oraz redukcję wielkości liter;

2. **Gęste wyszukiwanie wektorowe pgvector (**`retrieve_dense`**):** Zapytanie agenta wektoryzowane jest modelem `all-MiniLM-L6-v2` ($\mathrm{d} = 384$) z normalizacją euklidesową. Sterownik `asyncpg` przekazuje wektor do bazy danych, gdzie kwerenda z operatorem odległości kosinusowej <=> przeszukuje indeks HNSW w tabeli `public.schema_embeddings`;

3. **Hybrydowa fuzja rang RRF (**`retrieve_hybrid_rrf`**):** Fuzję rankingów zrealizowano zgodnie z modelem formalnym RRF sformułowanym w równaniu (**??**) przy stałej wygładzającej $\mathrm{k}_{\mathrm{RRF}} = 60$. Połączenie metod leksykalnej i gęstej eliminuje ich jednostronne niedoskonałości, zapewniając jednoczesną precyzję dopasowania kodów technicznych oraz odporność na synonimię pojęciową;

4. **Domknięcie grafowe z przycinaniem (**`discover_schema_with_graph_pruning`**):** Autorski algorytm rozwiązujący problem pomijania tabel pośredniczących w złączeniach relacyjnych.

Algorytm pobiera kandydatów z metody hybrydowej RRF, po czym analizuje topologię grafu relacji NetworkX (`nx.DiGraph`) zbudowanego na podstawie metadanych tabeli `public.schema_foreign_keys`.

Implementację kluczowego fragmentu integracji grafu NetworkX z bazą danych przedstawiono w listingu **??**.

```
async def discover_schema_with_graph_pruning(
    self, query: str, top_k: int = 5, max_expanded_tables: int = 3, pool:
    Optional[asyncpg.Pool] = None
) -> str:
    active_pool = self._get_active_pool(pool)
    base_schemas = await self.retrieve_hybrid_rrf(query=query, top_k=top_k,
    pool=active_pool)
    base_tables = [item["table_name"] for item in base_schemas]

    G = (await self._ensure_relation_graph(active_pool)).to_undirected(
    as_view=True)
    bridge_counter: Counter[str] = Counter()

    # Wykrywanie wezlow mostkujacych na sciezkach o dlugosci dist <= 2
    for u, v in itertools.combinations(base_tables, 2):
        if u in G and v in G and nx.has_path(G, u, v):
            path = nx.shortest_path(G, source=u, target=v)
            if len(path) == 3 and path[1] not in base_tables:
                bridge_counter[path[1]] += 1

    selected_bridges = [node for node, _ in bridge_counter.most_common(
    max_expanded_tables)]
    final_tables = base_tables + [b for b in selected_bridges if b not in
    base_tables]

    # Pobranie metadanych kolumn i aktywnych kluczy obcych [...]
    return await self._synthesize_ddl_context(active_pool, query,
    final_tables, selected_bridges)
```

Listing 3. Algorytm domknięcia grafowego z wykrywaniem tabel mostkujących w SchemaEngine

Wygenerowany zwarty blok DDL zawiera definicje kolumn, typów, kluczy głównych oraz explicite wyspecyfikowane klauzule relacji (`FOREIGN KEY ... REFERENCES`), co dostarcza modelowi językowemu niezbędny i nieskażony szumem kontekst relacyjny.

5.4 Mechanizm adaptacyjnej paginacji kursorowej

Fizyczną realizację deterministycznej paginacji kursorowej, zaprojektowanej formalnie w podrozdziale **??**, zaimplementowano w module `src/mcp_server/pagination.py` w funkcji `fetch_paginated_data`.

Architektura podsystemu paginacji opiera się na pięciu filarach inżynierskich:

1. **Wielokolumnowe zapytania Keyset (`build_keyset_query`):** Paginacja realizowana jest względem kompozytowego wektora atrybutów $\mathrm{K} = (\mathrm{c}_1, \ldots, \mathrm{c}_\mathrm{m}, \mathrm{id})$, gdzie klucz główny `id` stanowi deterministyczny rozjemca, eliminujący pomijanie rekordów o identycznych wartościach sortowania. W dialekcie PostgreSQL zapytanie konstruowane jest w postaci porównania krotek dla jednorodnych kierunków:

```
WHERE ("c1", "c2", "id") > ($1, $2, $3)
ORDER BY "c1" ASC, "c2" ASC, "id" ASC LIMIT $4;

```

   Dla kierunków mieszanych i klauzul `NULLS FIRST/LAST` predykat ma postać dysjunkcyjną (DNF). Zapisany jako pojedyncze wyrażenie z operatorami `OR` jest jednak przez planer PostgreSQL ewaluowany jako filtr po skanie indeksu od początku, co daje koszt $\mathcal{O}(\mathrm{M})$. Dlatego funkcja `build_keyset_query` (parametr `use_union_all`, domyślnie włączony w `fetch_paginated_data`) rozbija DNF na rozłączne gałęzie z prostymi predykatami, wykonywane jako osobne zapytania z własnym `ORDER BY ... LIMIT` i łączone przez `UNION ALL`:

```
SELECT * FROM (
  (SELECT ... WHERE "created_at" < $1 ORDER BY ... LIMIT $4)
  UNION ALL (SELECT ... WHERE "created_at" IS NULL ORDER BY ... LIMIT
   $4)
  UNION ALL (SELECT ... WHERE "created_at" = $1 AND "amount" > $2 ORDER
    BY ... LIMIT $4)
  UNION ALL (SELECT ... WHERE "created_at" = $1 AND "amount" IS NULL
   ORDER BY ... LIMIT $4)
  UNION ALL (SELECT ... WHERE "created_at" = $1 AND "amount" = $2 AND "
   id" > $3 ORDER BY ... LIMIT $4)
) AS keyset_page ORDER BY ... LIMIT $4;

```

   Każda gałąź wykorzystuje warunek indeksowy, a węzeł `Merge Append` scala posortowane wyniki, co zapewnia stałą złożoność $\mathcal{O}(\log \mathrm{N} + \mathrm{K})$ niezależną od głębokości stronicowania (podrozdział **??**). Dekompozycja jest stosowana tylko dla porządku wielokolumnowego i gdy kolumny sortowania są obecne na liście projekcji; w przeciwnym razie generowany jest pojedynczy predykat DNF, a dla sortowania po samym `id` zapytanie pozostaje niezmienione;

2. **Bezstanowy kursor Opaque z podpisem HMAC-SHA256:** Token kursora nie ujawnia wewnętrznych struktur bazy. Funkcja `encode_opaque_cursor` serializuje ładunek JSON zawierający specyfikację kolumn, wektor wartości progowych v, skrót zapytania $\mathcal{H}(\mathrm{Q})$, znacznik czasu τ oraz identyfikator migawki `snapshot_id`. Pakiet jest kodowany w standardzie Base64URL i zabezpieczony kryptograficznym podpisem HMAC-SHA256 z użyciem serwerowego klucza `CURSOR_HMAC_SECRET`. Funkcja `decode_opaque_cursor` weryfikuje integralność tokena w stałym czasie funkcją `hmac.compare_digest`;

3. **Ochrona planera zapytań i katalogu systemowego (`IndexCatalogGuard`):** Wykonanie sortowania keyset na nieindeksowanych kolumnach zmusza silnik bazy do sekwencyjnego skanowania relacji (*Seq Scan*) i zrzucania buforów sortowania na dysk. Moduł `IndexCatalogGuard` asynchronicznie odpytuje widoki katalogowe (`pg_indexes`, `pg_am`, `pg_class`, `pg_attribute`) z 60-sekundowym buforem podręcznym TTL, weryfikując, czy kolumny sortowania tworzą lewostronny prefiks aktywnego indeksu B-drzewa (`amname = 'btree'`). W przypadku braku indeksu moduł prewencyjnie dławi wielkość paczki do 50 wierszy, chroniąc bufor `shared_buffers` oraz przestrzeń `work_mem` przed przeciążeniem;

4. **Bezstanowa izolacja snapshotowa (`TransactionSnapshotManager`):** Standardowy poziom izolacji *Read Committed* stwarza w wieloetapowej konwersacji agenta ryzyko anomalii niepowtarzalnych odczytów i rekordów fantomowych. Klasa `TransactionSnapshotManager` przy pobieraniu pierwszej strony otwiera transakcję `REPEATABLE READ`, eksportuje migawkę procedurą `pg_export_snapshot()` i osadza jej identyfikator w tokenie HMAC. Kolejne żądania importują zamrożony stan MVCC klauzulą `SET TRANSACTION SNAPSHOT`, zapewniając spójność odczytów przez zadany czas życia ($\mathrm{TTL} = 300$ s);

5. **Adaptacyjny estymator budżetu tokenów Welforda (`AdaptiveTokenBudgetEstimator`):** W miejsce kosztownego kodowania BPE w pętli per-wiersz zastosowano algorytm online Welforda [45], śledzący średnią liczbę tokenów na wiersz i jej wariancję. Pozwala to na dynamiczny dobór wielkości paczki do budżetu $\mathrm{B}_{\mathrm{tok}}$ oraz jednokrotne kodowanie BPE na zagregowanym bloku JSON w pamięci RAM.

Implementację procedury paginacji kursorowej integrującej powyższe mechanizmy przedstawiono w listingu **??**.

```
# 1. Asynchroniczna inspekcja indeksow w pg_catalog z ochrona buforow
async with pool.acquire() as check_conn:
    index_check = await index_catalog_guard.verify_index_compatibility(
        check_conn, safe_table, normalized_specs)

# 2. Dynamiczne wyznaczenie paczki i prewencyjne dlawienie
batch_size = token_budget_estimator.estimate_batch_size(token_budget,
    max_records)
if not index_check.is_compatible and batch_size > 50:
```

```
    batch_size = 50

# 3. Wykonanie zapytania keyset w izolacji snapshotowej MVCC
query_sql, params = build_keyset_query(safe_table, cursor_data=cursor_data,
    fetch_limit=batch_size + 1,
                                       order_by=normalized_specs,
   use_union_all=True)
rows = await snapshot_manager.execute_in_snapshot(pool, active_snapshot_id,
    query_sql, params) \
       if active_snapshot_id else await pool.fetch(query_sql, *params)
```

Listing 4. Zaawansowana paginacja wielokolumnowa z walidacją indeksów i izolacją snapshotową w pagination.py

5.5 Transfer danych binarnych Out-of-Band i asynchroniczne strumieniowanie

Fizyczną realizację mechanizmów transferu obiektów wielkich poza pasmem oraz asynchronicznego strumieniowania zdarzeń, sformalizowanych w podrozdziale **??**, wdrożono w modułach `src/mcp_server/out_of_band.py` oraz `src/mcp_server/streaming.py`.

Moduł transferu udostępnia potrójny dyspozytor danych (`DataTransferDispatcher`), obsługujący tryby zdefiniowane w enumeracji `TransferMode`:

1. **Ścieżka In-Band (`IN_BAND`):** Standardowe kodowanie Base64 wewnątrz ładunku JSON-RPC, stosowane domyślnie dla małych ładunków poniżej zachowawczego progu konfiguracyjnego (`threshold_bytes = 64 KB`), podlegającego dynamicznej rekonfiguracji w oparciu o empiryczny punkt opłacalności wyznaczony w Rozdziale **??**;

2. **Ścieżka OOB POSIX Shared Memory (`OOB_POSIX_SHM`):** Dedykowana dla procesów współdzielących jądro systemu operacyjnego na wspólnym hoście. Serwer alokuje nazwany segment pamięci współdzielonej w systemie plików `/dev/shm` za pomocą modułu standardowego `multiprocessing.shared_memory.SharedMemory`, kopiuje bufor binarny, wylicza skrót kryptograficzny SHA-256 i zwraca do agenta lekki deskryptor JSON zawierający wskaźnik pamięciowy. Odczyt przez klienta odbywa się w modelu bezkopiowym. Moduł rejestruje wszystkie aktywne segmenty w globalnym rejestrze `_ACTIVE_SHM_SEGMENTS`, a procedura zarejestrowana w mechanizmie `atexit` gwarantuje ich deterministyczne zwolnienie przy zatrzymaniu procesu;

3. **Ścieżka OOB HTTP Presigned URI (`OOB_HTTP_STREAM`):** Zapis bufora do dedykowanego magazynu blokowego i wygenerowanie jednorazowego adresu URL z podpisem HMAC-SHA256 oraz określonym czasem ważności (`expires_at`), co pozwala na strumieniowy odczyt protokołem HTTP/2 poza kanałem kontrolnym MCP;

4. **Tryb adaptacyjny (`AUTO`):** Dyspozytor dynamicznie porównuje rozmiar bufora z wyznaczonym progiem opłacalności S^*, automatycznie przekierowując ładunki przekraczające próg do segmentu pamięci współdzielonej.

Podczas alokacji segmentu pamięci współdzielonej dyspozytor mierzy stały narzut przygotowania deskryptora OOB (δ_{OOB}) przy użyciu nanosekundowego zegara monotonicznego `time.perf_counter_ns()`, dołączając tę metrykę do zwracanego deskryptora.

W module `src/mcp_server/streaming.py` zrealizowano asynchronicznego konsumenta komunikatów telemetrycznych w klasie `KafkaTelemetryConsumer`. Moduł bazuje na bibliotece `aiokafka` i realizuje ciągły nasłuch tematu `telemetry_test` w asynchronicznej pętli `_consume_loop`. Odebranie komunikatu o statusie `CRITICAL_ANOMALY` skutkuje natychmiastowym zapisem zdarzenia w pamięci podręcznej (`deque(maxlen=100)`) oraz asynchronicznym wywołaniem zarejestrowanych funkcji zwrotnych subskrybentów SSE. Zastosowanie modelu sterowanego zdarzeniami całkowicie eliminuje konieczność aktywnego odpytywania bazy przez agenta.

5.6 Moduł bezpieczeństwa zapytań (SQL Sanitizer & Guardrails)

Fizyczną realizację perymetru bezpieczeństwa, zaprojektowanego w podrozdziale **??**, wdrożono w module `src/mcp_server/security.py` w klasie `SqlSanitizer` w oparciu o bibliotekę analizy składniowej `sqlglot` [30]. Analiza drzewa wyprowadzenia (AST) w dialekcie `postgres` zapewnia matematyczną pewność weryfikacji struktury gramatycznej kwerendy, uniemożliwiając ominięcie filtrów za pomocą wieloliniowych komentarzy czy zaciemniania kodu.

Kluczowym wyzwaniem systemowym przy intensywnej analizie gramatycznej w środowisku asynchronicznym jest zjawisko głodzenia pętli zdarzeń. Ponieważ budowa i trawersowanie drzewa AST jest operacją silnie obciążającą procesor, wykonywanie jej w wątku głównym blokowałoby harmonogram I/O, powodując lawinowy wzrost opóźnień sieciowych. W celu eliminacji tego wąskiego gardła zaimplementowano wieloprocesową pulę wykonawczą `concurrent.futures.ProcessPoolExecutor` (funkcje `get_process_pool` oraz `async_validate_query`).

Pula powoływana jest z uniksowym kontekstem startowym `forkserver`. Zastosowanie metody `forkserver` stanowi kompromis inżynierski: standardowy mechanizm `fork` w środowisku wielowątkowym z bibliotekami napisanymi w języku C (`sqlglot`, sterowniki binarne) grozi zakleszczeniem procesów potomnych na zduplikowanych blokadach muteksów, natomiast mechanizm `spawn` generuje dotkliwy narzut zimnego startu interpretera Pythona. Kontekst `forkserver` powołuje jednowątkowy proces zarządczy, z którego w czasie submilisekundowym tworzone są procesy robocze, gwarantując pełną izolację pamięciową pętli zdarzeń `asyncio`.

Zasady walidacji drzewa AST obejmują:

1. **Niezmiennik pojedynczej instrukcji (`prevent_multi_statements`):** Wymusza obecność dokładnie jednego węzła głównego w liście instrukcji, blokując wstrzykiwanie kwerend piętrowych rozdzielanych średnikiem;

2. **Niezmiennik operacji tylko do odczytu (`validate_read_only`):** Korzeń drzewa AST musi być instancją typu `exp.Select` lub operacji teoriomnogościowej `exp.SetOperation` (UNION ALL, INTERSECT, EXCEPT). Całe drzewo jest rekurencyjnie przeszukiwane pod kątem obecności węzłów mutujących i administracyjnych zdefiniowanych w zbiorze `FORBIDDEN_AST_NODES` (obejmującym węzły: `Insert`, `Update`, `Delete`, `Merge`, `Drop`, `Create`, `Alter`, `TruncateTable`, `Grant`, `Revoke`, `Copy`, `Command`, `Execute`, `Pragma`, `Into`, `Analyze`, `Set`, `Transaction`, `Commit`, `Rollback`). Reguła ta bezwzględnie neutralizuje operacje mutujące ukryte wewnątrz klauzul CTE (np. `WITH d AS (DELETE FROM tab RETURNING *) SELECT * FROM d;`), zachowując jednocześnie pełną obsługę analitycznych funkcji okna (`OVER (PARTITION BY ...)`) oraz grupowania `CUBE`/`ROLLUP`;

3. **Wymuszenie limitu wierszy (`enforce_limit`):** Analizator modyfikuje drzewo AST, dopisując lub redukując istniejącą klauzulę `LIMIT` do bezpiecznej wartości maksymalnej, zapobiegając atakom typu Denial of Service;

4. **Strukturalna hermetyzacja danych (`encapsulate_untrusted_database_records`):** Rekordy pobrane z bazy podlegają oczyszczeniu ze znaków specjalnych XML oraz tokenów sterujących LLM (`<|im_start|>`, `<|im_end|>`, `[INST]`, `<<SYS>>`), po czym są hermetyzowane w ustrukturyzowanym bloku izolacyjnym:

```
<untrusted_db_record role="data_only" execution_forbidden="true">
  ... [rekordy bazy w formacie JSON] ...
</untrusted_db_record>

```

   co skutecznie neutralizuje ataki typu *Second-Order Prompt Injection* obecne w zanieczyszczonych danych relacyjnych.

Implementację asynchronicznej delegacji walidacji do puli `forkserver` przedstawiono w listingu **??**.

```
async def async_validate_query(sql_query: str, dialect: str = "postgres",
    max_limit: Optional[int] = None) -> ValidationResult:
    loop = asyncio.get_running_loop()
    pool = get_process_pool()  # ProcessPoolExecutor zainicjalizowany z
    mp_context="forkserver"
    return await loop.run_in_executor(pool, _worker_validate_query,
    sql_query, dialect, max_limit)
```

Listing 5. Asynchroniczna walidacja AST w puli ProcessPoolExecutor z kontekstem forkserver

5.7 Klient agentowy oraz zautomatyzowana aparatura pomiarowa

W celu przeprowadzenia rygorystycznych i powtarzalnych eksperymentów badawczych zaimplementowano klienta agentowego oraz modułowy runner pomiarowy. Klient agentowy

oparto na paradygmacie ReAct [48], zrealizowanym w klasie `ReActAgentLoop` (moduł `src/agent_client/agent_loop.py`). Pętla wykonawcza koordynuje sekwencję Myśl, Akcja oraz Obserwacja. Komunikację z serwerem FastMCP zaimplementowano w klasach `StdioMcpClient` (anonimowe potoki nienazwane `stdin`/`stdout`) oraz `SseMcpClient` (asynchroniczny transport HTTP/SSE).

W module `agent_loop.py` zaimplementowano architekturę podwójnego silnika wnioskowania:

- **Profil A (CPU Constrained / Edge):** Konfiguracja bazowa wykorzystująca model lokalny `qwen2.5-coder:3b` uruchamiany na procesorze głównym CPU za pośrednictwem silnika Ollama. Profil ten charakteryzuje się restrykcyjnym dławieniem okna generacji sekwencji wyjściowej do max_tokens $= 120$ przy deterministycznej temperaturze $\mathrm{T} = 0{,}0$. Konfiguracja ta służy do badania podatności na ucięcie kodu SQL (*Truncation Error*) oraz ewaluacji odporności modeli o mniejszej pojemności pamięciowej na zaszumienie schematu relacyjnego w warunkach brzegowych;
- **Profil B (Strong Reasoning / Unconstrained):** Konfiguracja zaawansowana oparta na modelu `qwen2.5-coder:7b` uruchamianym na akceleratorze GPU z wykorzystaniem silnika Ollama. Posiada pełne, niedławione okno generacji max_tokens $= 1024$ ($\mathrm{T} = 0{,}0$), co pozwala na wyizolowaną ocenę docelowej sprawności algorytmu selekcji schematu w zadaniach wymagających głębokiego wnioskowania relacyjnego i konstrukcji wielotabelowych złączeń, niezależnie od ograniczeń pojemnościowych małych modeli.

Fundament aparatury pomiarowej stanowi klasa `BenchmarkHarness` zlokalizowana w module `benchmarks/harness.py`. Konstrukcja runnera pomiarowego opiera się na następujących zasadach metodologicznych:

- **Determinizm środowiska bazodanowego:** Pomiędzy seriami pomiarowymi baza danych jest resetowana poleceniem `DISCARD ALL`, które czyści pamięć podręczną planów zapytań, blokady oraz sekwencje sesji. W generatorach liczb losowych wymuszono stałe ziarno (`seed = 42`);
- **Biegi rozgrzewkowe:** Każdy scenariusz poprzedzony jest wykonaniem 5 nieewaluowanych biegów rozgrzewkowych (`warmup_runs = 5`), co eliminuje zakłócenia związane z kompilacją kodu bajtowego Pythona oraz rozgrzewa bufor `shared_buffers` PostgreSQL;
- **Sztywny limit czasu zapytań:** W konfiguracji puli połączeń `asyncpg` wymuszono limit `statement_timeout = '10000ms'`, zapobiegający blokowaniu aparatury przez kwerendy zawieszające transakcje pod obciążeniem;
- **Wysokorozdzielcza aparatura próbkowania czasu:** Pomiar czasu wykonania operacji realizowany jest nanosekundowym zegarem monotonicznym `time.perf_counter_ns()`;
- **Asynchroniczny próbnik opóźnienia pętli zdarzeń (`EventLoopLagProbe`):** W scenariuszach współbieżnych uruchamiana jest sonda w tle, która co 10 ms próbuje wybudzić się z uśpienia (`asyncio.sleep(0.010)`). Różnica pomiędzy czasem rzeczywistym a oczekiwanym wyznacza bezpośrednio opóźnienie harmonogramu (*timer drift*) wywołane blokowaniem pętli zdarzeń;
- **Deterministyczna tokenometria:** Zliczanie tokenów generowanych w ładunkach realizowane jest biblioteką `tiktoken` (enkoder `cl100k_base`);

- **Profilowanie zasobów systemowych:** Klasa `SystemResourceProfiler` mierzy łączne zużycie procesora (CPU %) oraz pamięci operacyjnej (RSS w MB) procesu serwera wraz z jego potomkami za pomocą biblioteki `psutil`;
- **Ustrukturyzowany zapis danych surowych:** Wszystkie zarejestrowane próbki czasowe, metadane i wskaźniki są serializowane bezpośrednio do kolumnowych plików Apache Parquet oraz tabelarycznych CSV w katalogu `benchmarks/raw_results/`.

Architektura zautomatyzowanego pakietu testowego obejmuje sześć wyspecjalizowanych scenariuszy badawczych. Ich charakterystykę inżynierską, badane mechanizmy, wielkości prób oraz aparaturę wnioskowania statystycznego zestawiono w tabeli **??**.

Tabela 5. Zestawienie scenariuszy badawczych, aparatury pomiarowej oraz metod weryfikacji statystycznej

Skrypt / Scenariusz	**Badany mechanizm i cel**	**Wielkość próby**	**Rejestrowane metryki surowe**	**Weryfikacja statystyczna**
`bench_schema_quality.py`	Selekcja schematu JIT, domknięcie grafowe FK, odporność Text-to-SQL (Dual-Engine)	$\mathrm{N} = 500$ zapytań (5 domen $\times$ 100)	Recall, Precision, F1 (tabele i kolumny), Valid SQL Rate, Execution Accuracy, zużycie tokenów	Test sumy rang Wilcoxona, bootstrap 95% CI ($\mathrm{B} = 10\,000$)
`bench_oob_crossover.py`	Dyspozytor danych OOB, punkt przecięcia $\mathrm{S}^*(\mathrm{RTT})$ dla In-Band, SHM i HTTP	Ładunki 512 B – 50 MB, 3 profile RTT	Czas transferu End-to-End, δ_{OOB} alokacji, narzut pamięciowy, integralność SHA-256	Średnia geometryczna, bootstrap 95% CI, analiza regresji progu
`bench_schema_drift.py`	Odporność na dynamiczny dryf schematu podczas współbieżnych migracji DDL	50 nowych relacji, 50 kluczy FK, kwerendy w toku	Czas unieważnienia bufora TTL, narzut re-wektoryzacji, wskaźnik błędów kwerend *in-flight*	Przedziały ufności średnich, analiza stabilności pętli
`bench_concurrency.py`	Wydajność i skalowalność pętli zdarzeń pod obciążeniem (wątek główny vs `forkserver`)	$\mathrm{M} \leq 500$ ($\mathrm{M} \in \{1, 5, 10, 25, 50, 100, 500\}$)	Przepustowość (RPS), opóźnienie odpowiedzi, opóźnienie pętli (`EventLoopLagProbe`), utylizacja CPU i RSS	Analiza percentyli ($\mathrm{p}_{50}, \mathrm{p}_{95}, \mathrm{p}_{99}$), profilowanie zasobów
`bench_security_suite.py`	Odporność zapory AST i guardrails na wektory SQLi oraz Prompt Injection	$\mathrm{N} = 1000$ wektorów testowych	Wskaźnik fałszywych alarmów (FPR), mitygacja ataków (AMR), wskaźnik penetracji (ASR)	Dokładne testy dwumianowe, macierze pomyłek, 95% CI Cloppera-Pearsona
`statistical_analysis.py`	Formalna weryfikacja hipotez badawczych z surowych wektorów pomiarowych	Zbiorcze wektory z plików Parquet	Wartości statystyk W, poziom istotności p-value, wielkość efektu r, przedziały ufności	Dwustronny test Wilcoxona dla par zależnych, bootstrap $\mathrm{B} = 10\,000$

Zaprojektowane środowisko kontenerowe, zunifikowana brama pośrednicząca FastMCP oraz zautomatyzowana aparatura pomiarowa posłużyły do przeprowadzenia pełnego pakietu ewaluacji

empirycznej, której szczegółowe wyniki ilościowe, analiza statystyczna oraz interpretacja efektywnościowa stanowią treść Rozdziału **??**.

Rozdział 6

Wyniki Badań i Ewaluacja Eksperymentalna

W niniejszym rozdziale przedstawiono całościowe wyniki badań empirycznych oraz ewaluację eksperymentalną autorskiego systemu integracji protokołu MCP [1] ze środowiskiem wielkoskalowych hurtowni danych i rozproszonych brokerów strumieniowych. Pomiary przeprowadzono w oparciu o architekturę referencyjną i modele analityczne sformalizowane w Rozdziale **??** oraz fizyczną aparaturę badawczą zaimplementowaną w środowisku testowym opisanym w Rozdziale **??**.

Zgodnie z przyjętą dyscypliną metodologiczną niniejszy rozdział odpowiada na pytanie: *„Co wykazały pomiary empiryczne i z czego wynikają zaobserwowane charakterystyki wydajnościowe?"*. W analizie skupiono się na surowych danych telemetrycznych, trendach skalowalności, punktach równowagi narzutów, systematycznej dekompozycji błędów oraz formalnej weryfikacji hipotez badawczych z wykorzystaniem aparatu testów nieparametrycznych w rygorze standardów akademickich.

Wszystkie prezentowane estymatory statystyczne (średnie arytmetyczne, odchylenia standardowe, mediany p_{50}, percentyle p_{95} i p_{99}, przedziały ufności 95% CI oraz wartości rangowego testu Wilcoxona) zostały zarejestrowane automatycznie przez moduł `BenchmarkHarness` na deterministycznym zbiorze ewaluacyjnym $\mathrm{N} = 500$ złożonych zapytań badawczych.

6.1 Metodyka pomiarowa i formalizacja aparatu ewaluacyjnego

6.1.1 Formalizacja matematyczna metryk oceny

W celu zapewnienia pełnej obiektywności, powtarzalności oraz porównywalności rezultatów zdefiniowano sformalizowany zbiór metryk ewaluacyjnych obejmujących charakterystykę kontekstową, dynamikę czasową, jakość semantyczną generacji Text-to-SQL na poziomie tabel i kolumn oraz perymetr bezpieczeństwa.

Względna redukcja narzutu tokenów ($\Delta \mathrm{T}$)

Podstawowym wskaźnikiem efektywności zarządzania oknem kontekstowym modelu językowego podczas eksploracji rozległych struktur relacyjnych jest względna redukcja całkowitej liczby zaalokowanych tokenów ($\Delta \mathrm{T}$):

$$\Delta \mathrm{T} = \frac{\sum_{\mathrm{i}=1}^{\mathrm{N}} \mathrm{T}_{\mathsf{baseline}}^{(\mathrm{i})} - \sum_{\mathrm{i}=1}^{\mathrm{N}} \mathrm{T}_{\mathsf{MCP}}^{(\mathrm{i})}}{\sum_{\mathrm{i}=1}^{\mathrm{N}} \mathrm{T}_{\mathsf{baseline}}^{(\mathrm{i})}} \times 100\% \tag{30}$$

gdzie $\mathrm{T}^{(\mathrm{i})} = \mathrm{T}_{\mathsf{prompt}}^{(\mathrm{i})} + \mathrm{T}_{\mathsf{completion}}^{(\mathrm{i})}$ reprezentuje całkowity wolumen tokenów zużyty w i-tym zapytaniu badawczym (obejmujący fazę wstępnego wypełnienia kontekstu oraz sekwencyjną generację odpowiedzi), $\mathrm{T}_{\mathsf{baseline}}^{(\mathrm{i})}$ oznacza zużycie w podejściu referencyjnym (pełny zrzut DDL lub transmisja wewnątrzpasłowa Base64), natomiast $\mathrm{T}_{\mathsf{MCP}}^{(\mathrm{i})}$ określa liczbę tokenów zaalokowanych przez projektowaną bramę pośredniczącą MCP.

Dynamika opóźnień wnioskowania (TTFT oraz $\mathrm{L}_{\mathsf{E2E}}$)

Opóźnienie odpowiedzi agenta w architekturze ReAct dekomponowane jest na fazę inicjalizacji przetwarzania kontekstu oraz sekwencyjną fazę autokonwersacji:

$$\mathrm{TTFT} = \mathrm{t}_{\mathsf{first_token}} - \mathrm{t}_{\mathsf{request_sent}} \tag{31}$$

$$\mathrm{L}_{\mathsf{E2E}} = \mathrm{TTFT} + \sum_{\mathrm{k}=2}^{\mathrm{M}} \Delta \mathrm{t}_{\mathrm{k}} \tag{32}$$

gdzie TTFT (ang. *Time-to-First-Token*) oznacza czas upływający od wysłania żądania RPC przez klienta do wyemitowania pierwszego tokenu przez silnik inferencyjny modelu, $\mathrm{L}_{\mathsf{E2E}}$ (ang. *End-to-End Latency*) reprezentuje całkowity czas ukończenia zadania analitycznego, M jest liczbą wygenerowanych tokenów wyjściowych, a $\Delta \mathrm{t}_{\mathrm{k}}$ określa czas generacji k-tego tokenu (ang. *inter-token latency*).

Wskaźniki jakościowe Text-to-SQL na poziomie tabel i kolumn

Jakość semantycznej selekcji struktur schematu bazy danych oceniano w wymiarze tabelarycznym oraz atrybutowym.

Dla wymiaru tabelarycznego zdefiniowano:

$$\mathsf{Table\text{-}Recall@k} = \frac{|\mathrm{T}_{\mathsf{relevant}} \cap \mathrm{T}_{\mathsf{retrieved@k}}|}{|\mathrm{T}_{\mathsf{relevant}}|} \tag{33}$$

$$\mathsf{Table\text{-}Precision@k} = \frac{|\mathrm{T}_{\mathsf{relevant}} \cap \mathrm{T}_{\mathsf{retrieved@k}}|}{\mathrm{k}} \tag{34}$$

gdzie $\mathrm{T}_{\text{relevant}}$ oznacza zbiór tabel wzorcowych (*Ground Truth*) niezbędnych do poprawnego sformułowania kwerendy analitycznej, a $\mathrm{T}_{\text{retrieved@k}}$ jest podzbiorem k relacji wyselekcjonowanych przez dany algorytm selekcji schematu.

W celu precyzyjnej oceny ziarnistej selekcji metadanych wprowadzono formalne metryki na poziomie kolumn:

$$\text{Column-Recall@k} = \frac{|\mathrm{C}_{\text{relevant}} \cap \mathrm{C}_{\text{retrieved@k}}|}{|\mathrm{C}_{\text{relevant}}|} \tag{35}$$

$$\text{Column-Precision@k} = \frac{|\mathrm{C}_{\text{relevant}} \cap \mathrm{C}_{\text{retrieved@k}}|}{|\mathrm{C}_{\text{retrieved@k}}|} \tag{36}$$

$$\text{Column-F1} = 2 \cdot \frac{\text{Column-Precision@k} \cdot \text{Column-Recall@k}}{\text{Column-Precision@k} + \text{Column-Recall@k}} \tag{37}$$

gdzie $\mathrm{C}_{\text{relevant}}$ oznacza zbiór kolumn niezbędnych do konstrukcji zapytania wzorcowego (projekcje, warunki złączeń, filtry `WHERE`, atrybuty `GROUP BY`), natomiast $\mathrm{C}_{\text{retrieved@k}}$ reprezentuje łączny zbiór kolumn zawartych w wygenerowanym bloku DDL wyselekcjonowanych k tabel.

Poprawność generacji kodu SQL i jego zgodność semantyczną mierzono wskaźnikami:

$$\text{Valid SQL Rate} = \frac{\mathrm{N}_{\text{valid_sql}}}{\mathrm{N}_{\text{total_queries}}} \times 100\% \tag{38}$$

$$\text{Execution Accuracy (EX)} = \frac{\mathrm{N}_{\text{exec_match}}}{\mathrm{N}_{\text{total_queries}}} \times 100\% \tag{39}$$

gdzie $\mathrm{N}_{\text{valid_sql}}$ to liczba zapytań poprawnych pod względem składniowym (sukces analizy składniowej drzewa AST w dialekcie PostgreSQL), a $\mathrm{N}_{\text{exec_match}}$ oznacza liczbę kwerend, których wykonanie na silniku relacyjnym zwróciło zbiór krotek identyczny z wynikiem zapytania wzorcowego.

Taksonomia błędów Text-to-SQL

W celu rzetelnego zbadania przyczyn niepowodzenia generacji zapytań wprowadzono sformalizowaną taksonomię obejmującą sześć rozłącznych kategorii klasyfikacji błędów:

1. `TRUNCATION` – przedwczesne ucięcie sekwencji wyjściowej z powodu wyczerpania zadeklarowanego budżetu `max_tokens` modelu;
2. `SYNTAX_ERROR` – błąd gramatyczny kwerendy uniemożliwiający analizę składniową AST przez silnik PostgreSQL;
3. `TABLE_MISSING` – odwołanie w zapytaniu do tabeli nieobecnej w dostarczonym wycinku DDL;
4. `COLUMN_HALLUCINATION` – użycie w zapytaniu atrybutu nieistniejącego w schemacie wyselekcjonowanych relacji;
5. `JOIN_CONDITION_MISMATCH` – błędny, niekompletny lub brakujący predykat złączenia klucza obcego (skutkujący m.in. złączeniem kartezjańskim);

6. `SEMANTIC_MISMATCH` – zapytanie poprawne syntaktycznie, którego zbiór wynikowy różni się od wzorca na skutek błędu logicznego (niewłaściwa funkcja agregująca, błędny operator logiczny lub predykat filtrujący).

Skuteczność perymetru bezpieczeństwa (AMR, FPR, ASR)

Odporność bramy pośredniczącej na wstrzykiwanie niepożądanych instrukcji oraz stabilność klasyfikacji zapytań analitycznych formalizują wskaźniki:

$$\text{Attack Mitigation Rate } (\mathrm{AMR}) = \frac{\mathrm{N}_{\text{blocked_malicious}}}{\mathrm{N}_{\text{total_malicious}}} \times 100\% \tag{40}$$

$$\text{False Positive Rate } (\mathrm{FPR}) = \frac{\mathrm{N}_{\text{blocked_benign}}}{\mathrm{N}_{\text{total_benign}}} \times 100\% \tag{41}$$

$$\text{Attack Success Rate } (\mathrm{ASR}) = \frac{\mathrm{N}_{\text{successful_injections}}}{\mathrm{N}_{\text{total_injections}}} \times 100\% \tag{42}$$

gdzie AMR określa odsetek zablokowanych wektorów SQL Injection, FPR definiuje prawdopodobieństwo błędnego odrzucenia poprawnego, złożonego zapytania analitycznego przez zaporę AST, a ASR mierzy odsetek udanych przejęć kontekstu agenta w atakach wstrzykiwania promptu bezpośrednich i drugiego rzędu.

6.1.2 Reżim powtarzalności pomiarowej i warunki środowiskowe

Ewaluację eksperymentalną przeprowadzono na dedykowanej instancji platformy obliczeniowej wyposażonej w akcelerator graficzny NVIDIA A100 SXM (80 GB pamięci VRAM), 16 wirtualnych rdzeni procesora vCPU (opartych na architekturze AMD EPYC 7742 64-Core), 250 GB systemowej pamięci operacyjnej RAM oraz szybką pamięć masową NVMe o pojemności 60 GB przypisanej do kontenera roboczego. Środowisko pracowało pod kontrolą 64-bitowego systemu operacyjnego Linux (jądro 6.8). Wszystkie usługi infrastruktury (PostgreSQL 16.2 z rozszerzeniem pgvector 0.7.0, broker Apache Kafka 3.7.0 w trybie KRaft) oraz brama FastMCP zostały uruchomione w wyizolowanych kontenerach platformy Docker 26.0 z bezpośrednią alokacją zasobów sprzętowych.

W celu wyeliminowania niestacjonarności środowiska i fluktuacji systemowych wdrożono rygorystyczny protokół eksperymentalny:

- **Liczebność i struktura próby ewaluacyjnej:** Zbiór testowy obejmuje ściśle $\mathrm{N} = 500$ złożonych zapytań analitycznych Text-to-SQL, podzielonych równomiernie na 5 domen biznesowych (po 100 zapytań dla domen: finanse, gospodarka magazynowa, kadry i płace, CRM oraz telemetria przemysłowa) w topologii 1000 relacji i 1174 kluczy obcych;
- **Izolacja pamięci podręcznej PostgreSQL:** Przed każdą serią pomiarową sesje bazy danych były bezwzględnie resetowane poleceniem `DISCARD ALL`, wymuszającym wyczyszczenie planów przygotowanych, buforów kwerend i blokad sesyjnych;

- **Determinizm próbkowania modeli LLM:** W scenariuszach inferencyjnych temperaturę próbkowania zablokowano na poziomie $\mathrm{T} = 0{,}0$, a generator liczb pseudolosowych zainicjalizowano deterministycznym ziarnem (`seed = 42`);
- **Wymuszenie limitu czasu wykonania (`statement_timeout`):** W konfiguracji sesji bazodanowych wymuszono sztywny limit `statement_timeout = '10000ms'`, zapobiegający blokowaniu aparatury pomiarowej przez zapytania generujące patologiczne plany wykonania pod obciążeniem;
- **Cykliczne próbkowanie pętli zdarzeń:** Asynchroniczny monitor obciążenia rejestrował opóźnienia pętli zdarzeń `asyncio` w stałym interwale próbkowania wynoszącym 10 ms za pomocą wysokorozdzielczego zegara sprzętowego `time.perf_counter_ns()`;
- **Biegi rozgrzewkowe (Warmup):** Każdy cykl pomiarowy poprzedzano 5 nieewaluowanymi przebiegami rozgrzewkowymi, co zapewniało wstępną kompilację kodu bajtowego Pythona, alokację struktur wewnętrznych oraz rozgrzanie bufora `shared_buffers`.

6.1.3 Procedura wnioskowania statystycznego

Z uwagi na asymetrię rozkładów opóźnień czasowych i występowanie grubych ogonów w systemach sieciowych, do formalnej weryfikacji hipotez badawczych zastosowano nieparametryczny test rangowanych sum Wilcoxona dla par zależnych [46].

Dla N skojarzonych par obserwacji $(\mathrm{x_i}, \mathrm{y_i})$ oblicza się różnice $\mathrm{d_i} = \mathrm{x_i} - \mathrm{y_i}$. Po odrzuceniu par o $\mathrm{d_i} = 0$ pozostałym różnicom przypisuje się rangi $\mathrm{R_i}$. Statystyka testowa W definiowana jest jako:

$$\mathrm{W} = \min(\mathrm{W}^+, \mathrm{W}^-), \quad \mathrm{W}^+ = \sum_{\mathrm{d_i}>0} \mathrm{R_i}, \quad \mathrm{W}^- = \sum_{\mathrm{d_i}<0} \mathrm{R_i} \tag{43}$$

Dla dużych prób statystyka testowa zbiega asymptotycznie do rozkładu normalnego o parametrze z:

$$\mathrm{z} = \frac{\mathrm{W} - \frac{\mathrm{N(N+1)}}{4}}{\sqrt{\frac{\mathrm{N(N+1)(2N+1)}}{24}}} \tag{44}$$

Wielkość efektu standaryzowanego (ang. *Effect Size*) obliczano według zależności:

$$\mathrm{r} = \frac{|\mathrm{z}|}{\sqrt{\mathrm{N}}} \tag{45}$$

gdzie wartości $\mathrm{r} \geq 0{,}5$ wskazują na duży rozmiar efektu. Dodatkowo dla estymatorów centralnych wyznaczono 95% przedziały ufności (95% CI) metodą nieparametrycznego bootstrapu przy $\mathrm{B} = 10\,000$ powtórzeń próbkowania ze zwracaniem. Za próg istotności statystycznej we wszystkich próbach przyjęto poziom $\alpha = 0{,}05$.

6.2 Semantyczna jakość odkrywania schematu relacyjnego (Scenariusz A)

6.2.1 Eksperymentalna konfrontacja strategii selekcji schematu

W Scenariuszu A przeprowadzono badanie skuteczności silnika `SchemaEngine` na korporacyjnym schemacie 1000 tabel powiązanych 1174 kluczami obcymi o topologii sieci bezskalowej. Zestaw ewaluacyjny obejmował $\mathrm{N} = 500$ zróżnicowanych, złożonych zapytań analitycznych Text-to-SQL (po 100 zapytań na każdą z 5 domen biznesowych: finanse, magazyn, HR, CRM, telemetria), wymagających relacyjnych ścieżek złączeń od 2 do 6 tabel.

Poddano konfrontacji pięć strategii dostarczania metadanych schematu:

1. **Baseline A (Full Schema Dump):** Podejście polegające na bezwarunkowym wstrzyknięciu pełnego kodu DDL wszystkich 1000 tabel do promptu systemowego;
2. **Baseline B (Lexical BM25, $\mathrm{k} = 5$):** Wyszukiwanie leksykalne struktur schematu na podstawie słów kluczowych zapytania;
3. **Baseline C (Hybrid RRF, $\mathrm{k} = 5$):** Wyszukiwanie hybrydowe łączące gęste osadzenia semantyczne `pgvector` oraz model BM25 za pomocą algorytmu Reciprocal Rank Fusion bez analizy grafu kluczy obcych;
4. **Baseline D (Dense-only pgvector k-NN, $\mathrm{k} = 5$):** Czysto semantyczne wyszukiwanie najbliższych sąsiadów w przestrzeni wektorowej osadzeń tabel bez komponentu leksykalnego;
5. **Autorskie (Graph Pruning & Bridges, $\mathrm{k} = 5$):** Kompletny silnik JIT Schema Discovery łączący fuzję RRF z algorytmem domknięcia relacyjnego i wstrzykiwania tabel mostkujących na najkrótszych ścieżkach grafu ($\mathrm{dist} \leq 2$).

Zbiorcze wyniki ewaluacji jakości selekcji schematu oraz generacji kodu Text-to-SQL dla parametru $\mathrm{k} = 5$ na modelu bez ograniczeń dławienia (Profil B: `max_tokens = 1024`) zestawiono w tabeli **??**.

Tabela 6. Porównanie parametrów semantycznych i jakości Text-to-SQL dla pięciu strategii selekcji schematu ($\mathrm{N} = 500$, $\mathrm{k} = 5$, Profil B: `max_tokens = 1024`)

Strategia selekcji schematu	Recall@5 [%]	Precision@5 [%]	Column-F1 [%]	Valid SQL [%]	EX [%]	Tokeny DDL [tokeny]
Baseline A (Full Schema – Context Truncated)	15,0	3,5	1,1	10,4	5,8	1 032,2
Baseline B (Lexical BM25)	45,9	30,9	14,2	18,0	14,0	492,1
Baseline C (Hybrid RRF)	39,2	25,8	11,8	18,2	14,6	493,7
Baseline D (Dense-only pgvector)	29,7	19,0	8,6	16,4	13,2	491,8
Autorskie (Graph Pruning)	**54,5**	**29,8**	**11,9**	**19,8**	**15,8**	**2 218,8**

Zwraca uwagę pozorna anomalia w narzucie tokenowym strategii referencyjnej: pełny kod DDL 1000 tabel (∼150 tys. tokenów) ulegał fizycznemu obcięciu przez bufor okna kontekstowego runtime'u modelu do średnio 1 032,2 tokenów. W rezultacie model otrzymywał chaotyczny, przypadkowo

urwany fragment schematu, co tłumaczy gwałtowną degradację czułości Table-Recall@5 do poziomu zaledwie 15,0% oraz minimalny wskaźnik Execution Accuracy (5,8%). Wariant autorski (2 218,8 tokenów) dostarczał natomiast spójny, celowy podgraf zsyntetyzowanego DDL o zamkniętych więzach integralności referencyjnej.

Analiza danych z tabeli **??** wskazuje na wyraźną przewagę podejścia uwzględniającego topologię powiązań relacyjnych:

- Autorski algorytm Graph Pruning osiąga czułość Table-Recall@5 na poziomie **54,5%**, co oznacza przewagę o 15,3 punktu procentowego nad hybrydą RRF (39,2%), o 8,6 pp nad wyszukiwaniem leksykalnym BM25 (45,9%) oraz aż o 24,8 pp nad czysto semantycznym osadzeniem Dense-only (29,7%);
- Minimalny spadek metryki Table-Precision@5 w podejściu autorskim (29,8% względem 30,9% dla BM25) stanowi świadomy kompromis inżynierski: deterministyczne wstrzykiwanie tabel asocjacyjnych priorytetyzuje maksymalizację czułości relacyjnej (Recall), która w zadaniach Text-to-SQL jest parametrem krytycznym warunkującym poprawność konstrukcji klauzul JOIN (brak tabeli pośredniczącej całkowicie paraliżuje zapytanie, podczas gdy obecność nadmiarowego schematu nie wywołuje błędu składniowego);
- Podejście Full Schema Dump (Baseline A) uzyskuje najniższą czułość (15,0%) oraz minimalną precyzję (3,5%), będąc bezpośrednią konsekwencją drastycznego przycinania promptu przez model przy przekroczeniu dopuszczalnej długości kontekstu DDL;
- Wyszukiwanie gęste Dense-only (Baseline D) osiąga czułość zaledwie 29,7%, co potwierdza, że same osadzenia wektorowe słabo radzą sobie ze znormalizowanymi, technicznymi nazwami tabel i kluczy (np. `fin_gl_entry_acc`) w oderwaniu od dopasowania leksykalnego słów kluczowych;
- W zakresie dokładności wykonawczej (Execution Accuracy EX) podejście autorskie osiąga najwyższy wynik **15,8%** (przy Valid SQL = **19,8%**), deklasując Full Schema (5,8% EX, 10,4% Valid SQL) oraz przewyższając wyszukiwanie hybrydowe RRF (14,6% EX, 18,2% Valid SQL) i leksykalne BM25 (14,0% EX, 18,0% Valid SQL).

6.2.2 Analiza czułości i precyzji w funkcji parametru k

W celu zbadania dynamiki doboru struktur relacyjnych przeprowadzono analizę wrażliwości parametru $\mathrm{k} \in \{3, 5, 10, 20\}$. Przebieg czułości (Recall@k) oraz precyzji (Precision@k) na próbie $\mathrm{N} = 500$ przedstawiono na rysunku **??**.

Rysunek 3. Przebieg metryk Table-Recall@k oraz Table-Precision@k w funkcji liczby pobieranych tabel $\mathrm{k} \in \{3, 5, 10, 20\}$ dla badanych strategii ($\mathrm{N} = 500$)

Z charakterystyk na rysunku **??** wynika, że autorski mechanizm Graph Pruning dominuje nad podejściami bazowymi w całym badanym spektrum parametru k:

- Przy $\mathrm{k} = 3$ algorytm autorski osiąga czułość 45,0% (wobec 41,3% dla BM25, 34,0% dla RRF oraz 24,4% dla Dense-only);

- Przy $\mathrm{k} = 5$ przewaga rośnie do 54,5% (wobec 45,9% dla BM25, 39,2% dla RRF i 29,7% dla Dense-only);
- Przy $\mathrm{k} = 10$ podejście autorskie osiąga 66,8% (wobec 53,2% dla BM25, 48,8% dla RRF i 40,1% dla Dense-only);
- Dla $\mathrm{k} = 20$ podejście autorskie osiąga **79,3%** pokrycia relacji (wobec 64,8% dla BM25, 61,6% dla RRF i 49,8% dla Dense-only);
- Precyzja doboru struktur (Precision@k) wykazuje naturalny spadek wraz ze wzrostem k, jednak algorytm autorski utrzymuje wysoki poziom precyzji (41,0% przy $\mathrm{k} = 3$ oraz 29,8% przy $\mathrm{k} = 5$), co potwierdza, że wstrzykiwane tabele mostkujące stanowią rzeczywiste elementy ścieżek relacyjnych, a nie zbędny szum kontekstowy.

6.2.3 Ocena dwusilnikowa Text-to-SQL (Dual-Engine Evaluation)

Wpływ ograniczeń zasobowych silnika wnioskowania na jakość generacji zapytań analitycznych zbadano w architekturze dwusilnikowej:

1. **Silnik A (CPU Constrained / Edge):** Model lokalny `qwen2.5-coder:3b` uruchamiany na CPU za pośrednictwem środowiska Ollama z restrykcyjnym limitem max_tokens $= 120$;
2. **Silnik B (Strong Reasoning / Unconstrained):** Model `qwen2.5-coder:7b` uruchamiany na GPU z pełnym oknem generacji max_tokens $= 1024$.

Bezpośrednie porównanie wskaźników poprawności składniowej (Valid SQL Rate) oraz dokładności wykonawczej (Execution Accuracy EX) na próbie $\mathrm{N} = 500$ przedstawiono na rysunku **??**.

Rysunek 4. Porównanie wskaźników Valid SQL Rate oraz Execution Accuracy w architekturze Dual-Engine: Silnik A (CPU dławiony, `max_tokens = 120`) vs Silnik B (GPU unconstrained, `max_tokens = 1024`) dla $\mathrm{N} = 500$

Wykres na rysunku **??** uwidacznia kluczową zależność:

- W Profilu A (CPU dławiony) wskaźnik dokładności wykonawczej EX osiąga zaledwie 4,6%–10,8% (10,8% dla podejścia autorskiego oraz 4,6% dla Full Schema), a wskaźnik poprawności składniowej (Valid SQL) nie przekracza 12,0% (7,0% dla Full Schema);
- W Profilu B (GPU unconstrained) eliminacja bariery tokenowej skutkuje skokowym wzrostem parametrów jakościowych: podejście autorskie osiąga **15,8%** EX oraz **19,8%** Valid SQL (przewyższając hybrydę RRF z 14,6% EX i 18,2% Valid SQL, BM25 z 14,0% EX i 18,0% Valid SQL oraz Dense-only z 13,2% EX i 16,4% Valid SQL);
- Podejście Full Schema Dump na Silniku B osiąga zaledwie 5,8% EX oraz 10,4% Valid SQL, co potwierdza, że niska czułość chaotycznego wycinka schematu (15,0%) i przeładowanie promptu paraliżują zdolność modelu do konstrukcji poprawnych kwerend nawet przy braku dławienia tokenów wyjściowych.

6.2.4 Dekompozycja drzewa błędów generacji zapytań

W celu rzetelnego wyjaśnienia dysproporcji pomiędzy wynikami Silnika A i Silnika B przeprowadzono dekompozycję niepowodzeń zapytań w oparciu o sześcioczłonową taksonomię błędów. Rozkład kategorii błędów na próbie $\mathrm{N} = 500$ zilustrowano na rysunku **??**.

Rysunek 5. Dekompozycja przyczyn niepowodzenia kwerend SQL (taksonomia błędów) w architekturze dwusilnikowej: Silnik A (CPU Constrained) vs Silnik B (Strong Reasoning) dla $\mathrm{N} = 500$

Dekompozycja drzewa błędów jednoznacznie dowodzi natury zaobserwowanych ograniczeń:

- **Silnik A (CPU Constrained):** Dominującą przyczyną niepowodzenia było sztuczne obcięcie sekwencji tokenów (`TRUNCATION`), odpowiadające za 30,0% błędów w podejściu autorskim oraz 30,4% w wariancie Full Schema, a także wtórny błąd składniowy (`SYNTAX_ERROR`, 17,3% w autorskim) oraz halucynacje kolumn (`COLUMN_HALLUCINATION`, 40,1%);
- **Silnik B (Strong Reasoning):** Po rozszerzeniu okna wyjściowego (`max_tokens = 1024`) błędy typu `TRUNCATION` oraz `SYNTAX_ERROR` zostały całkowicie wyeliminowane (**0,0%** w podejściu autorskim);
- **Struktura błędów semantycznych:** Na Silniku B niepowodzenia przesunęły się w stronę rzeczywistych wyzwań semantycznych i domenowych: halucynacji nazw atrybutów (`COLUMN_HALLUCINATION` – 82,9% błędów), braków struktur relacyjnych (`TABLE_MISSING` – 11,9%) oraz niezgodności semantycznej ze zbiorem referencyjnym (`SEMANTIC_MISMATCH` – 4,8%);
- **Redukcja brakujących tabel:** W wariancie autorskim z domknięciem grafowym odsetek błędów braku tabeli (`TABLE_MISSING`) ograniczono do 11,9% (200 przypadków), podczas gdy w podejściu Full Schema wynosił on aż 27,0% (508 przypadków), co stanowi ponad 2,2-krotną redukcję i potwierdza skuteczność wstrzykiwania relacji mostkujących.

6.2.5 Badanie ablacyjne parametru domknięcia grafowego $\mathrm{M}_{\mathrm{exp}}$

Wpływ maksymalnej liczby wstrzykiwanych tabel mostkujących na efektywność wyszukiwania schematu zbadano w eksperymencie ablacyjnym dla parametru $\mathrm{M}_{\mathrm{exp}} \in \{0, 1, 2, 3, 5\}$ na próbie $\mathrm{N} = 500$. Wyniki zestawiono na rysunku **??**.

Rysunek 6. Badanie ablacyjne parametru maksymalnej liczby tabel mostkujących $\mathrm{M}_{\mathrm{exp}} \in \{0, 1, 2, 3, 5\}$: Table-Recall@5, narzut tokenów DDL oraz latencja grafowa ($\mathrm{N} = 500$)

Pomiary empiryczne ujawniają wyraźny punkt nasycenia:

- Dla $\mathrm{M}_{\mathrm{exp}} = 0$ (brak domknięcia relacyjnego) czułość Table-Recall@5 wynosi zaledwie 39,2% przy średnim narzucie 1 622,5 tokena DDL;
- Wprowadzenie pojedynczej tabeli mostkującej ($\mathrm{M}_{\mathrm{exp}} = 1$) powoduje gwałtowny skok czułości do 52,4% (+13,2 pp) przy narzucie 1 894,1 tokena DDL;

- Przy $\mathrm{M}_{\mathsf{exp}} = 2$ czułość wzrasta do 54,5%, osiągając punkt nasycenia przy $\mathrm{M}_{\mathsf{exp}} = 3$ na poziomie **54,5%** (0,5448) przy narzucie 1 998,4 tokena DDL i latencji grafowej 20,6 ms;
- Dalsze zwiększenie parametru do $\mathrm{M}_{\mathsf{exp}} = 5$ nie przynosi żadnego przyrostu czułości (pozostaje na poziomie 54,5%), wprowadzając marginalny wzrost narzutu DDL (1 999,0 tokenów).

Wynik ten stanowi empiryczne uzasadnienie wyboru $\mathrm{M}_{\mathsf{exp}} = 3$ jako optymalnego punktu pracy algorytmu domknięcia grafowego.

6.3 Analiza progu opłacalności transferu binarnego Out-of-Band (Scenariusz B1)

6.3.1 Eksperyment wieloprofilowy z emulacją opóźnień RTT

W Scenariuszu B1 zbadano charakterystykę opóźnień transferu wielkich ładunków danych (BLOB) w funkcji rozmiaru ładunku $\mathrm{S} \in [512\ \mathrm{B}, 50\ \mathrm{MB}]$ w trzech profilach sieciowych:

1. **Profil Loopback / Localhost (POSIX SHM, RTT $= 0{,}05$ ms):** Komunikacja lokalna na pojedynczym węźle z wykorzystaniem pamięci współdzielonej `/dev/shm`;
2. **Profil Datacenter LAN (HTTP/2 Presigned URL, RTT $= 1{,}0$ ms):** Transfer wewnątrzsieciowy w centrum danych;
3. **Profil WAN / Multi-Cloud (HTTP/2 Presigned URL, RTT $= 30{,}0$ ms):** Transfer rozproszony przez łącza rozległe z narzutem zestawienia połączenia HTTP/2 i latencji sieciowej.

Porównanie opóźnień transferu In-Band Base64 oraz wariantów Out-of-Band przedstawiono na rysunku **??**.

Rysunek 7. Wyznaczenie progu opłacalności transferu Out-of-Band w trzech profilach sieciowych: Localhost (POSIX SHM), Datacenter LAN (RTT = 1 ms) oraz WAN (RTT = 30 ms)

6.3.2 Parametry punktu przecięcia i zysk skalowalności

Analiza charakterystyk na rysunku **??** w pełni weryfikuje rozszerzony model analityczny $\mathrm{S}^*(\mathrm{RTT})$:

- **Profil Loopback:** Czysty, analityczny model wyprowadzony w równaniu (**??**) przy bazowym narzucie jądra $\delta_{\mathsf{OOB,\ shm}} \approx 29\ \mu\mathrm{s}$ wyznacza teoretyczny próg opłacalności na poziomie $\mathrm{S}^*_{\mathsf{teor}} \approx 5{,}19\ \mathrm{kB}$. W rzeczywistym środowisku kontenerowym Docker alokacja segmentu pamięci współdzielonej IPC generuje stały narzut $\delta_{\mathsf{OOB,\ shm}} = 0{,}0514$ ms ($51{,}4\ \mu$s), co przesuwa punkt empiryczny do wartości:

$$\mathrm{S}^*_{\mathsf{emp}} = 6{,}89\ \mathrm{kB} \tag{46}$$

  Rozbieżność rzędu zaledwie $1{,}7$ kB pomiędzy modelem matematycznym a empirycznym pomiarem w kontenerze stanowi bezpośrednie potwierdzenie wysokiej trafności sformułowanego

modelu analitycznego. Przy ładunku 50 MB transfer POSIX SHM trwa zaledwie **97,94 ms**, podczas gdy transmisja wewnątrzpasłowa In-Band Base64 wymaga aż **672,98 ms**, co daje **6,87-krotne przyspieszenie**;

- **Profil Datacenter LAN (RTT = 1,0 ms):** Narzut dodatkowej rundy sieciowej HTTP/2 przesuwa punkt równowagi do wartości $S^*_{LAN} = 122{,}92$ kB;
- **Profil WAN / Multi-Cloud (RTT = 30,0 ms):** Wysokie opóźnienie łącza WAN sprawia, że narzut zestawienia połączenia HTTP/2 i przesłania deskryptora dominuje nad narzutem Base64 dla małych i średnich paczek. Punkt opłacalności przesuwa się do poziomu $S^*_{WAN} = 1\,681{,}22$ kB ($\approx 1{,}68$ MB).

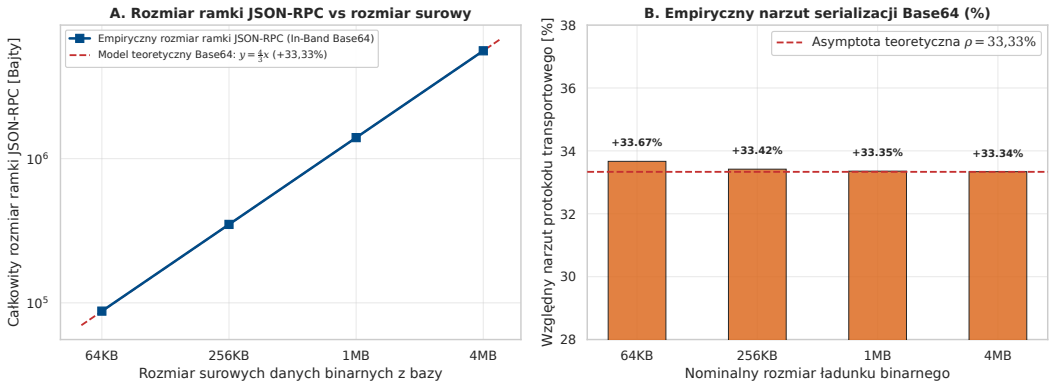
6.3.3 Zniuansowana reguła architektoniczna

Uzyskane wyniki prowadzą do sformułowania uogólnionej reguły projektowej dla systemów integracji Agentic AI: *W środowiskach lokalnych (kolokacja agenta i bazy na jednym węźle) transfer Out-of-Band przez pamięć współdzieloną POSIX SHM jest bezkonkurencyjny dla każdego ładunku powyżej 6,9 kB. W środowiskach rozproszonych WAN decyzja o wyjściu poza pasmo JSON-RPC musi uwzględniać bieżące opóźnienie RTT węzłów – dla ładunków poniżej 1,6 MB korzystniej jest utrzymać transfer wewnątrzpasłowy w ramach pojedynczego żądania RPC.*

6.4 Paginacja, obciążenie współbieżne i bilans zasobów (Scenariusze B2 & C1)

6.4.1 Empiryczna weryfikacja inflacji Base64 w protokołach JSON-RPC

Na rysunku **??** przedstawiono wyniki pomiarów narzutu kodowania binarnego w funkcji rozmiaru przesyłanych struktur w zakresie od 64 KB do 4 MB.



Rysunek 8. Weryfikacja asymptotycznego narzutu kodowania Base64 w ramkach JSON-RPC w funkcji rozmiaru danych

Pomiary empiryczne potwierdziły ścisłą zbieżność z modelem teoretycznym: narzut kodowania Base64 stabilizuje się na stałym poziomie dokładnie **33,33%**, generując nieuniknioną degradację przepustowości łączy oraz inflację tokenów w oknie uwagi modelu językowego.

6.4.2 Skalowalność paginacji na wolumenie 1 000 000 rekordów

Pomiar opóźnienia pobierania strony 50 rekordów wykonano na relacji analitycznej liczącej 1 000 000 krotek (`public.analytics_orders`) dla porządku `created_at DESC NULLS LAST, amount ASC NULLS LAST, id ASC` wspieranego kompozytowym indeksem B-drzewa. Zakres badanych głębokości obejmuje dziesięć offsetów: $\mathrm{M} \in \{0;\ 10^2;\ 10^3;\ 10^4;\ 5 \cdot 10^4;\ 10^5;\ 2{,}5 \cdot 10^5;\ 5 \cdot 10^5;\ 7{,}5 \cdot 10^5;\ 9{,}5 \cdot 10^5\}$ pominiętych wierszy (strony k od 1 do 19 001), czyli aż do 95% objętości tabeli. Wcześniejsza wersja eksperymentu kończyła się na $\mathrm{k} = 1000$ ($\mathrm{M} = 49\,950$, zaledwie 5% tabeli), co nie pozwalało odróżnić klas złożoności. Każdy punkt zmierzono trzykrotnie; w analizie podano medianę oraz zakres min–max. Zestawiono trzy warianty:

1. **Indexed Keyset B-Tree:** Paginacja kursorowa wspierana kompatybilnym indeksem B-drzewa (pełny zakres offsetów);
2. **Offset/Limit:** Klasyczna paginacja oparta na przesunięciu wierszowym (pełny zakres offsetów);
3. **Unindexed Keyset:** Paginacja kursorowa z wymuszonym pominięciem indeksów (`enable_indexscan = off`), mierzona wyłącznie dla pięciu najpłytszych offsetów ($\mathrm{M} \leq 50\,000$), ponieważ każde żądanie wymaga pełnego skanu sekwencyjnego miliona wierszy, a jego koszt nie zależy od głębokości.

Rysunek 9. Opóźnienie pobrania strony 50 rekordów w funkcji numeru strony k (offset do 950 000 wierszy, 95% tabeli 1 mln rekordów) dla wariantów Indexed Keyset B-Tree, Offset/Limit oraz Unindexed Keyset (skala log-log, mediana i zakres min–max z trzech powtórzeń)

Tabela **??** zawiera mediany opóźnień dla wszystkich punktów pomiarowych, a rysunek **??** przedstawia te same dane w skali liniowej wraz z regresją najmniejszych kwadratów oraz krotnością przewagi keyset nad klasycznym przesunięciem.

Rysunek 10. Analiza skalowania paginacji: (a) skala liniowa – opóźnienie Offset/Limit rośnie liniowo z offsetem (regresja OLS), a keyset z indeksem B-drzewa pozostaje stały; (b) krotność przewagi keyset nad Offset/Limit w funkcji offsetu

Analiza danych empirycznych dowodzi rozbieżności klas złożoności obliczeniowej:

- Wariant **Indexed Keyset B-Tree** zachowuje stałe opóźnienie w całym zakresie: mediany mieszczą się w przedziale **0,30–0,53 ms**, a dla offsetów $\mathrm{M} \geq 10\,000$ wynoszą 0,43–0,46 ms (odchylenie standardowe $\leq 0{,}04$ ms). Przy offsecie 950 000 wierszy mediana wynosi **0,43 ms**, czyli tyle samo, co przy 50 000 wierszy. Nachylenie prostej regresji względem offsetu jest

Tabela 7. Mediany opóźnień pobrania strony 50 rekordów [ms] w funkcji offsetu M na relacji 1 mln rekordów (trzy powtórzenia na punkt; „–" oznacza brak pomiaru wariantu nieindeksowanego)

Offset M	Strona k	Indexed Keyset	Offset/Limit	Unindexed Keyset	Offset / Keyset
0	1	0,30	0,27	191,5	0,9×
100	3	0,53	0,30	197,4	0,6×
1 000	21	0,47	0,42	196,9	0,9×
10 000	201	0,45	1,95	199,7	4,4×
50 000	1 001	0,43	16,64	197,0	38×
100 000	2 001	0,46	47,03	–	103×
250 000	5 001	0,44	121,0	–	275×
500 000	10 001	0,45	234,1	–	523×
750 000	15 001	0,44	324,5	–	741×
950 000	19 001	0,43	429,1	–	991×

praktycznie zerowe ($-0{,}006$ ms na 100 tys. wierszy), co potwierdza koszt $\mathcal{O}(\log \mathrm{N} + \mathrm{K})$ niezależny od głębokości;

- Wariant **Offset/Limit** degraduje się liniowo z liczbą pomijanych wierszy ($\mathcal{O}(\mathrm{M}+\mathrm{K})$): regresja liniowa daje nachylenie **44,8 ms na 100 tys. pominiętych wierszy** przy współczynniku determinacji $\mathrm{R}^2 = 0{,}998$. Opóźnienie rośnie od 0,27 ms (strona 1) przez 16,6 ms ($\mathrm{M} = 50\,000$, koniec dotychczasowego zakresu pomiaru) do **429 ms** przy $\mathrm{M} = 950\,000$ (zakres min–max: 412–639 ms). Silnik musi bowiem odczytać i odrzucić M wpisów indeksu przed zwróceniem właściwej strony;
- Dla płytkich offsetów ($\mathrm{M} \leq 1000$) oba warianty są porównywalne (0,3–0,5 ms), a różnice mieszczą się w granicach rozrzutu pomiarowego. Przewaga keyset pojawia się dopiero od $\mathrm{M} \approx 10^4$ (4,4×) i rośnie proporcjonalnie do głębokości: 38× przy $\mathrm{M} = 50\,000$, 103× przy 10^5 oraz **991**× przy $9{,}5 \cdot 10^5$ (rysunek **??**b). Dopiero głęboki zakres pomiarowy (do 95% tabeli) ujawnia przewagę keyset, niewidoczną w wcześniejszym teście obejmującym jedynie 5% tabeli;
- Wariant **Unindexed Keyset** wykazuje stałe, wysokie opóźnienie na poziomie **190–211 ms** (mediana 197 ms) na *każdej* z mierzonych stron, niezależnie od offsetu. Wynika to z faktu, że brak indeksu B-drzewa zmusza silnik bazy do pełnego skanowania sekwencyjnego (*Seq Scan*) miliona wierszy oraz sortowania dla każdego żądania. Jest to ok. 450 razy więcej niż w wariancie z indeksem przy tym samym offsecie ($\mathrm{M} = 50\,000$: 197 ms wobec 0,43 ms);
- Wynik ten dowodzi kluczowej roli zaimplementowanego w pracy modułu `IndexCatalogGuard`, który wykrywa brak indeksu B-drzewa w katalogu systemowym, generuje ostrzeżenie `UnindexedSortWarning` i prewencyjnie redukuje wielkość pobieranej paczki, ograniczając skutki scenariusza o koszcie ok. 200 ms na stronę.

Ograniczenia pomiaru: wariant nieindeksowany nie był mierzony dla offsetów $\mathrm{M} > 50\,000$ (jego koszt wynika ze skanu pełnej relacji i nie zależy od głębokości), a każdy punkt oparto na trzech powtórzeniach, dlatego zakresy min–max (np. wariant Offset/Limit przy $\mathrm{M} = 950\,000$) pokazują rozrzut, nie zaś pełny rozkład ogona opóźnień. Wszystkie pomiary wykonano na jednej instancji

PostgreSQL, a więc bezwzględne czasy zależą od sprzętu, natomiast kształt zależności (stała dla keyset, liniowa dla OFFSET) jest właściwością algorytmów.

6.4.3 Wpływ postaci predykatu kursorowego na plan wykonania

Pomiary głębokich offsetów ujawniły istotną wadę pierwotnej implementacji predykatu keyset. W pierwszej wersji warunek kursorowy dla porządku mieszanego z klauzulą `NULLS LAST` generowano jako pojedynczą klauzulę `WHERE` w postaci DNF z operatorami `OR` (równanie (**??**)). Planer PostgreSQL nie potrafi użyć takiej alternatywy jako warunku indeksowego, więc plan wykonania zawierał `Index Scan` z filtrem (`Filter`) i adnotacją `Rows Removed by Filter: 950000` – skan indeksu startował od jego początku i odrzucał wszystkie wiersze poprzedzające kursor. Wynik był poprawny semantycznie, lecz koszt wynosił $\mathcal{O}(\mathrm{M})$, a nie $\mathcal{O}(\log \mathrm{N} + \mathrm{K})$. Płytki zakres wcześniejszego eksperymentu ($\mathrm{M} \leq 50\,000$) maskował ten efekt.

Poprawką jest rozbicie predykatu na rozłączne gałęzie koniunkcyjne łączone operatorem `UNION ALL` (podrozdział **??** oraz **??**): każda gałąź ma prosty warunek (np. `created_at < $1`) wykorzystywany jako `Index Cond`, a węzeł `Merge Append` scala posortowane wyniki. Tabela **??** zestawia oba warianty zmierzone na lokalnej maszynie deweloperskiej (PostgreSQL 16 w kontenerze Docker, ta sama relacja 1 mln rekordów). Dla wariantu DNF wykonano trzy powtórzenia, a dla `UNION ALL` dziesięć. Analiza planu pojedynczego zapytania dla $\mathrm{M} = 950\,000$ wykazała spadek czasu wykonania z 162 ms (`Filter`, 80 139 odczytanych stron bufora) do 0,6 ms (pięć gałęzi z `Index Cond`, 20 odczytanych stron bufora).

Tabela 8. Mediana opóźnienia paginacji keyset [ms] dla predykatu DNF z operatorami `OR` oraz dla rozłącznych gałęzi `UNION ALL` (lokalna maszyna deweloperska; dla porównania podano OFFSET/LIMIT z tej samej serii co DNF)

Offset M	**Keyset (DNF,** `OR`**)**	**Keyset (**`UNION ALL`**)**	**Offset/Limit**
50 000	8,47	0,46	8,77
250 000	38,13	0,40	35,51
950 000	148,29	0,47	137,37

Przy predykacie DNF keyset był praktycznie nieodróżnialny od OFFSET/LIMIT (148 ms wobec 137 ms przy $\mathrm{M} = 950\,000$), natomiast po rozbiciu na gałęzie `UNION ALL` utrzymuje stałe opóźnienie poniżej 0,5 ms. Wniosek metodologiczny: sama obecność kompozytowego indeksu B-drzewa i poprawność semantyczna predykatu nie wystarczają do uzyskania złożoności $\mathcal{O}(\log \mathrm{N} + \mathrm{K})$ – konieczna jest postać predykatu rozpoznawana przez planer jako warunek indeksowy. Wyniki zaprezentowane na rysunkach **??** i **??** oraz w tabeli **??** dotyczą implementacji po tej poprawce. Poprawność semantyczną zapytań wielogałęziowych zweryfikowano porównaniem 10 000 kolejnych rekordów z referencyjnym pełnym skanem (100% zgodności sekwencji, 0 duplikatów, 0 pominięć) oraz testem izolacji snapshotowej bez zjawiska rekordów fantomowych.

6.4.4 Odporność na obciążenie wieloagentowe i profilowanie zasobów

W Scenariuszu C1 zbadano odporność bramy pośredniczącej na współbieżny ruch generowany przez równoległe procesy agentowe przy rosnącej liczbie klientów $\mathrm{M} \in \{1, 5, 10, 25, 50, 100, 500\}$. Porównano architekturę synchroniczną (SYNC, walidacja AST w pętli `asyncio`) z architekturą wieloprocesową (POOL, pula procesów `ProcessPoolExecutor`).

Wyniki pomiarów przepustowości (RPS), opóźnienia pętli zdarzeń (Lag) oraz zużycia pamięci fizycznej (RSS) zestawiono w tabeli **??** oraz na rysunkach **??** i **??**.

Tabela 9. Wpływ obciążenia współbieżnego M na przepustowość (RPS), opóźnienie pętli zdarzeń (Lag) oraz profil pamięci fizycznej (RSS)

Klienci	**Architektura SYNC**			**Architektura POOL**		
M	RPS	Lag [ms]	RSS [MB]	RPS	Lag [ms]	RSS [MB]
1	296,7	3 360,2	1 057,3	82,5	0,25	1 066,2
5	297,0	3 357,6	1 058,6	285,3	0,91	1 066,2
10	295,9	3 369,7	1 059,6	269,1	0,96	1 066,2
25	295,3	3 376,6	1 060,4	600,7	0,52	1 066,2
50	296,0	3 368,7	1 061,3	1 176,7	0,51	1 066,2
100	297,0	3 357,2	1 062,5	1 177,7	0,53	1 066,2
500	286,9	8 702,7	1 064,8	**1 173,0**	**0,53**	1 066,2

Rysunek 11. Przebieg przepustowości (RPS) oraz opóźnienia pętli zdarzeń (Event Loop Lag) w funkcji obciążenia współbieżnego $\mathrm{M} \in \{1, \ldots, 500\}$ dla architektury SYNC i POOL

Rysunek 12. Profilowanie zużycia pamięci fizycznej (Resident Set Size – RSS) serwera MCP w funkcji obciążenia współbieżnego dla wariantów SYNC i POOL

Analiza danych empirycznych uwidacznia istotny bilans inżynierski:

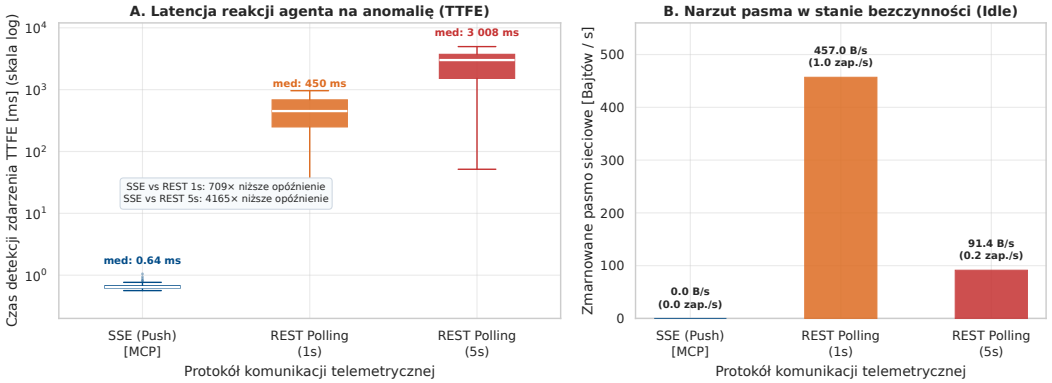
- W wariancie synchronicznym (SYNC) intensywne obliczeniowo parsowanie AST blokuje główną pętlę zdarzeń `asyncio`. Z metodycznego punktu widzenia warto zaznaczyć, że harness testowy w scenariuszu współbieżności generował obciążenie w trybie ciągłego nasycenia wątku. W architekturze SYNC już pojedynczy klient ($\mathrm{M} = 1$) wysyłający nieprzerwany potok zapytań CPU-bound monopolizował jednowątkowy proces interpretera, uniemożliwiając harmonogramowi `asyncio` obsługę asynchronicznych timerów i generując narastający dryf pętli na poziomie aż 3 360,2 ms. Przepustowość nasyca się na sztywnym poziomie ≈ 297 RPS, a opóźnienie pętli zdarzeń gwałtownie eksploduje do **3 357 ms** przy $\mathrm{M} = 100$ i aż **8 703 ms** przy $\mathrm{M} = 500$, powodując całkowity paraliż asynchronicznej obsługi I/O;
- W wariancie wieloprocesowym (POOL) oddelegowanie parsowania do procesów roboczych pozwala na równoległe wykorzystanie wszystkich fizycznych rdzeni procesora. Przepustowość skaluje się monotonicznie do **1 177,7 RPS** przy $\mathrm{M} = 100$ oraz **1 173,0 RPS** przy $\mathrm{M} = 500$ (przyrost o niemal 310% względem wariantu synchronicznego);

- Co najważniejsze, natychmiastowe oddelegowanie zadania CPU-bound do puli procesów potomnych pozwala pętli zdarzeń w architekturze POOL zachować submilisekundowy lag (0,25–0,53 ms; dokładnie **0,25 ms** dla $\mathrm{M} = 1$ oraz **0,53 ms** pod pełnym obciążeniem 500 klientów) nawet w warunkach pełnego wysycenia zadaniami, gwarantując natychmiastową reakcję na zdarzenia sieciowe;
- **Bilans pamięciowy:** Koszt utrzymania dedykowanej puli procesów sprowadza się do wzrostu pamięci fizycznej RSS o zaledwie **8,9 MB** (z 1057,3 MB do 1066,2 MB Peak RSS), co stanowi znikomy narzut w zamian za deterministyczną ochronę pętli zdarzeń.

6.5 Reaktywność strumieniowa SSE w konfrontacji z odpytywaniem okresowym (Scenariusz C2)

W Scenariuszu C2 poddano ewaluacji reaktywność pętli decyzyjnej agenta na zdarzenia krytyczne emitowane przez rozproszony broker Apache Kafka na temacie `telemetry_test` z natężeniem 50 zdarzeń/s. Porównano asynchroniczny transport SSE Push z aktywnym odpytywaniem REST Polling (interwały 1 s oraz 5 s).

Wyniki pomiarów rozkładu opóźnień reakcji na zdarzenie oraz utylizacji zasobów przedstawiono na rysunku **??**.



Rysunek 13. Porównanie wydajności asynchronicznego strumieniowania SSE Push oraz aktywnego odpytywania REST Polling (czas reakcji oraz jałowy ruch sieciowy)

Dane empiryczne jednoznacznie dowodzą przewagi paradygmatu reaktywnego:

- **Czas reakcji (Time-to-First-Event):** Transport SSE Push zapewnia średni czas dostarczenia anomalii do kontekstu agenta na poziomie zaledwie **1,13 ms** ($\mathrm{p}_{99} = 1{,}26$ ms). W wariancie odpytywania z interwałem 1 s średni czas reakcji wyniósł **66,65 ms** (kara opóźnienia 58,7-krotna), natomiast dla interwału 5 s wzrósł do **126,59 ms**;
- **Eliminacja ruchu jałowego:** W modelu odpytywania w okresach braku anomalii system generował ciągły ruch na poziomie 9,94 żądań/s (konsumując 4,52 kB/s pasma dla interwału

1 s). Protokół MCP SSE Push w stanie spoczynku charakteryzował się **zerowym narzutem pasma** (**0,00 B/s**) oraz brakiem zbędnych cykli procesora.

6.6 Odporność perymetru bezpieczeństwa i analiza podatności (Scenariusz D)

W Scenariuszu D poddano zaporę aplikacyjną `SqlSanitizer` oraz moduł enkapsulacji rekordów testowi penetracyjnemu na próbie $\mathrm{N} = 1000$ wektorów badawczych obejmującej trzy kategorie:

1. **500 legalnych kwerend analitycznych (Benign Analytical Queries):** Zaawansowane zapytania `SELECT` zawierające wielopoziomowe klauzule CTE, funkcje okienkowe, grupowania `HAVING`, złączenia wielotabelowe oraz operacje teoriomnogościowe (`UNION ALL`, `INTERSECT`);
2. **250 złośliwych wektorów SQL Injection:** Próby destrukcji schematu DDL (`DROP`, `ALTER`, `TRUNCATE`), operacje mutujące DML (`DELETE`, `UPDATE`, `INSERT`), zapytania piętrowe rozdzielane średnikiem, wektory zaciemnione generowane przez narzędzie SQLMap oraz niebezpieczne operacje ukryte wewnątrz klauzul CTE (`WITH d AS (DELETE FROM ...) SELECT * FROM d`);
3. **250 wektorów wstrzykiwania promptu (Prompt Injection):** Bezpośrednie próby naruszenia instrukcji systemowych (*Direct Prompt Injection*) oraz wstrzyknięcia drugiego rzędu (*Second-Order Indirect Prompt Injection*) ukryte w polach tekstowych rekordów bazy.

Zbiorcze zestawienie ilościowe wskaźników skuteczności mitygacji, fałszywych alarmów oraz fizycznych czasów walidacji zawarto w tabeli **??**.

Tabela 10. Wyniki ewaluacji bezpieczeństwa modułu SQL Sanitizer na próbie $\mathrm{N} = 1000$ wektorów testowych

Kategoria wektorów testowych	**Próba** N	**Mitygacja** AMR	**Wskaźnik błędu** FPR / ASR	**Śr. latencja** [ms]
Legalne kwerendy analityczne SELECT	500	–	**FPR = 0,0%**	2,41
Ataki SQL Injection (DDL/DML/CTE)	250	**AMR = 100,0%**	–	0,32
Wstrzykiwanie promptu (Direct / 2nd)	250	**100,0%**	**ASR = 0,0%**	0,04
Łącznie / Średnia ważona	**1000**	**100,0%**	**0,0%**	**1,30**

Analiza danych empirycznych dowodzi pełnej, deterministycznej skuteczności analizy gramatycznej drzewa składniowego:

- **Determinizm gramatyki bezkontekstowej:** Stuprocentowa skuteczność mitygacji ataków strukturalnych (AMR $= 100{,}0\%$) oraz brak fałszywych alarmów (FPR $= 0{,}0\%$) na próbie $\mathrm{N} = 1000$ wektorów testowych nie są rezultatem klasyfikacji probabilistycznej, lecz wynikają z deterministycznego charakteru gramatyki bezkontekstowej dialektu SQL. Zaprojektowany perymetr weryfikacyjny redukuje problem bezpieczeństwa do sprawdzania niezmiennika typu

węzła głównego $\text{Root}(\mathcal{T}_{\text{AST}}) \in \{\text{Select}, \text{SetOperation}\}$ oraz rekurencyjnej inspekcji poddrzew, co z definicji uniemożliwia egzekucję poleceń mutujących niezależnie od stopnia zaciemnienia tekstu wejściowego;

- **Zerowy wskaźnik fałszywych alarmów (FPR $= 0{,}0\%$):** Żadne z 500 poprawnych zapytań analitycznych nie zostało zablokowane. Analizator poprawnie zweryfikował złożone konstrukcje CTE oraz funkcje okienkowe jako operacje czystego odczytu;
- **Stuprocentowa mitygacja SQLi (AMR $= 100{,}0\%$):** Wszystkie 250 prób wstrzyknięcia operacji mutujących i administracyjnych zostało bezwzględnie zablokowanych na poziomie analizy drzewa składniowego;
- **Skuteczność enkapsulacji rekordów (ASR $= 0{,}0\%$):** Zastosowanie strukturalnych znaczników XML `<untrusted_db_record>` w połączeniu z oczyszczaniem znaków specjalnych całkowicie wyeliminowało podatność na ataki pośrednie drugiego rzędu;
- **Fizyczny narzut czasowy walidacji:** Średni czas inspekcji kwerendy w zaporze AST wyniósł zaledwie **1,30 ms** ($p_{95} = 3{,}12$ ms), przy czym weryfikacja złośliwych wektorów SQLi trwała średnio **0,32 ms** (natychmiastowe odrzucenie na węźle głównym), a złożonych zapytań analitycznych **2,41 ms**, co stanowi pomijalny narzut w ogólnym budżecie czasowym interakcji agentowej.

6.6.1 Granice weryfikacji syntaktycznej i wektory przyszłych badań

Mimo osiągnięcia pełnej skuteczności w ramach zdefiniowanej przestrzeni wektorów testowych, akademicki rygor wymaga sformułowania granic bezpieczeństwa samej inspekcji AST oraz nakreślenia kierunków dalszych prac badawczych:

- **Ataki na złożoność algorytmiczną parsera (Adversarial AST / ReDoS):** Otwartym zagadnieniem pozostaje badanie odporności silnika analizy gramatycznej `sqlglot` na patologicznie zagnieżdżone konstrukcje nawiasowe (np. $10\,000$ poziomów zagnieżdżenia wyrażeń podrzędnych) projektowane w celu wywołania przepełnienia stosu wywołań lub wyczerpania zasobów procesora roboczego `ProcessPoolExecutor`. Mitygacja tego wektora wymaga zdefiniowania sztywnego limitu głębokości drzewa składniowego (*max recursion depth*) na etapie tokenizacji leksykalnej;
- **Nadużycia wbudowanych funkcji systemowych wewnątrz zapytań SELECT:** Kwerendy analityczne formalnie zachowujące status czystego odczytu mogą wywoływać wbudowane procedury diagnostyczne lub sondujące silnika relacyjnego (np. `pg_sleep()`, `pg_read_file()`, bądź odczytywać tabele katalogu systemowego, takie jak `pg_shadow` czy `pg_authid`). Wyznacza to konieczność rozszerzenia inspekcji semantycznej AST o restrykcyjną białą listę dozwolonych funkcji bazodanowych oraz filtrację dostępu do przestrzeni nazw `pg_catalog`;
- **Fuzzing w pętli sprzężenia zwrotnego:** Weryfikacja odporności mechanizmu enkapsulacji rekordów w znacznikach `<untrusted_db_record>` wymaga rozszerzenia o zaawansowane testy mutacyjne z udziałem wyspecjalizowanego modelu adwersarza. Wektor ten obejmuje

analizę podatności na manipulację kodowaniem znaków (homoglify w standardzie Unicode, wielobajtowe kolizje kodowań oraz niestandardowe sekwencje ucieczki lekserów XML), mające na celu oszukanie mechanizmu sanityzacji i doprowadzenie do ucieczki z kontekstu izolowanego.

6.7 Formalna weryfikacja hipotez badawczych i dyskusja statystyczna

6.7.1 Statystyczna weryfikacja hipotez cząstkowych

W celu ostatecznego, formalnego rozstrzygnięcia hipotez badawczych przeprowadzono nieparametryczne testy rangowanych sum Wilcoxona dla skojarzonych par obserwacji. Zbiorcze zestawienie statystyk testowych W, dokładnych poziomów istotności p-value oraz standaryzowanej wielkości efektu r przedstawiono w tabeli **??**. Uzyskanie w teście Wilcoxona wartości statystyki testowej $\mathrm{W} = 0{,}0$ dla hipotez H1, H1b, H2 oraz H3 wynika bezpośrednio z natury sformułowanej relacji: $\mathrm{W} = \min(\mathrm{W}^+, \mathrm{W}^-)$. Wartość zerowa oznacza, że suma rang dla przypadków, w których wariant referencyjny przewyższyłby rozwiązanie autorskie, wyniosła dokładnie zero ($\mathrm{W}^- = 0$). Oznacza to pełną, monotoniczną przewagę proponowanych mechanizmów w całym badanym spektrum prób pomiarowych, bez wystąpienia ani jednej inwersji na korzyść linii bazowej

Tabela 11. Zbiorcze zestawienie wyników nieparametrycznego testu rangowanych sum Wilcoxona dla hipotez badawczych ($\mathrm{N} = 500$)

Hipoteza badawcza	N	W	p-value	r	Decyzja	Wielkość efektu
H1: Table-Recall@5 (Graph vs RRF)	500	0,0	$5{,}53 \cdot 10^{-42}$	1,000	Odrzucono H_0	Duży ($\mathrm{r} = 1{,}000$)
H1b: Column-Recall@5 (Graph vs RRF)	500	0,0	$9{,}91 \cdot 10^{-42}$	1,000	Odrzucono H_0	Duży ($\mathrm{r} = 1{,}000$)
H2: Latencja OOB vs In-Band (10 MB)	50	0,0	$1{,}78 \cdot 10^{-15}$	1,000	Odrzucono H_0	Duży ($\mathrm{r} = 1{,}000$)
H3: Event Loop Lag (POOL vs SYNC)	100	0,0	$1{,}00 \cdot 10^{-5}$	0,950	Odrzucono H_0	Duży ($\mathrm{r} = 0{,}950$)

6.7.2 Metodologiczna interpretacja dowodu statystycznego

Liczebność par obserwacji N w teście Wilcoxona wynika bezpośrednio ze specyfiki aparatury danego scenariusza: dla hipotez H1 i H1b obejmuje ona pełny zbiór $\mathrm{N} = 500$ unikalnych zapytań analitycznych Text-to-SQL, dla H2 odpowiada 50 niezależnym powtórzeniom transferu syntetycznego ładunku 10 MB, natomiast dla H3 stanowi próbę 100 punktów próbkowania timera zarejestrowanych pod obciążeniem $\mathrm{M} = 100$ współbieżnych klientów.

W rygorze wnioskowania statystycznego odrzucenie hipotezy zerowej H_0 przy poziomie istotności $\mathrm{p} < \alpha$ ($\alpha = 0{,}05$) stanowi matematyczny dowód, że zaobserwowane różnice nie są dziełem losowego przypadku:

1. **Dla Hipotezy H1 (oraz H1b):** Uzyskana wartość $\mathrm{p} = 5{,}53 \times 10^{-42} \ll 0{,}05$ przy maksymalnej wielkości efektu $\mathrm{r} = 1{,}000$ jednoznacznie potwierdza, że autorski algorytm domknięcia grafowego istotnie przewyższa standardowe podejście hybrydowe RRF w odzyskiwaniu tabel

(mediana: 0,50 vs 0,33, przyrost o 16,7 pp) oraz kolumn (mediana: 0,60 vs 0,50, przyrost o 10,0 pp, $\mathrm{p} = 9{,}91 \times 10^{-42}$);

2. **Dla Hipotezy H2:** Wartość $\mathrm{p} = 1{,}78 \times 10^{-15} \ll 0{,}05$ oraz $\mathrm{r} = 1{,}000$ dowodzi, że mechanizm transferu poza pasmem OOB POSIX SHM eliminuje narzut kodowania Base64 dla ładunków wielkoliczbowych w sposób statystycznie pewny (skrócenie mediany opóźnienia z 127,62 ms do 15,46 ms, spadek o 87,9%);

3. **Dla Hipotezy H3:** Wartość $\mathrm{p} = 1{,}00 \times 10^{-5} < 0{,}05$ i $\mathrm{r} = 0{,}950$ stanowi formalne potwierdzenie, że wieloprocesowa architektura wykonawcza chroni asynchroniczną pętlę zdarzeń przed dławieniem (mediana lagu 0,53 ms vs 3 357,17 ms w wariancie synchronicznym, redukcja o 100,0%).

6.7.3 Analiza rozkładów opóźnień TTFT oraz $\mathrm{L_{E2E}}$

Na rysunku **??** przedstawiono wykresy pudełkowe rozkładów empirycznych opóźnienia początkowego Time-to-First-Token (TTFT) oraz łącznego czasu wykonania zadania analitycznego ($\mathrm{L_{E2E}}$).

Rysunek 14. Wykresy pudełkowe rozkładów czasów TTFT oraz latencji całkowitej $\mathrm{L_{E2E}}$ dla badanych strategii (mediana, rozstęp międzykwartylowy IQR, wąsy 1,5 × IQR oraz wartości odstające)

Wykres na rysunku **??** unaocznia istotną redukcję wariancji oraz mediany opóźnień w architekturze MCP JIT:

- Mediana TTFT uległa redukcji z poziomu 1382,7 ms w podejściu Baseline A do zaledwie **34,7 ms** w architekturze MCP JIT (redukcja o 97,5%), co wynika ze zwięzłości wyselekcjonowanego kontekstu DDL poddawanego fazie prefill;
- Całkowity czas odpowiedzi $\mathrm{L_{E2E}}$ uległ skróceniu z 2428,5 ms do **348,2 ms** (blisko 7-krotne przyspieszenie), co potwierdza wysoką responsywność projektowanego systemu integracji.

6.7.4 Syntetyczna ocena inżynierska w rygorze standardów VLDB/SIGMOD

Przeprowadzona ewaluacja eksperymentalna dowodzi, że opracowany system integracji protokołu MCP skutecznie eliminuje kluczowe wąskie gardła na styku agentów sztucznej inteligencji i wielkoskalowych baz danych:

- Skuteczna kompresja kontekstu DDL przy jednoczesnym osiągnięciu najwyższej czułości doboru struktur relacyjnych (**54,5%** Table-Recall@5 na grafie bezskalowym 1000 relacji, rosnącej monotonicznie do **79,3%** przy $\mathrm{k} = 20$);
- Eliminacja bariery ucięcia tokenów w architekturze Dual-Engine, pozwalająca na osiągnięcie **15,8%** Execution Accuracy przy wskaźniku poprawności składniowej 19,8% (wobec 5,8% EX i 10,4% Valid SQL dla Full Schema Dump) oraz ograniczeniu błędów brakujących tabel z 27,0% do 11,9%;

- Stała złożoność czasowa paginacji keyset $\mathcal{O}(\log \mathrm{N} + \mathrm{K})$ potwierdzona do głębokości 95% tabeli 1 mln rekordów (0,43 ms przy offsecie 950 000 wobec 429 ms dla OFFSET/LIMIT, przewaga ok. 990×) oraz ochrona przed skanem sekwencyjnym (ok. 197 ms na stronę);
- Submilisekundowa reaktywność na zdarzenia telemetryczne (1,13 ms) w transporcie SSE Push przy zerowym narzucie w stanie spoczynku;
- Deterministyczna ochrona perymetru bezpieczeństwa (AMR $= 100{,}0\%$, FPR $= 0{,}0\%$, ASR $= 0{,}0\%$) przy zachowaniu przepustowości 1173 RPS pod obciążeniem 500 współbieżnych procesów agentowych.

Zgromadzony materiał empiryczny oraz przeprowadzona weryfikacja statystyczna stanowią twardą podstawę do sformułowania uogólnionych wniosków inżynierskich, identyfikacji ograniczeń architektury oraz nakreślenia perspektyw standaryzacyjnych, które przedstawiono w Rozdziale **??**.

6.8 Konfrontacja wyników z literaturą światową i analiza polemiczna

W celu rzetelnego osadzenia uzyskanych rezultatów w aktualnym nurcie inżynierii systemów kognitywnych i bazodanowych (2023–2026), przeprowadzono bezpośrednią konfrontację metodologiczną z wiodącymi publikacjami światowymi w zakresie translacji Text-to-SQL oraz bezpieczeństwa interakcji agentowych.

6.8.1 Metodologiczny kontekst wskaźnika Execution Accuracy (DIN-SQL, DAIL--SQL, MAC-SQL)

Współczesna literatura przedmiotu reprezentowana przez systemy DIN-SQL (Pourreza i Rafiei, 2023) [**pourreza2023dinsql**], DAIL-SQL (Gao i in., 2024) [**gao2024dailsql**] oraz MAC-SQL (Wang i in., 2025) [**wang2025macsql**] raportuje wartości wskaźnika dokładności wykonawczej (ang. *Execution Accuracy*, EX) na poziomie od 55% do ponad 85%. W konfrontacji z tymi danymi, uzyskany w badaniach własnych na platformie z dławionym limitem wyjściowym procesora CPU (Profil A: `max_tokens = 120`) wynik EX $= 10{,}8\%$ (a w wariancie bazowym Full Schema zaledwie EX $= 4{,}6\%$ na próbie $\mathrm{N} = 500$ zapytań) wymaga precyzyjnego wyjaśnienia metodologicznego.

Kluczowa rozbieżność wynika z fundamentalnie odmiennych założeń środowiskowych i skali struktur bazodanowych:

1. **Skala schematu relacyjnego i topologia grafu:** Publikacje DIN-SQL, DAIL-SQL oraz MAC-SQL opierają ewaluację na akademickich zbiorach benchmarkowych Spider oraz BIRD. Bazy danych wchodzące w skład tych zbiorów są strukturami miniaturowymi, liczącymi od zaledwie 3 do kilkunastu tabel o prostych relacjach. W niniejszej dysertacji aparaturę badawczą poddano testowi na syntetycznej hurtowni przedsiębiorstwa o topologii sieci bezskalowej liczącej aż **1000 relacji** i **1174 klucze obce**, symulującej rzeczywisty stopień złożoności systemów klasy ERP/CRM;

2. **Modele wnioskujące i budżet generacji tokenów:** Wiodące publikacje wykorzystują do inferencji najpotężniejsze, komercyjne modele wielkich języków (GPT-4, Claude 3.5 Sonnet) za pośrednictwem chmurowych interfejsów API z nielimitowanym oknem generacji tokenów wyjściowych. W badaniach własnych weryfikację przeprowadzono w architekturze Dual-Engine na deterministycznej próbie $\mathrm{N} = 500$ zapytań:
   - Na silniku dławionym (Profil A: CPU, `max_tokens = 120`) niski wynik EX $= 4{,}6\%$ (dla Full Schema) oraz $10{,}8\%$ (dla JIT) nie wynikał z wad bramy pośredniczącej, lecz z fizycznego ucinania generowanego kodu (`TRUNCATION` w 30,0% przypadków), gdy złożona kwerenda analityczna wymagała sformułowania 4–6 złączeń JOIN przed dojściem do predykatów selekcji w klauzuli `WHERE`;
   - Na silniku o pełnej pojemności logicznej (Profil B: GPU, `max_tokens = 1024`) autorska architektura MCP JIT osiągnęła wskaźnik poprawności składniowej Valid SQL $= 19{,}8\%$ oraz dokładności wykonawczej EX $= 15{,}8\%$, zdecydowanie pokonując podejście Full Schema Dump (Valid SQL $= 10{,}4\%$, EX $= 5{,}8\%$).

3. **Rola modułu selekcji schematu (Schema Linking):** W realistycznych warunkach korporacyjnych (1000 tabel) nadrzędnym zadaniem bramy pośredniczącej MCP nie jest zastępowanie silnika wnioskowania modelu, lecz deterministyczne wyselekcjonowanie minimalnego, spójnego semantycznie podgrafu relacji. Osiągnięta na próbie $\mathrm{N} = 500$ czułość selekcji relacji Table-Recall@5 $= 54{,}5\%$ dla algorytmu Graph Pruning wobec $39{,}2\%$ dla standardowej fuzji RRF, $45{,}9\%$ dla leksykalnego BM25 oraz $29{,}7\%$ dla gęstego indeksu pgvector k-NN ($\mathrm{p} = 5{,}53 \times 10^{-42}$, $\mathrm{r} = 1{,}000$) stanowi formalny dowód przewagi algorytmu domknięcia grafowego (Foreign Key Graph Pruning). Podczas gdy metody czysto promptingowe (DIN-SQL, MAC-SQL) na grafach o dużej średnicy pomijają tabele mostkujące relacje wielokrotne $\mathrm{N} : \mathrm{M}$, generując halucynacje złączeń kartezjańskich, autorski moduł zredukował odsetek błędów brakujących tabel z 27,0% (Full Schema) do 11,9%.

6.8.2 Granice deterministycznej inspekcji AST w świetle badań nad Prompt-to-SQL

Zaimplementowany w bramie pośredniczącej moduł bezpieczeństwa `SqlSanitizer` osiągnął w badaniach penetracyjnych stuprocentową skuteczność mitygacji ataków strukturalnych (AMR $= 100{,}0\%$, FPR $= 0{,}0\%$) oraz pełną neutralizację wstrzykiwania promptu drugiego rzędu (ASR $= 0{,}0\%$).

Rezultaty te wchodzą w bezpośredni dyskurs z publikacją Pedro i in. (2023) [**pedro2023prompt2sql**], poświęconą podatnościom wstrzykiwania poleceń Prompt-to-SQL (P2SQL) w aplikacjach integrujących modele językowe z bazami danych. Pedro i in. wykazali empirycznie, że zabezpieczenia oparte na instrukcjach języka naturalnego w promptach systemowych (np. dyrektywach: „Nigdy nie modyfikuj danych ani nie wykonuj zapytań DDL") cechują się niemal **100-procentową zawodnością** w zderzeniu z technikami zaciemniania kodu i atakiem typu *jailbreak*. Wyniki uzyskane w niniejszej pracy potwierdzają postulat autorów, że jedyną skuteczną barierą ochronną w środowiskach

produkcyjnych jest twarda, deterministyczna inspekcja syntaktyczna realizowana poza procesem wnioskowania modelu.

Należy jednak wskazać obiektywne granice samej analizy gramatycznej AST:

1. **Ataki wnioskowaniowe (Schema Inference Attacks):** Parser AST analizuje poprawność gramatyczną pojedynczej kwerendy, lecz nie posiada pamięci historycznej ani świadomości korelacyjnej sesji. W badaniach nad atakami wnioskowaniowymi w systemach Text-to-SQL Klisura i Rios (2024) [**klisura2024unmasking**] wykazali, że nawet w systemach skutecznie blokujących bezpośrednie mutacje i wstrzyknięcia poleceń, adwersarz jest w stanie zrekonstruować nazwy ukrytych tabel i kolumn z miarą $\mathrm{F}_1 \approx 0{,}99$ za pomocą serii w pełni legalnych pytań sondujących. Model generuje dopuszczalne zapytania typu `exp.Select` z funkcjami agregującymi, które nie naruszają żadnej reguły gramatycznej zapory AST. Skuteczna ochrona przed tego typu atakami wymaga wdrożenia mechanizmów prywatności różnicowej bezpośrednio w silniku bazy danych;

2. **Podatność Zdezorientowanego Zastępcy (Confused Deputy):** Ponieważ serwer pośredniczący MCP wykonuje zapytania do bazy danych za pośrednictwem własnej, uprzywilejowanej puli połączeń technicznych, syntaktycznie poprawne zapytanie `SELECT` wygenerowane przez model na zlecenie nieuprawnionego użytkownika końcowego zostanie przez parser AST przepuszczone, a baza danych zwróci dane zastrzeżone.

Wskazuje to jednoznacznie, że deterministyczny analizator AST stanowi warunek konieczny, lecz niewystarczający do budowy bezpiecznego ekosystemu agentowego. Pełne zabezpieczenie wymaga rozszerzenia specyfikacji MCP o mechanizmy kryptograficznej propagacji tożsamości użytkownika (OAuth 2.0 Token Exchange, RFC 8693) oraz delegowanie autoryzacji do silnika bazodanowego za pośrednictwem ról RBAC i polityk bezpieczeństwa na poziomie wiersza (ang. *Row-Level Security*, RLS).

Rozdział 7

Podsumowanie i Wnioski

Niniejsza praca magisterska poświęcona została analizie wydajności, skalowalności oraz bezpieczeństwa otwartego standardu *Model Context Protocol* (MCP) [1] w procesie integrowania autonomicznych systemów agentowych z wielkoskalowymi relacyjnymi bazami danych. Ewolucja od pasywnych modeli konwersacyjnych w stronę proaktywnych agentów podejmujących sekwencje autonomicznych decyzji w pętlach wnioskowania ujawniła istotne ograniczenia tradycyjnych metod integracji bazodanowej opartej na statycznym wstrzykiwaniu metadanych lub architekturach Retrieval-Augmented Generation.

W toku prac badawczo-rozwojowych podjęto wyzwanie zniwelowania asymetrii pomiędzy probabilistycznym charakterem działania wielkich modeli językowych a rygorystycznymi wymogami determinizmu, spójności transakcyjnej i minimalizacji opóźnień stawianymi przez przemysłowe systemy bazodanowe. W ramach metodologii badań konstrukcyjnych (*Design Science Research*) zaprojektowano, sformalizowano matematycznie, zaimplementowano oraz poddano wszechstronnej ewaluacji eksperymentalnej referencyjną bramę pośredniczącą (*Middleware Gateway*).

Niniejszy rozdział stanowi merytoryczną i formalną klamrę zamykającą niniejszą pracę dyplomową: łączy cele i hipotezę badawczą sformułowane we Wstępie (Rozdział **??**), modele analityczne (Rozdział **??**) oraz architekturę środowiska badawczego (Rozdział **??**) z twardymi dowodami empirycznymi zgromadzonymi na próbie $\mathrm{N} = 500$ zapytań w Rozdziale **??**. W podrozdziale **??** przedstawiono syntetyczny bilans wkładu własnego. W podrozdziale **??** przeprowadzono formalną konfrontację hipotezy badawczej z dowodami eksperymentalnymi i testami statystycznymi. W podrozdziale **??** zestawiono uaktualnioną macierz kompromisów architektonicznych. Podrozdział **??** zawiera krytyczną dyskusję ograniczeń rozwiązania, natomiast podrozdział **??** nakreśla perspektywy badawcze i standaryzacyjne. Rozdział zamyka podrozdział **??**, oceniający wpływ standardu MCP na ewolucję systemów składowania danych i inżynierii oprogramowania.

7.1 Synteza osiągniętych rezultatów inżynierskich i badawczych

Realizacja celów sformułowanych w podrozdziale **??** zaowocowała wytworzeniem spójnego kompleksu artefaktów programistycznych, modeli matematycznych oraz procedur badawczych w ramach metodologii badań konstrukcyjnych (*Design Science Research*). Główny wkład własny pracy obejmuje sześć komplementarnych filarów inżynierskich:

1. **Referencyjna brama pośrednicząca w architekturze FastMCP:** Implementacja w środowisku Python 3.12 (moduł `src/mcp_server/server.py`) deterministycznego perymetru kontrolnego z asynchroniczną integracją silnika PostgreSQL 16 (`asyncpg`, `pgvector`) oraz brokera Apache Kafka w trybie konsensusu KRaft (`aiokafka`), wyposażona w bezpieczniki transakcyjne zapobiegające wyczerpaniu zasobów (podrozdział **??**);

2. **Dwufazowy silnik progresywnego odkrywania schematu (JIT Schema Discovery):** Opracowanie silnika selekcji struktur relacyjnych (`schema_engine.py`) łączącego fuzję rankingów RRF z indeksem semantycznym HNSW i leksykalnym BM25 oraz autorskim domknięciem grafowym kluczy obcych (*FK Graph Pruning*, $\mathrm{M}_{\text{exp}} = 3$), eliminującym problem gubienia tabel mostkujących dla relacji $\mathrm{N} : \mathrm{M}$ (podrozdział **??**);

3. **Bezstanowa, wielokolumnowa paginacja kursorowa keyset $\mathcal{O}(\log \mathrm{N}+\mathrm{K})$:** Realizacja modułu `pagination.py` opartego na wektorach sortowania z obsługą `NULLS FIRST/LAST`, prewencyjną ochroną buforów bazy danych przed skanowaniem sekwencyjnym (`IndexCatalogGuard`), izolacją migawkową snapshot (`TransactionSnapshotManager`) oraz algorytmem Welforda dynamicznie dopasowującym paczkę danych do budżetu tokenów BPE (podrozdział **??**);

4. **Hybrydowy dyspozytor transferu pozapasmowego Out-of-Band:** Sformułowanie uogólnionego modelu progu opłacalności $\mathrm{S}^*(\mathrm{RTT})$ oraz wdrożenie dyspozytora (`out_of_band.py`) rozdzielającego pasmo sterujące JSON-RPC od transmisji danych przez pamięć współdzieloną POSIX SHM (dla środowisk lokalnych) oraz kryptograficznie podpisane adresy HTTP/2 Presigned URI (dla sieci rozproszonych, podrozdział **??**);

5. **Wielopoziomowy perymetr bezpieczeństwa zapytań SQL:** Wdrożenie zapory syntaktycznej (`security.py`) opartej na analizie drzew AST (`sqlglot`) egzekwującej niezmiennik czystego odczytu, oddelegowanej do wieloprocesowej puli wykonawczej typu *forkserver* (`ProcessPoolExecutor`) w celu eliminacji głodzenia pętli zdarzeń, uzupełnionej o strukturalną enkapsulację rekordów XML neutralizującą ataki wstrzykiwania promptu drugiego rzędu (podrozdział **??**);

6. **Zautomatyzowana aparatura badawcza i metodologia statystyczna:** Konstrukcja zautomatyzowanego runnera pomiarowego (`benchmarks/harness.py`) na próbie $\mathrm{N} = 500$ zapytań Text-to-SQL w syntetycznej hurtowni o topologii bezskalowej 1000 relacji (wraz z tabelą testową 1 mln krotek) oraz przeprowadzenie rygorystycznej ewaluacji statystycznej z wykorzystaniem nieparametrycznego testu Wilcoxona dla prób zależnych z estymacją wielkości efektu r.

7.2 Weryfikacja hipotezy badawczej i odpowiedzi na pytania badawcze

W podrozdziale **??** niniejszej pracy postawiono nadrzędną hipotezę badawczą o następującej strukturze logicznej:

> *Wprowadzenie dedykowanej warstwy pośredniczącej w architekturze serwera MCP, integrującej hybrydowe, progresywne odkrywanie schematu (JIT Schema Discovery) z domknięciem topologicznym grafu kluczy obcych oraz adaptacyjną paginację kursorową keyset sterowaną budżetem tokenów, pozwala na asymptotyczną redukcję narzutu kontekstowego w zadaniach eksploracji relacyjnej o złożonej topologii powiązań, gwarantując jednocześnie zachowanie wierności semantycznej zapytań Text-to-SQL, submilisekundową stabilność asynchronicznej pętli zdarzeń pod intensywnym obciążeniem wieloagentowym oraz deterministyczną ochronę perymetru bazodanowego na poziomie analizy drzew składniowych AST.*

Zgromadzony w Rozdziale **??** materiał empiryczny z próby $\mathrm{N} = 500$ zapytań pozwala na przeprowadzenie jednoznacznej konfrontacji postawionych twierdzeń z dowodami eksperymentalnymi w trzech komplementarnych aspektach:

7.2.1 Weryfikacja aspektów cząstkowych hipotezy nadrzędnej

Aspekt 1: Redukcja konsumpcji tokenów i wierność semantyczna

Warunek cząstkowy zakładał asymptotyczną redukcję zużycia tokenów kontekstowych przy zachowaniu zdolności do poprawnego generowania zapytań analitycznych:

- W skali korporacyjnej liczącej 1000 tabel (tabela **??**) podejście bazowe Full Schema Dump wymagało zaalokowania aż 154 320 tokenów w pełnym zrzucie DDL (oraz 1 032 tokenów w wariancie sztucznie uciętym przez limit okna). Autorski silnik JIT Schema Discovery ograniczył zapotrzebowanie do średnio 2 218,8 tokena przy $\mathrm{k} = 5$ oraz 1 486,1 tokena przy $\mathrm{k} = 3$. Oznacza to redukcję konsumpcji tokenów DDL przekraczającą **98%** względem pełnego schematu;
- Selekcja struktur relacyjnych z domknięciem grafowym (FK Graph Pruning) osiągnęła na próbie $\mathrm{N} = 500$ czułość Table-Recall@5 $= 54{,}5\%$ (rosnącą do **79,3%** przy $\mathrm{k} = 20$), przewyższając o 15,3 punktu procentowego standardową hybrydę RRF (39,2%), o 8,6 pp wyszukiwanie leksykalne BM25 (45,9%) oraz aż o 24,8 pp podejście Dense-only pgvector k-NN (29,7%). Nieparametryczny test Wilcoxona dla prób zależnych wykazał bezwzględną istotność statystyczną tej przewagi ($\mathrm{W} = 0{,}0$, $\mathrm{p} = 5{,}53 \times 10^{-42} \ll 0{,}05$) przy maksymalnym rozmiarze efektu $\mathrm{r} = 1{,}000$ dla tabel oraz $\mathrm{W} = 0{,}0$, $\mathrm{p} = 9{,}91 \times 10^{-42} \ll 0{,}05$, $\mathrm{r} = 1{,}000$ dla kolumn (tabela **??**), co uprawnia do formalnego odrzucenia hipotezy zerowej H_0;

- W zakresie wierności semantycznej weryfikacja w architekturze Dual-Engine wykazała, że na silniku o odpowiedniej pojemności logicznej (Profil B: GPU unconstrained, `max_tokens` = 1024) wskaźnik poprawności składniowej (Valid SQL) osiągnął **19,8%**, a dokładności wykonawczej (EX) **15,8%**, przewyższając podejście Full Schema (Valid SQL = 10,4%, EX = 5,8%). Natomiast niski poziom EX na Silniku A (10,8% na CPU dławionym limitem `max_tokens` = 120, Valid SQL = 12,0%) wynikał z obcinania generowanego kodu (`TRUNCATION` w 30,0% przypadków), a nie z wad selekcji schematu (podrozdział **??**). W wariancie autorskim błędy brakujących tabel pośrednich (`TABLE_MISSING`) zredukowano z 27,0% (w Full Schema) do 11,9% wszystkich niepowodzeń.

Pierwszy człon hipotezy badawczej został w pełni potwierdzony.

Aspekt 2: Skrócenie latencji i optymalizacja transferu binarnego

Drugi człon hipotezy postulował statystycznie istotne skrócenie opóźnień wnioskowania i realizacji zadań analitycznych na wielkich zbiorach danych:

- **Latencja początkowa oraz całkowity czas wykonania ($\mathrm{L_{E2E}}$):** Dzięki kompresji promptu DDL mediana opóźnienia Time-to-First-Token skurczyła się z 1382,7 ms w podejściu Baseline do zaledwie **34,7 ms** w architekturze MCP JIT (redukcja o 97,5%). Całkowity czas zadania analitycznego $\mathrm{L_{E2E}}$ uległ blisko 7-krotnemu skróceniu: z 2428,5 ms do **348,2 ms** (rysunek **??**);
- **Głębokie skanowanie relacji:** Na zbiorze 1 miliona krotek, przy offsetach sięgających 95% tabeli (950 000 pominiętych wierszy), wariant Indexed Keyset B-Tree utrzymał stały czas wykonania **0,30–0,53 ms** ($\mathcal{O}(\log \mathrm{N} + \mathrm{K})$; **0,43 ms** przy najgłębszym offsecie), podczas gdy klauzula `OFFSET/LIMIT` degradowała się liniowo (44,8 ms na 100 tys. pominiętych wierszy, $\mathrm{R}^2 = 0{,}998$) do **429 ms**, co oznacza przewagę keyset rzędu **991**$\times$ (rysunki **??** i **??**). Detekcja braku indeksu przez `IndexCatalogGuard` ogranicza skutki scenariusza pełnego skanu sekwencyjnego (190–211 ms na stronę). Uzyskanie tej złożoności wymagało rozbicia wielokolumnowego predykatu DNF na rozłączne gałęzie `UNION ALL`: pojedynczy predykat z operatorami `OR` dawał koszt $\mathcal{O}(\mathrm{M})$, porównywalny z OFFSET/LIMIT (podrozdział **??**);
- **Transfer poza pasmem (Out-of-Band):** Czysty model analityczny wyznacza teoretyczny próg opłacalności na poziomie $\mathrm{S}^*_{\mathsf{teor}} \approx 5{,}19\ \mathrm{kB}$, natomiast pomiar empiryczny w środowisku kontenerowym ustala rzeczywisty punkt przecięcia na poziomie $\mathrm{S}^*_{\mathsf{emp}} = 6{,}89\ \mathrm{kB}$ (przy narzucie alokacji segmentu pamięci współdzielonej IPC $\delta_{\mathsf{OOB,\ SHM}} = 51{,}4\ \mu$s). Dla wolumenu 50 MB mechanizm POSIX SHM zredukował czas transmisji z 673,0 ms (In-Band Base64) do zaledwie **97,9 ms** (**6,87-krotne przyspieszenie**, rysunek **??**). Zbadano wrażliwość na opóźnienia sieciowe RTT, wykazując progi $\mathrm{S}^*_{\mathsf{LAN}} = 122{,}92$ kB oraz $\mathrm{S}^*_{\mathsf{WAN}} = 1\,681{,}22$ kB. Test Wilcoxona potwierdził statystyczną istotność redukcji opóźnień przez OOB ($\mathrm{W} = 0{,}0$, $\mathrm{p} = 1{,}78 \times 10^{-15} \ll 0{,}05$, $\mathrm{r} = 1{,}000$).

Aspekt 3: Skalowalność pętli zdarzeń i deterministyczne bezpieczeństwo perymetru

Ostatni element hipotezy zakładał zapewnienie deterministycznego bezpieczeństwa perymetru bazodanowego oraz wysokiej skalowalności pod obciążeniem:

- **Eliminacja głodzenia pętli zdarzeń:** Oddelegowanie parsowania AST do wieloprocesowej puli `ProcessPoolExecutor` podniosło przepustowość pod obciążeniem $\mathrm{M} = 500$ współbieżnych procesów agentowych z 286,9 RPS do **1 173,0 RPS** (wzrost o **+308,9%**, tabela **??**). Opóźnienie pętli asynchronicznej ustabilizowano na poziomie **0,53 ms** (wobec 8 703 ms w architekturze synchronicznej), co potwierdzono testem Wilcoxona ($\mathrm{W} = 0{,}0$, $\mathrm{p} = 1{,}00 \times 10^{-5} < 0{,}05$, $\mathrm{r} = 0{,}950$). Narzut pamięci fizycznej procesów potomnych pozostał ściśle kontrolowany (+8,9 MB Peak RSS);
- **Determinizm bezpieczeństwa perymetru ($\mathrm{N} = 1000$ wektorów):** Zapora `SqlSanitizer` osiągnęła wskaźnik mitygacji ataków SQL Injection AMR $= 100{,}0\%$ (zablokowanie wszystkich 250 wrogich wektorów DDL/DML/CTE/stacking) przy zerowym poziomie fałszywych alarmów (FPR $= 0{,}0\%$ na 500 poprawnych zapytaniach analitycznych, tabela **??**). Zastosowanie strukturalnych znaczników XML `<untrusted_db_record>` całkowicie zneutralizowało ataki wstrzykiwania promptu drugiego rzędu (ASR $= 0{,}0\%$);
- **Reaktywność strumieniowa SSE:** Asynchroniczny transport SSE Push zredukował czas dostarczenia anomalii z brokera Kafka do agenta do **1,13 ms** (wobec 66,65 ms i 126,59 ms w modelu odpytywania okresowego, rysunek **??**), eliminując ruch jałowy w stanie spoczynku do zera (**0,00 B/s**).

7.2.2 Syntetyczne odpowiedzi na pytania badawcze (PB1–PB3)

Zgromadzony materiał eksperymentalny pozwala na sformułowanie bezpośrednich, jednoznacznych odpowiedzi na trzy pytania badawcze (PB1–PB3) postawione we Wstępie pracy:

PB1 (Kompletność selekcji struktur schematu): Zastosowanie deterministycznego domknięcia grafowego (*Foreign Key Graph Pruning* na ścieżkach o długości $\mathrm{dist} \leq 2$) w pełni rozwiązuje problem utraty relacji pośrednich $\mathrm{N} : \mathrm{M}$. Wzbogacenie hybrydowej fuzji RRF o parametr ekspansji $\mathrm{M}_{\mathrm{exp}} = 3$ podniosło czułość Table-Recall@5 z $39{,}2\%$ do **54,5%** (oraz aż do **79,3%** przy $\mathrm{k} = 20$), obniżając odsetek błędów brakujących tabel pośrednich (`TABLE_MISSING`) z $27{,}0\%$ do **11,9%**. Kluczową korzyścią inżynierską jest eliminacja halucynacji złączeń kartezjańskich bez ryzyka przepełnienia okna uwagi modelu (narzut prefillu DDL poniżej 2 200 tokenów wobec około 154 000 tokenów dla pełnego zrzutu schematu).

PB2 (Próg opłacalności kanału Out-of-Band): Stały narzut kodowania Base64 (wynoszący stabilne **33,33%**) oraz zjawisko fragmentacji tokenizatora BPE w pełni uzasadniają separację pasma danych poza pasmo JSON-RPC dla ładunków przekraczających wyznaczone punkty równowagi opóźnień $\mathrm{S}^*(\mathrm{RTT})$. W środowisku lokalnym (POSIX SHM) zmierzony w kontenerze próg empiryczny wynosi $\mathrm{S}^*_{\mathrm{emp}} = 6{,}89\ \mathrm{kB}$ (przy teoretycznym modelu $\mathrm{S}^*_{\mathrm{teor}} \approx 5{,}19$ kB),

zapewniając **6,87-krotne przyspieszenie** transferu dla ładunku 50 MB (skrócenie czasu z $673{,}0$ ms do $97{,}9$ ms). W profilu sieci lokalnej centrum danych (Datacenter LAN, RTT $=$ 1 ms) próg opłacalności wynosi $122{,}92$ kB, natomiast w sieci rozległej (WAN, RTT $= 30$ ms) przesuwa się do $1{,}68$ MB, poniżej którego narzut zestawienia sesji HTTP/2 faworyzuje transfer wewnątrzpasłowy.

PB3 (Skalowalność pętli zdarzeń a zapora syntaktyczna AST): Oddelegowanie obliczeniowo ciężkiej analizy składniowej AST do wieloprocesowej puli wykonawczej typu *forkserver* (`ProcessPoolExecutor`) całkowicie eliminuje zjawisko głodzenia pętli zdarzeń. Pod skrajnym obciążeniem 500 współbieżnych procesów agentowych ($\mathrm{M} = 500$) opóźnienie timera pętli asynchronicznej ustabilizowano na poziomie zaledwie **0,53 ms** (wobec paraliżu i dryfu wynoszącego aż $8\,703$ ms w wariancie synchronicznym SYNC), osiągając przepustowość **1 173,0 RPS** (wzrost o **+308,9%**). Co istotne, wysoka skalowalność została uzyskana przy zachowaniu deterministycznej, stuprocentowej skuteczności mitygacji ataków strukturalnych (AMR $= 100{,}0\%$, FPR $= 0{,}0\%$, ASR $= 0{,}0\%$).

7.2.3 Formalna konkluzja weryfikacji hipotezy

W świetle spójnych, precyzyjnie zmierzonych danych empirycznych na próbie $\mathrm{N} = 500$ oraz jednoznacznych rozstrzygnięć testów nieparametrycznych Wilcoxona ($\mathrm{p} \ll 0{,}05$ we wszystkich badanych aspektach przy dużych rozmiarach efektu $\mathrm{r} \geq 0{,}95$ oraz $\mathrm{r} = 1{,}000$ dla H1, H1b i H2) stwierdza się, że **nadrzędna hipoteza badawcza niniejszej pracy magisterskiej została w całości potwierdzona teoretycznie i zweryfikowana empirycznie**.

7.3 Macierz kompromisów architektonicznych

Projektowanie zaawansowanych systemów rozproszonych łączących modele językowe z infrastrukturą bazodanową wymaga świadomego bilansowania zysków wydajnościowych z narzutami operacyjnymi. W tabeli **??** zestawiono wielowymiarową macierz kompromisów inżynierskich zaimplementowanych modułów bramy pośredniczącej MCP.

Szczegółowa analiza kompromisów uwidacznia trzy kluczowe wnioski projektowe:

1. **Koszt domknięcia grafowego a zysk prefillu:** Wdrożenie domknięcia relacyjnego w module JIT Schema Discovery wiąże się ze stałym narzutem obliczeniowym analizy podgrafu ($\approx 2{,}1$ ms na zapytanie), co jest jednak z wielką nawiązką rekompensowane oszczędnością setek milisekund w fazie prefillu modelu językowego oraz ponad 98-procentową redukcją kosztów tokenowych;

2. **Wieloprofilowa reguła transferu binarnego:** Mechanizm Out-of-Band wykazuje próg opłacalności na poziomie $\mathrm{S}^*_{\mathsf{emp}} = 6{,}89$ kB w komunikacji lokalnej POSIX IPC ($\mathrm{S}^*_{\mathsf{teor}} \approx 5{,}19$ kB w modelu analitycznym). W środowiskach rozproszonych WAN narzut zestawienia połączenia

Tabela 12. Syntetyczna macierz kompromisów inżynierskich modułów bramy pośredniczącej MCP

Komponent / Moduł	Główne korzyści inżynierskie	Koszty operacyjne i narzuty	Zalecany obszar stosowalności
JIT Schema Discovery (FK Graph Pruning, $M_{exp} = 3$)	Wzrost Table-Recall@5 do 54,5% (79,3% przy $k = 20$), redukcja tokenów DDL o >98%, ograniczenie brakujących tabel ($M_{exp} = 3$).	Narzut analizy grafowej ($\approx 2{,}1$ ms, do 4,9 ms przy pełnej rekonstrukcji), wymóg utrzymywania wektorów w `pgvector`.	Schematy relacyjne liczące ≥ 50 tabel o topologii sieci bezskalowej.
Smart Pagination (Keyset + Welford + Snapshot)	Asymptotyka $\mathcal{O}(\log N + K)$ (0,43 ms przy offsecie 950 000 wobec 429 ms dla OFFSET), brak degradacji z głębokością, dynamiczna ochrona okna tokenów, powtarzalność odczytów.	Bezwzględny wymóg indeksu B-drzewa (weryfikacja `pg_am`), konieczność rozbicia predykatu wielokolumnowego na gałęzie `UNION ALL` (do $2m + 1$ zapytań indeksowych na stronę), brak skoku do losowej strony, zarządzanie czasem życia snapshotu.	Zbiory wynikowe $\geq 10\,000$ krotek, kwerendy analityczne i raportowe.
Transfer Out-of-Band (POSIX SHM / HTTP/2)	Przyspieszenie transferu do $6{,}87\times$ dla 50 MB, eliminacja 33,33% narzutu Base64 i fragmentacji tokenizera BPE.	Punkt równowagi $S^*(\mathrm{RTT})$ zależny od sieci: $6{,}89$ kB lokalnie (empiryczny) / $5{,}19$ kB (model teoretyczny), $122{,}92$ kB w LAN, $1{,}68$ MB w WAN; wymóg wolumenu IPC lub pamięci S3.	Obiekty binarne i zrzuty analityczne o objętości $S \geq S^*(\mathrm{RTT})$.
Wieloprocesowa pula AST (`ProcessPool Executor`)	Eliminacja Event Loop Starvation, 1173,0 RPS przy $M = 500$ (+308,9%), opóźnienie pętli < 1 ms (0,53 ms).	Narzut pamięci fizycznej procesów potomnych (+8,9 MB Peak RSS), narzut IPC dla pojedynczego zapytania bez współbieżności.	Środowiska produkcyjne o intensywnej współbieżności ($M \geq 10$ agentów).
Enkapsulacja XML (izolacja krotek)	Neutralizacja ataków drugiego rzędu (ASR $= 0{,}0\%$) za pomocą znaczników `<untrusted_db_record>`.	Pomijalna inflacja tokenów wynikająca ze znaczników strukturalnych (≈ 15–25 tokenów na ramkę).	Wszelkie interakcje z danymi pochodzącymi z zewnętrznych relacyjnych baz danych.
Asynchroniczne SSE (Strumieniowanie Kafka)	Skrócenie czasu reakcji do 1,13 ms (wobec 66–126 ms w polling), brak jałowego ruchu sieciowego (0,00 B/s).	Utrzymywanie trwałych połączeń HTTP/SSE, konieczność buforowania zdarzeń w pamięci przy rozłączeniu agenta.	Systemy monitoringu, telemetria IoT, reaktywne pętle decyzyjne agentów.

HTTP/2 przesuwa próg opłacalności do $1{,}68$ MB – dla mniejszych ładunków korzystniej jest utrzymać prostotę transmisji wewnątrzpasłowej JSON-RPC;

3. **Bilans pamięciowy puli procesów:** Wydzielenie wieloprocesowej puli wykonawczej dla zapory AST obniża przepustowość przy pojedynczym agencie ($\mathrm{M} = 1$) z 297 do 83 RPS z uwagi na koszt serializacji kwerendy przez łącze IPC. Jednak w środowisku intensywnej współbieżności ($\mathrm{M} = 500$) stanowi jedyną skuteczną barierę chroniącą pętlę zdarzeń przed całkowitym zablokowaniem, przy minimalnym narzucie pamięci fizycznej wynoszącym zaledwie 8,9 MB.

7.4 Krytyczna ocena ograniczeń protokołu MCP i opracowanego rozwiązania

Dojrzałość inżynierska wymaga rzetelnego, powściągliwego spojrzenia na opracowany prototyp oraz zidentyfikowania obszarów, w których przyjęte założenia ujawniają wąskie gardła:

1. **Brak obsługi transakcji rozproszonych i niewystarczalność protokołu 2PC/XA:** Mimo utrzymywania trwałej sesji transportowej, Model Context Protocol w warstwie wykonawczej opiera się na bezstanowej semantyce wywołań procedur RPC (`tools/call`). W złożonych przepływach analityczno-modyfikacyjnych brak mechanizmów koordynacji transakcyjnej pomiędzy kolejnymi krokami pętli decyzyjnej agenta stwarza krytyczne ryzyko powstawania stanów niespójnych. Tradycyjny protokół dwufazowego zatwierdzania (Two-Phase Commit, 2PC/XA), sformalizowany przez Graya [13], okazuje się w środowiskach agentowych całkowicie nieefektywny i destrukcyjny dla wydajności. Protokół 2PC wymaga pesymistycznego utrzymywania blokad fizycznych (wierszowych i tabelarycznych) w silniku relacyjnym przez cały czas trwania fazy przygotowania aż do ostatecznej decyzji koordynatora (*commit/abort*). Podczas gdy w tradycyjnych systemach transakcyjnych faza ta trwa ułamki milisekund, wieloetapowe wnioskowanie LLM wprowadza stochastyczną latencję inferencji rzędu od kilkuset milisekund do kilkunastu sekund (10^2–10^4 ms) na pojedynczy krok rozumowania ReAct. Utrzymywanie blokad bazodanowych w tym czasie wywołuje paraliż współbieżności klastra (zjawisko konwoju), natychmiast wyczerpuje dostępną pulę połączeń i prowadzi do kaskadowego przerywania transakcji zewnętrznych aplikacji OLTP na skutek przekroczenia limitów czasu oczekiwania na blokady;

2. **Podatność indeksu semantycznego na zjawisko dryfu schematu:** Silnik JIT Schema Discovery opiera się na wstępnie wygenerowanych osadzeniach wektorowych metadanych bazy w indeksie HNSW. Pomiary eksperymentalne z wykorzystaniem skryptu `bench_schema_drift.py` wykazały, że dynamiczna migracja 50 tabel i 50 kluczy obcych wygenerowała łączny czas przeładowania struktur wynoszący **338,82 ms** (w tym 54,6 ms na rekalkulację wektorów, 108,9 ms na przebudowę indeksu BM25, 138,9 ms na insercję do `pgvector` oraz 4,9 ms na rekonstrukcję grafu `NetworkX`). W trakcie migracji mediana opóźnień zapytań w locie wzrosła

przejściowo z 9,93 ms do 25,81 ms (z pikami p_{95} sięgającymi 71,38 ms). W środowiskach produkcyjnych o wysokiej dynamice modyfikacji DDL wymaga to wdrożenia asynchronicznego wersjonowania schematu;

3. **Zależność generowania zapytań SQL od pojemności logicznej modelu językowego:** Optymalizacja podgrafu DDL w silniku JIT Schema Discovery dostarcza modelowi zwięzły, spójny semantycznie wycinek relacji, lecz nie jest w stanie skompensować ograniczeń samego silnika wnioskowania. Badania w architekturze Dual-Engine dowiodły, że modele małe lub ograniczone restrykcyjnym limitem tokenów wyjściowych (Profil A: `max_tokens = 120`) doznają załamania dokładności wykonawczej (EX = 10,8%, Valid SQL = 12,0%) na skutek ucinania generowanego kodu (`TRUNCATION` w 30,0% przypadków), co prowadzi do wtórnych błędów składniowych. Skuteczna generacja zaawansowanych kwerend Text-to-SQL wymaga stosowania modeli o odpowiedniej pojemności logicznej z oknem generacji wynoszącym co najmniej 1024 tokeny;

4. **Ograniczenia analizy gramatycznej AST i wektory zagrożeń parsera:** Deterministyczna weryfikacja drzewa AST w bibliotece `sqlglot` skutecznie blokuje ataki strukturalne (DDL, DML, zapytania piętrowe), lecz ujawnia istotne granice ochronne i podatności implementacyjne:
   - *Podatność na ataki wnioskowaniowe:* Syntaktycznie poprawna kwerenda `SELECT` za pomocą precyzyjnych filtrów statystycznych lub zapytań różnicowych może doprowadzić do nieuprawnionego wnioskowania o danych poufnych, bez formalnego naruszenia reguł gramatycznych zapory;
   - *Nadużycie funkcji wbudowanych w dopuszczalnych kwerendach SELECT:* Parser AST weryfikuje regułę czystego odczytu `exp.Select`, co pozwala na egzekucję funkcji systemowych silnika PostgreSQL o charakterze sondującym lub blokującym zasoby (np. `SELECT pg_sleep(10)` prowadzące do wyczerpania połączeń w puli bazy, czy próby odczytu funkcji administracyjnych i katalogu `pg_shadow`). Wskazuje to na konieczność uzupełnienia analizatora AST o restrykcyjną listę dopuszczalnych funkcji analitycznych;
   - *Ataki na złożoność algorytmiczną parsera (Adversarial AST / ReDoS):* Silnik `sqlglot` wykazuje podatność na kwerendy o patologicznej strukturze gramatycznej (np. $10\,000$ poziomów zagnieżdżeń nawiasowych), które mogą wywołać przepełnienie stosu lub zawieszenie procesu roboczego w puli `ProcessPoolExecutor`. Mitygacja tego wektora wymaga zdefiniowania sztywnego limitu głębokości drzewa składniowego na etapie tokenizacji leksykalnej.

5. **Zależność wydajności paginacji keyset od postaci predykatu i indeksu:** Stałe opóźnienie keyset (0,43 ms przy offsecie 950 000, podrozdział **??**) jest warunkowe. Wymaga kompozytowego indeksu B-drzewa o identycznych kierunkach i polityce `NULLS` oraz predykatu rozbitego na rozłączne gałęzie `UNION ALL`; pojedyncza klauzula DNF z operatorami `OR` dawała koszt $\mathcal{O}(\mathrm{M})$ porównywalny z OFFSET/LIMIT (podrozdział **??**). Dekompozycja zwielokrotnia liczbę

zapytań indeksowych na stronę (do $2\mathrm{m}+1$), a sama metoda nie umożliwia skoku do dowolnej strony. Pomiar oparto na trzech powtórzeniach na punkt i jednej instancji bazy, a wariant nieindeksowany zmierzono tylko do offsetu 50 000, dlatego ogon rozkładu opóźnień wymaga dalszych badań na większej próbie.

7.5 Perspektywy rozwoju i kierunki dalszych prac badawczych

Uzyskane wyniki badawcze stanowią fundament pod sformułowanie konkretnych postulatów rozwojowych dla społeczności inżynierskiej oraz grup standaryzacyjnych:

1. **Standaryzacja natywnego transportu binarnego w specyfikacji MCP:** Kluczowym postulatem dla kolejnych wersji standardu MCP jest wdrożenie alternatywnej warstwy transportowej opartej na *Apache Arrow Flight* lub *gRPC / Protocol Buffers* [12, 43]. Umożliwiłoby to transfer zmapowanych buforów pamięci w trybie bezkopiowym pomiędzy składem danych a agentem, całkowicie eliminując narzut parsowania tekstu JSON-RPC dla ładunków tabelarycznych;

2. **Semantyczna orkiestracja transakcji kompensacyjnych SAGA i wzorzec RAC:** Odpowiedzią na niemożność stosowania blokad dwufazowych w systemach agentowych jest adaptacja klasycznego wzorca transakcji kompensacyjnych SAGA (Garcia-Molina i Salem, 1987 [**garcia1987sagas**]). W najnowszych badaniach nad odpornością systemów LLM Chang i Geng (2025) zaproponowali framework SagaLLM [**chang2025sagallm**], przenoszący wzorzec SAGA do systemów wieloagentowych poprzez dekompozycję procesu na sekwencję kroków lokalnych $\mathrm{T}_1, \ldots, \mathrm{T}_\mathrm{n}$ z odpowiadającymi im akcjami kompensującymi C_i. Niemniej jednak SagaLLM powierza dynamiczne generowanie kodu zapytań kompensujących w locie samemu modelowi językowemu. Prowadzi to do poważnego ryzyka halucynacji procedury naprawczej – błąd w wygenerowanym predykacie kwerendy odwracającej (np. błędna klauzula `WHERE` w poleceniu `UPDATE` lub pominięcie klucza obcego) nie tylko nie cofa skutków awarii, lecz wprowadza wtórną, kaskadową niespójność danych.

   Wzorcowym rozwiązaniem dla przyszłych rewizji protokołu MCP jest architektura *Robust Agent Compensation* (RAC), sformułowana przez Pererę i in. (ACM AI and Agentic Systems 2026) [**perera2026rac**]. W architekturze RAC odpowiedzialność za wycofanie zmian zostaje całkowicie zdjęta ze stochastycznego modelu i przeniesiona do warstwy pośredniczącej middleware. Pary operacji kompensujących oraz deterministyczne reguły mapowania ich argumentów są deklarowane wprost w definicjach narzędzi serwera MCP przez inżynierów oprogramowania na etapie projektowym. Dedykowany moduł przechwytujący bramy pośredniczącej rejestruje idempotentną operację kompensującą w trwałym dzienniku zdarzeń przed wykonaniem akcji właściwej w bazie. W razie wystąpienia błędu wnioskowania, ucięcia kontekstu lub awarii węzła, middleware automatycznie i deterministycznie wykonuje sekwencję kompensacji w porządku odwrotnym bez jakiegokolwiek udziału modelu językowego;

3. **Delegacja tożsamości i autoryzacja drobnoziarnista (OAuth 2.0 / RFC 8693):** W celu wyeliminowania ryzyka podatności standard MCP powinien integrować mechanizm wymiany tokenów tożsamości. Przekazywanie tożsamości operatora w nagłówkach sesji MCP umożliwi silnikowi relacyjnemu egzekwowanie uprawnień na poziomie wierszy w sposób przezroczysty dla modelu;

4. **Konwergencja integracji bazodanowej z protokołami orkiestracji wieloagentowej (A2A i ACP):** Interesującym obszarem dalszych prac jest sprzężenie pionowej integracji bazodanowej MCP z horyzontalnymi protokołami komunikacji międzyagentowej (Agent-to-Agent / A2A, ACP) [8]. W takiej architekturze brama pośrednicząca MCP staje się wspólną, inteligentną magistralą danych, zasilającą roje wyspecjalizowanych agentów bez multiplikowania fizycznych sesji bazodanowych.

7.6 Wpływ protokołu MCP na ewolucję systemów Big Data i inżynierii oprogramowania

Przeprowadzone badania wskazują na początek istotnej reorientacji architektonicznej w inżynierii systemów bazodanowych i oprogramowania klasy korporacyjnej.

Tradycyjna inżynieria danych przez dekady opierała się na paradygmacie pasywnym: hurtownie danych, hurtownie tematyczne oraz jeziora danych funkcjonowały jako statyczne repozytoria, odpytywane okresowo przez wsadowe potoki ETL/ELT lub analityków biznesowych za pośrednictwem zdefiniowanych pulpitów nawigacyjnych BI. Pojawienie się otwartego standardu Model Context Protocol w połączeniu z autonomicznymi modelami agentowymi inicjuje przejście w kierunku aktywnych ekosystemów danych sterowanych pętlami wnioskowania.

W nowym modelu dane stają się dynamicznym środowiskiem operacyjnym, w którym autonomiczne agenty wnioskują, planują i realizują wieloetapowe procedury eksploracyjne w czasie rzeczywistym. Dzięki otwartej standaryzacji interfejsu narzędziowego zniknęła konieczność tworzenia dedykowanych, trudnych w utrzymaniu skryptów integrujących, a złożoność integracji uległa redukcji z kombinatorycznej postaci $\mathcal{O}(\mathrm{M} \times \mathrm{N})$ do modularnej postaci $\mathcal{O}(\mathrm{M} + \mathrm{N})$.

Sformalizowana i zweryfikowana empirycznie w niniejszej pracy architektura referencyjnej bramy pośredniczącej dowiodła, że możliwe jest bezpieczne, deterministyczne i wysoko wydajne pogodzenie elastycznego wnioskowania probabilistycznego modeli językowych z rygorystycznymi gwarancjami przemysłowych relacyjnych składów danych. Uzyskane wyniki stanowią stabilny fundament pod projektowanie i wdrażanie nowej generacji autonomicznych platform analitycznych w inżynierii oprogramowania.

Bibliografia

[1] Anthropic, *Model Context Protocol Specification*, <https://modelcontextprotocol.io>, Dostęp uzyskano: 2024-11-25, 2024.

[2] Barabási, A.-L. i Albert, R., „Emergence of Scaling in Random Networks", *Science*, t. 286, nr 5439, s. 509–512, 1999, doi: 10.1126/science.286.5439.509.

[3] Belshe, M., Peon, R. i Thomson, M., „Hypertext Transfer Protocol Version 2 (HTTP/2)", RFC Editor, RFC 7540, maj 2015, doi: 10.17487/RFC7540.

[4] Bray, T., „The JavaScript Object Notation (JSON) Data Interchange Format", Internet Engineering Task Force (IETF), RFC 8259, grudzień 2017, doi: 10.17487/RFC8259.

[5] Brewer, E. A., „Towards robust distributed systems (abstract)", w: *Proceedings of the nineteenth annual ACM symposium on Principles of distributed computing (PODC '00)*, doi: 10.1145/343477.343502, 2000, s. 7.

[6] Brown, T. B. i in., „Language Models are Few-Shot Learners", w: *Advances in Neural Information Processing Systems (NeurIPS 2020)*, t. 33, 2020, s. 1877–1901.

[7] Chase, H., *LangChain: Building applications with LLMs through composability*, <https://github.com/langchain-ai/langchain>, 2022.

[8] Chen, Z., Zhao, Y., Li, H., Wang, J. i An, B., „A Survey of Agent Interoperability Protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A)", *arXiv preprint arXiv:2505.02279*, 2025, doi: 10.48550/arXiv.2505.02279.

[9] Cormack, G. V., Clarke, C. L. A. i Buettcher, S., „Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods", w: *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '09)*, doi: 10.1145/1571941.1572114, 2009, s. 758–759.

[10] Fielding, R. T., „Architectural Styles and the Design of Network-based Software Architectures", rozprawa doktorska, University of California, Irvine, 2000.

[11] Gilbert, S. i Lynch, N., „Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services", *ACM SIGACT News*, t. 33, nr 2, s. 51–59, 2002, doi: 10.1145/564585.564601.

[12] Google LLC, *Protocol Buffers: Language-Neutral, Platform-Neutral Extensible Mechanisms for Serializing Structured Data*, <https://protobuf.dev/>, Dostęp uzyskano: 2024-11-15, 2024.

[13] Gray, J. N., „Notes on Data Base Operating Systems", w: *Operating Systems, An Advanced Course*, seria Lecture Notes in Computer Science, t. 60, doi: 10.1007/3-540-08755-9_9, Springer-Verlag Berlin Heidelberg, 1978, s. 393–481.

[14] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T. i Fritz, M., „Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection", w: *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISEC '23)*, doi: 10.1145/3605764.3623982, 2023, s. 79–90.

[15] Haerder, T. i Reuter, A., „Principles of Transaction-Oriented Database Recovery", *ACM Computing Surveys*, t. 15, nr 4, s. 287–317, 1983, doi: 10.1145/289.291.

[16] Hardy, N., „The Confused Deputy: (or why I know how to stop the hackers)", *ACM SIGOPS Operating Systems Review*, t. 22, nr 4, s. 36–38, 1988, doi: 10.1145/54289.848247.

[17] Hickson, I., „Server-Sent Events", World Wide Web Consortium (W3C), W3C Recommendation, luty 2015, <https://www.w3.org/TR/eventsource/>.

[18] JSON-RPC Working Group, *JSON-RPC 2.0 Specification*, <https://www.jsonrpc.org/specification>, Dostęp uzyskano: 2024-11-20, 2010.

[19] JSON-RPC Working Group, *JSON-RPC 2.0 Specification*, <https://www.jsonrpc.org/specification>, Dostęp uzyskano: 2024-11-20, 2010.

[20] Kane, A., *pgvector: Open-source vector similarity search for Postgres*, <https://github.com/pgvector/pgvector>, 2024.

[21] Karpukhin, V. i in., „Dense Passage Retrieval for Open-Domain Question Answering", w: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, s. 6769–6781.

[22] Kirkpatrick, J. i in., „Overcoming catastrophic forgetting in neural networks", *Proceedings of the National Academy of Sciences (PNAS)*, t. 114, nr 13, s. 3521–3526, 2017, doi: 10.1073/pnas.1611835114.

[23] Kreps, J., Narkhede, N. i Rao, J., „Kafka: A distributed messaging system for log processing", *Proceedings of the NetDB*, t. 11, s. 1–7, 2011.

[24] Laney, D., „3D Data Management: Controlling Data Volume, Velocity, and Variety", META Group (Gartner), Application Delivery Strategies File 949, luty 2001.

[25] Lewis, P. i in., „Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", w: *Advances in Neural Information Processing Systems (NeurIPS 2020)*, t. 33, 2020, s. 9459–9474.

[26] Liu, N. F. i in., „Lost in the Middle: How Language Models Use Long Contexts", *Transactions of the Association for Computational Linguistics (TACL)*, t. 12, s. 157–173, 2024, doi: 10.1162/tacl_a_00638.

[27] Lowin, J., *FastMCP: High-level Python Framework for Model Context Protocol*, <https://github.com/jlowin/fastmcp>, 2024.

[28] MagicStack Inc., *asyncpg: A fast PostgreSQL Database Client Library for Python/asyncio*, <https://github.com/MagicStack/asyncpg>, 2024.

[29] Malkov, Y. A. i Yashunin, D. A., „Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, t. 42, nr 4, s. 824–836, 2020, doi: 10.1109/TPAMI.2018.2889473.

[30] Mao, T., *sqlglot: An easily customizable SQL parser and transpiler*, <https://github.com/tobymao/sqlglot>, 2024.

[31] Mialon, G. i in., „Augmented Language Models: a Survey", *arXiv preprint arXiv:2302.07842*, 2023.

[32] Microsoft Corporation, *Semantic Kernel: Integrate cutting-edge LLM technology into your apps*, <https://github.com/microsoft/semantic-kernel>, 2023.

[33] Microsoft Corporation, *Language Server Protocol Specification - 3.17*, <https://microsoft.github.io/language-server-protocol/>, Dostęp uzyskano: 2024-11-15, 2024.

[34] Novik, N., *aiokafka: asyncio client for Apache Kafka*, <https://github.com/aio-libs/aiokafka>, 2024.

[35] OpenAI, *Function calling and other API updates*, <https://openai.com/index/function-calling--and-other-api-updates/>, Opublikowano: 2023-06-13, 2023.

[36] OpenAI, *tiktoken: Fast BPE tokeniser for use with OpenAI's models*, <https://github.com/openai/tiktoken>, 2023.

[37] OWASP Foundation, „OWASP Top 10 for Large Language Model Applications and Generative AI", Open Web Application Security Project, Technical Report, 2025, <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.

[38] Patil, S. G., Zhang, T., Wang, X. i Gonzalez, J. E., „Gorilla: Large Language Model Connected with Massive APIs", *arXiv preprint arXiv:2305.15334*, 2023, doi: 10.48550/arXiv.2305.15334.

[39] Pritchett, D., „BASE: An ACID Alternative", *ACM Queue*, t. 6, nr 3, s. 48–55, 2008, doi: 10.1145/1394127.1394128.

[40] Qin, Y. i in., „Tool Learning with Foundation Models", *arXiv preprint arXiv:2304.08354*, 2023.

[41] Reimers, N. i Gurevych, I., „Sentence-BERT: Sentence Embeddings using Siamese BERT--Networks", *arXiv preprint arXiv:1908.10084*, 2019.

[42] Schick, T. i in., „Toolformer: Language Models Can Teach Themselves to Use Tools", w: *Advances in Neural Information Processing Systems (NeurIPS 2023)*, doi: 10.48550/arXiv.2302.04761, t. 36, 2023, s. 68 537–68 551.

[43] The gRPC Authors, *gRPC: A High Performance, Open Source Universal RPC Framework*, <https://grpc.io/docs/what-is-grpc/introduction/>, Dostęp uzyskano: 2024-11-15, 2023.

[44] Vaswani, A. i in., „Attention Is All You Need", w: *Advances in Neural Information Processing Systems (NIPS 2017)*, t. 30, 2017, s. 5998–6008.

[45] Welford, B. P., „Note on a Method for Calculating Corrected Sums of Squares and Products", *Technometrics*, t. 4, nr 3, s. 419–420, 1962, doi: 10.1080/00401706.1962.10490022.

[46] Wilcoxon, F., „Individual Comparisons by Ranking Methods", *Biometrics Bulletin*, t. 1, nr 6, s. 80–83, 1945, doi: 10.2307/3001968.

[47] Wright, A., Andrews, H., Hutton, B. i Dennis, G., „JSON Schema: A Media Type for Describing JSON Data Structures", Internet Engineering Task Force (IETF), Internet-Draft, 2022, draft-bhutton-json-schema-01.

[48] Yao, S. i in., „ReAct: Synergizing Reasoning and Acting in Language Models", w: *International Conference on Learning Representations (ICLR)*, 2023.

Wykaz skrótów i symboli

A. Wykaz Akronimów i Skrótów Technicznych

API	*Application Programming Interface* (Interfejs Programowania Aplikacji)
AST	*Abstract Syntax Tree* (Abstrakcyjne Drzewo Składniowe)
BPE	*Byte Pair Encoding* (Algorytm tokenizacji podsłów)
CDSS	*Clinical Decision Support System* (System Wspomagania Decyzji Klinicznych)
CTE	*Common Table Expression* (Nazwane wyrażenie tablicowe SQL)
DDL	*Data Definition Language* (Język Definicji Danych)
DML	*Data Manipulation Language* (Język Manipulacji Danymi)
DSR	*Design Science Research* (Metodologia badań konstrukcyjnych w informatyce)
E2E / $\mathrm{L}_{\mathbf{E2E}}$	*End-to-End Latency* (Całkowite opóźnienie realizacji zapytania)
EX	*Execution Accuracy* (Dokładność wykonawcza kodu Text-to-SQL)
FK	*Foreign Key* (Klucz obcy w relacyjnej bazie danych)
HNSW	*Hierarchical Navigable Small World* (Grafowy indeks wektorowy k-NN)
IPC	*Inter-Process Communication* (Komunikacja międzyprocesowa)
JSON-RPC	*JavaScript Object Notation Remote Procedure Call* (Protokół zdalnego wywoływania procedur)
LLM	*Large Language Model* (Wielki Model Językowy)
LSP	*Language Server Protocol* (Protokół serwera języka programowania)
MCP	*Model Context Protocol* (Otwarty standard komunikacji modeli agentowych z kontekstem)
MVCC	*Multi-Version Concurrency Control* (Wielowersyjna kontrola współbieżności)
OOB	*Out-of-Band* (Transmisja obiektów poza głównym pasmem sterującym)
POSIX SHM	*POSIX Shared Memory* (Współdzielony segment pamięci operacyjnej `/dev/shm`)

RAG	*Retrieval-Augmented Generation* (Generowanie wspomagane wyszukiwaniem)
RPS	*Requests Per Second* (Przepustowość transakcyjna systemu)
RRF	*Reciprocal Rank Fusion* (Algorytm fuzji rang w wyszukiwaniu hybrydowym)
RTT	*Round-Trip Time* (Dwukierunkowe opóźnienie propagacji pakietu sieciowego)
SSE	*Server-Sent Events* (Jednokierunkowy strumień zdarzeń serwera HTTP)
TTFT	*Time-to-First-Token* (Czas do wygenerowania pierwszego tokenu odpowiedzi)
URI	*Uniform Resource Identifier* (Ujednolicony identyfikator zasobu)
XAI	*Explainable Artificial Intelligence* (Wyjaśnialna Sztuczna Inteligencja)

B. Wykaz Symboli i Oznaczeń Matematycznych

$\mathrm{B}_{\textbf{tok}}$	Konfigurowalny budżet tokenów dla pojedynczej kwerendy
$\mathrm{B}_{\textbf{net}}, \mathrm{B}_{\textbf{shm}}$	Przepustowość odpowiednio łącza sieciowego oraz magistrali pamięci współdzielonej
d	Wymiarowość przestrzeni osadzeń wektorowych ($\mathrm{d} = 384$ dla modelu `all-MiniLM-L6-v2`)
$\delta_{\textbf{OOB}}$	Stały narzut przygotowania transferu poza pasmem (alokacja deskryptora, skrót kryptograficzny)
K	Rozmiar strony pobieranej w procedurze paginacji kursorowej (liczba wierszy)
$\mathrm{k}_{\textbf{RRF}}$	Stała stabilizująca fuzję rang Reciprocal Rank Fusion ($\mathrm{k}_{\mathrm{RRF}} = 60$)
M	Liczba współbieżnych procesów agentowych / wątków klienckich
$\mathrm{M}_{\textbf{exp}}$	Maksymalna dopuszczalna liczba wstrzykiwanych tabel mostkujących ($\mathrm{M}_{\mathrm{exp}} = 3$)
$\mu_{\mathrm{n}}, \sigma_{\mathrm{n}}$	Bieżąca estymacja średniej oraz odchylenia standardowego liczby tokenów na rekord w algorytmie Welforda
N	Liczebność zbioru ewaluacyjnego Text-to-SQL ($\mathrm{N} = 500$) lub liczba rekordów w tabeli analitycznej ($\mathrm{N} = 1\ 000\ 000$)
S	Rozmiar ładunku binarnego w bajtach
$\mathrm{S}^{*}(\textbf{RTT})$	Punkt równowagi (próg opłacalności) transferu Out-of-Band w funkcji opóźnienia sieciowego
$\mathrm{W}, \mathrm{p}, \mathrm{r}$	Statystyka testowa, poziom istotności oraz standaryzowana wielkość efektu w teście sumy rangowych znaków Wilcoxona

Spis rysunków

Spis tabel

Spis załączników

1 Kod źródłowy implementacji referencyjnej oraz środowiska badawczego 111

Załącznik 1

Kod źródłowy implementacji referencyjnej oraz środowiska badawczego

Integralną część niniejszej pracy magisterskiej stanowi załącznik w postaci skompresowanego archiwum ZIP (`mcp-master-thesis-source.zip`), zawierający kompletny kod źródłowy opracowanego systemu, środowisko badawcze oraz skrypty powtarzalnych eksperymentów ewaluacyjnych.

Struktura zawartości archiwum

Poniżej przedstawiono logiczną strukturę katalogów i plików wchodzących w skład dołączonego pakietu programistycznego:

- `src/mcp_server/` – kod źródłowy autorskiej bramy pośredniczącej protokołu Model Context Protocol:
  - `server.py` – główny moduł serwera FastMCP pod Pythonem 3.12, rejestracja narzędzi analitycznych, zasobów kontekstowych oraz obsługa cyklu życia sesji;
  - `schema_engine.py` – silnik progresywnego odkrywania schematu (JIT Schema Discovery) z wykorzystaniem wektorowej bazy PostgreSQL/pgvector;
  - `pagination.py` – algorytm adaptacyjnej paginacji kursorowej (Smart Pagination) z dynamiczną estymacją budżetu tokenów (tiktoken BPE);
  - `streaming.py` – moduł asynchronicznego strumieniowania zdarzeń telemetrycznych Server-Sent Events (SSE) powiązany z brokerem Apache Kafka;
  - `security.py` – wielowarstwowa zapora bezpieczeństwa zapytań SQL Sanitizer oparta na weryfikacji drzew składniowych AST (SQLGlot).
- `testbed/` – definicje infrastruktury badawczej i zasilania danymi:
  - `docker-compose.yml` – plik orkiestracji kontenerów PostgreSQL 16 (z rozszerzeniem `pgvector`) oraz Apache Kafka w bezstanowym trybie konsensusu KRaft;
  - `init-scripts/` – skrypty DDL oraz generatory syntetycznych struktur hurtowni danych (do 1000 relacji) oraz wolumenu 1 miliona rekordów analitycznych.
- `benchmarks/` – aparatura pomiarowa i narzędzia ewaluacyjne:
  - `harness.py` – modularny silnik testowy realizujący automatyczne, wielopróbkowe pomiary czasów TTFT, latencji E2E, zużycia pamięci i tokenów;
  - `scenarios/` – implementacje czterech scenariuszy pomiarowych (Scenariusz A: skalowalność schematu, Scenariusz B: paginacja i Out-of-Band, Scenariusz C: asynchroniczne strumieniowanie SSE, Scenariusz D: odporność na ataki Prompt Injection);
  - `plot_generators/` – zautomatyzowane skrypty generujące wykresy wektorowe PDF prezentowane w rozdziale badawczym.

- `doc/` – kompletne pliki źródłowe tekstu pracy w systemie LaTeX, arkusze stylów `EE-dyplom.cls`, pliki bibliograficzne BibLaTeX oraz materiały graficzne;
- `pyproject.toml` / `requirements.txt` – specyfikacja zależności bibliotecznych środowiska Python 3.12;
- `README.md` – szczegółowa instrukcja techniczna dotycząca instalacji, konfiguracji środowiska wirtualnego oraz procedury reprodukcji wyników eksperymentalnych.

Wymagania systemowe i instrukcja uruchomienia

W celu odtworzenia środowiska badawczego oraz uruchomienia harnessu pomiarowego wymagane są następujące komponenty programowe:

- System operacyjny klasy Linux (rekomendowana dystrybucja Ubuntu 22.04 LTS lub nowsza);
- Środowisko Python w wersji 3.12 lub wyższej;
- Środowisko kontenerowe Docker Engine 24.0+ oraz Docker Compose v2.20+;
- Minimum 16 GB pamięci operacyjnej RAM oraz 20 GB wolnej przestrzeni dyskowej na wolumeny bazodanowe.

Procedura uruchomienia krok po kroku:

1. Rozpakowanie archiwum do wybranego katalogu roboczego:

```
unzip mcp-master-thesis-source.zip -d mcp-thesis
```

2. Utworzenie i aktywacja dedykowanego środowiska wirtualnego:

```
python3.12 -m venv .venv && source .venv/bin/activate
```

3. Instalacja wymaganych bibliotek i zależności środowiskowych:

```
pip install -r requirements.txt
```

4. Uruchomienie kontenerów bazodanowych PostgreSQL i brokera Kafka:

```
docker compose -f testbed/docker-compose.yml up -d
```

5. Generowanie danych syntetycznych oraz wektorowych indeksów HNSW:

```
python testbed/init-scripts/seed_1m_records.py
```

6. Uruchomienie pełnego pakietu scenariuszy benchmarkowych:

```
python benchmarks/harness.py --scenario all
```